

```

# ==== main.py (44行) ====
import sys
from PyQt6.QtWidgets import QApplication, QStackedWidget
from PyQt6.QtCore import Qt
from core.constants import APP_NAME
from core.styles import GLOBAL_STYLE
from ui.login_window import LoginWindow
from ui.main_window import MainWindow
class Application(QStackedWidget):
    def __init__(self):
        super().__init__()
        self._login = LoginWindow()
        self._main: MainWindow | None = None
        self.addWidget(self._login)
        self._login.login_success.connect(self._on_login)
        self.setWindowTitle(APP_NAME)
        self.resize(1100, 680)
    def _on_login(self, username: str) -> None:
        self._main = MainWindow(username)
        self.addWidget(self._main)
        self.setCurrentWidget(self._main)
        self.resize(self._main.size())
        self.setWindowTitle(self._main.windowTitle())
def main() -> int:
    QApplication.setHighDpiScaleFactorRoundingPolicy(
        Qt.HighDpiScaleFactorRoundingPolicy.PassThrough
    )
    app = QApplication(sys.argv)
    app.setApplicationName(APP_NAME)
    app.setStyle("Fusion")
    app.setStyleSheet(GLOBAL_STYLE)
    window = Application()
    window.show()
    return app.exec()
if __name__ == "__main__":
    sys.exit(main())

# ==== core/__init__.py (0行) ====

# ==== core/styles.py (185行) ====
BG_DARK = "#0b1426"
BG_PANEL = "#121e3a"
BG_CARD = "#1a2a4e"
BG_INPUT = "#1f3060"
ACCENT = "#4fc3f7"
ACCENT_DIM = "#29b6f6"
ACCENT_GLOW = "#81d4fa"
TEXT_PRIMARY = "#e8edf5"
TEXT_SECONDARY = "#8fa8c8"
BORDER = "#2a4070"
WARN = "#ffab40"
DANGER = "#ff5252"
GLOBAL_STYLE = f"""
QMainWindow, QWidget {{
    background-color: {BG_DARK};

```

```

        color: {TEXT_PRIMARY};
        font-family: "Microsoft YaHei", "Segoe UI", sans-serif;
        font-size: 13px;
    }}
    QPushButton {{
        background-color: {BG_CARD};
        color: {TEXT_PRIMARY};
        border: 1px solid {BORDER};
        border-radius: 6px;
        padding: 8px 14px;
        min-height: 28px;
    }}
    QLabel {{
        background: transparent;
        color: {TEXT_PRIMARY};
        border: none;
    }}
    QLineEdit, QComboBox, QSpinBox, QDoubleSpinBox, QTextEdit, QPlainTextEdit {{
        background-color: {BG_INPUT};
        color: {TEXT_PRIMARY};
        border: 1px solid {BORDER};
        border-radius: 6px;
        padding: 8px 12px;
        selection-background-color: {ACCENT_DIM};
    }}
    QLineEdit:focus, QComboBox:focus, QSpinBox:focus, QDoubleSpinBox:focus {{
        border: 1px solid {ACCENT};
    }}
    QPushButton:hover {{
        background-color: #1f3a6a;
        border-color: {ACCENT_DIM};
    }}
    QPushButton:pressed {{
        background-color: #1565c0;
        color: #ffffff;
    }}
    QPushButton[primary="true"] {{
        background: qlineargradient(x1:0,y1:0,x2:1,y2:0, stop:0 #1565c0, stop:1 #1e88e5);
        color: #ffffff;
        border: 1px solid #42a5f5;
        font-weight: bold;
        font-size: 13px;
    }}
    QPushButton[primary="true"]:hover {{
        background: qlineargradient(x1:0,y1:0,x2:1,y2:0, stop:0 #1e88e5, stop:1 #42a5f5);
        color: #ffffff;
        border-color: #64b5f6;
    }}
    QPushButton[primary="true"]:pressed {{
        background: #0d47a1;
        color: #ffffff;
    }}
    QPushButton[danger="true"] {{
        background-color: #4a1520;

```

```

        border: 1px solid {DANGER};
        color: #ffffff;
        font-weight: bold;
    }}
QPushButton[danger="true"]:hover {{
    background-color: #6a2030;
    color: #ffffff;
}}
QTableWidget {{
    background-color: {BG_PANEL};
    alternate-background-color: {BG_PANEL};
    color: {TEXT_PRIMARY};
    gridline-color: {BORDER};
    border: 1px solid {BORDER};
    border-radius: 8px;
    selection-background-color: #1a3a6a;
    selection-color: {TEXT_PRIMARY};
}}
QTableWidget::item {{
    padding: 6px;
    border: none;
}}
QHeaderView::section {{
    background-color: {BG_CARD};
    color: {ACCENT};
    border: none;
    border-bottom: 1px solid {BORDER};
    border-right: 1px solid {BORDER};
    padding: 10px 8px;
    font-weight: bold;
}}
QHeaderView::section:last {{
    border-right: none;
}}
QTableCornerButton::section {{
    background-color: {BG_CARD};
    border: none;
}}
QScrollBar:vertical {{
    background: {BG_PANEL};
    width: 10px;
    border-radius: 5px;
}}
QScrollBar::handle:vertical {{
    background: {BORDER};
    border-radius: 5px;
    min-height: 30px;
}}
QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical {{
    height: 0;
}}
QTabWidget::pane {{
    border: 1px solid {BORDER};
    border-radius: 8px;

```

```

        background: {BG_PANEL};
    }}
    QTabBar::tab {{
        background: {BG_CARD};
        color: {TEXT_SECONDARY};
        padding: 10px 20px;
        border-top-left-radius: 6px;
        border-top-right-radius: 6px;
        margin-right: 2px;
    }}
    QTabBar::tab:selected {{
        background: {BG_PANEL};
        color: {ACCENT};
        border-bottom: 2px solid {ACCENT};
    }}
    QGroupBox {{
        border: 1px solid {BORDER};
        border-radius: 8px;
        margin-top: 14px;
        padding-top: 18px;
        background-color: {BG_PANEL};
        font-weight: bold;
        color: {ACCENT};
    }}
    QGroupBox::title {{
        subcontrol-origin: margin;
        left: 14px;
        padding: 0 8px;
        background-color: {BG_PANEL};
    }}
    QProgressBar {{
        border: 1px solid {BORDER};
        border-radius: 6px;
        background: {BG_INPUT};
        text-align: center;
        color: {TEXT_PRIMARY};
        height: 22px;
    }}
    QProgressBar::chunk {{
        background: qlineargradient(x1:0,y1:0,x2:1,y2:0, stop:0 {ACCENT_DIM}, stop:1 {ACCENT});
        border-radius: 5px;
    }}
    QSlider::groove:horizontal {{
        height: 6px;
        background: {BG_INPUT};
        border-radius: 3px;
    }}
    QSlider::handle:horizontal {{
        width: 16px;
        margin: -5px 0;
        background: {ACCENT};
        border-radius: 8px;
    }}
    QToolTip {{

```

```

        background-color: {BG_CARD};
        color: {TEXT_PRIMARY};
        border: 1px solid {ACCENT_DIM};
        padding: 6px;
        border-radius: 4px;
    }}
    """

```

===== core/constants.py (8行) =====

```

APP_NAME = "信息系统运行日志智能分析与故障预测及根因定位平台"
APP_VERSION = "1.0"
DEFAULT_USERNAME = "admin"
DEFAULT_PASSWORD = "123456"
WINDOW_MIN_WIDTH = 1280
WINDOW_MIN_HEIGHT = 720
DEFAULT_WIDTH = 1463
DEFAULT_HEIGHT = 851

```

===== core/auth.py (58行) =====

```

import json
import hashlib
from pathlib import Path
class AuthManager:
    def __init__(self) -> None:
        self._path = Path.home() / ".sv200x_agc_avc" / "users.json"
        self._path.parent.mkdir(parents=True, exist_ok=True)
        self._users: dict[str, str] = {}
        self._load()
    def _hash(self, password: str) -> str:
        return hashlib.sha256(password.encode("utf-8")).hexdigest()
    def _load(self) -> None:
        if self._path.exists():
            try:
                data = json.loads(self._path.read_text(encoding="utf-8"))
                self._users = {k: v for k, v in data.items() if isinstance(v, str)}
            except (json.JSONDecodeError, OSError):
                self._users = {}
        if "admin" not in self._users:
            self._users["admin"] = self._hash("123456")
            self._save()
    def _save(self) -> None:
        self._path.write_text(
            json.dumps(self._users, ensure_ascii=False, indent=2),
            encoding="utf-8",
        )
    def login(self, username: str, password: str) -> tuple[bool, str]:
        username = username.strip()
        if not username or not password:
            return False, "账号和密码不能为空"
        stored = self._users.get(username)
        if stored is None:
            return False, "账号不存在"
        if stored != self._hash(password):
            return False, "密码错误"

```

```

        return True, "登录成功"
def register(self, username: str, password: str, confirm: str) -> tuple[bool, str]:
    username = username.strip()
    if not username or not password:
        return False, "账号和密码不能为空"
    if len(username) < 3:
        return False, "账号长度至少3个字符"
    if len(password) < 6:
        return False, "密码长度至少6个字符"
    if password != confirm:
        return False, "两次输入的密码不一致"
    if username in self._users:
        return False, "该账号已存在"
    self._users[username] = self._hash(password)
    self._save()
    return True, "注册成功, 请登录"

# ==== core/dialogs.py (217行) ====
from PyQt6.QtCore import Qt
from PyQt6.QtGui import QFont
from PyQt6.QtWidgets import (
    QDialog, QVBoxLayout, QHBoxLayout, QLabel, QPushButton, QTextEdit, QFrame,
)
from core.styles import BG_PANEL, ACCENT, TEXT_PRIMARY, TEXT_SECONDARY, BORDER,
DANGER, WARN
_BTN_PRIMARY = """
QPushButton {
    background: qlineargradient(x1:0,y1:0,x2:1,y2:0, stop:0 #1565c0, stop:1 #1e88e5);
    color: #ffffff;
    border: 1px solid #42a5f5;
    border-radius: 6px;
    padding: 8px 20px;
    font-size: 13px;
    font-weight: bold;
    min-height: 32px;
}
QPushButton:hover {
    background: qlineargradient(x1:0,y1:0,x2:1,y2:0, stop:0 #1e88e5, stop:1 #42a5f5);
    color: #ffffff;
}
QPushButton:pressed {
    background: #0d47a1;
    color: #ffffff;
}
"""
_BTN_SECONDARY = """
QPushButton {
    background-color: #1f3060;
    color: #e8edf5;
    border: 1px solid #2a4070;
    border-radius: 6px;
    padding: 8px 20px;

```

```

        font-size: 13px;
        min-height: 32px;
    }
    QPushButton:hover {
        background-color: #25366a;
        border-color: #4fc3f7;
        color: #ffffff;
    }
}

_BTN_DANGER = ""
QPushButton {
    background-color: #8b2030;
    color: #ffffff;
    border: 1px solid #ff5252;
    border-radius: 6px;
    padding: 8px 20px;
    font-size: 13px;
    font-weight: bold;
    min-height: 32px;
}
QPushButton:hover {
    background-color: #a82838;
    color: #ffffff;
}

def _style_button(btn: QPushButton, kind: str = "primary") -> None:
    styles = {"primary": _BTN_PRIMARY, "secondary": _BTN_SECONDARY, "danger":
_BTN_DANGER}
    btn.setStyleSheet(styles.get(kind, _BTN_PRIMARY))
    btn.setFont(QFont("Microsoft YaHei", 13, QFont.Weight.Bold if kind != "secondary" else
QFont.Weight.Normal))
class StyledDialog(QDialog):
    def __init__(self, title: str, message: str, dialog_type: str = "info", parent=None):
        super().__init__(parent)
        self.setWindowTitle(title)
        self.setModal(True)
        self.setMinimumWidth(420)
        self.setWindowFlags(self.windowFlags()
~Qt.WindowType.WindowContextHelpButtonHint)
        self._build(message, dialog_type)
    def _build(self, message: str, dialog_type: str) -> None:
        icons = {"info": "i", "success": "✓", "warning": "⚠", "error": "✗"}
        colors = {"info": ACCENT, "success": "#81c784", "warning": WARN, "error": DANGER}
        icon = icons.get(dialog_type, "i")
        color = colors.get(dialog_type, ACCENT)
        layout = QVBoxLayout(self)
        layout.setContentsMargins(24, 24, 24, 20)
        layout.setSpacing(16)
        header = QHBoxLayout()
        icon_label = QLabel(icon)
        icon_label.setFont(QFont("Segoe UI", 22))
        icon_label.setStyleSheet(f"color: {color}; background: transparent;")
        icon_label.setFixedWidth(40)
        title_label = QLabel(self.windowTitle())

```

```

title_label.setFont(QFont("Microsoft YaHei", 14, QFont.Weight.Bold))
title_label.setStyleSheet(f"color: {TEXT_PRIMARY}; background: transparent;")
header.addWidget(icon_label)
header.addWidget(title_label, 1)
layout.addLayout(header)
line = QFrame()
line.setFrameShape(QFrame.Shape.HLine)
line.setStyleSheet(f"background: {BORDER}; max-height: 1px; border: none;")
layout.addWidget(line)
body = QLabel(message)
body.setWordWrap(True)
body.setStyleSheet(f"color: {TEXT_SECONDARY}; background: transparent; padding: 4px
0;")

layout.addWidget(body)
btn_row = QHBoxLayout()
btn_row.addStretch()
ok_btn = QPushButton("确定")
_style_button(ok_btn, "primary")
ok_btn.setMinimumWidth(100)
ok_btn.clicked.connect(self.accept)
btn_row.addWidget(ok_btn)
layout.addLayout(btn_row)
self.setStyleSheet(f"QDialog {{ background-color: {BG_PANEL}; border: 1px solid
{BORDER}; }}")
class ConfirmDialog(QDialog):
    def __init__(self, title: str, message: str, parent=None):
        super().__init__(parent)
        self.setWindowTitle(title)
        self.setModal(True)
        self.setMinimumWidth(440)
        self._result = False
        self._build(message)
    def _build(self, message: str) -> None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(24, 24, 24, 20)
        layout.setSpacing(16)
        title_label = QLabel(self.windowTitle())
        title_label.setFont(QFont("Microsoft YaHei", 14, QFont.Weight.Bold))
        title_label.setStyleSheet(f"color: {WARN}; background: transparent;")
        layout.addWidget(title_label)
        body = QLabel(message)
        body.setWordWrap(True)
        body.setStyleSheet(f"color: {TEXT_SECONDARY}; background: transparent;")
        layout.addWidget(body)
        btn_row = QHBoxLayout()
        btn_row.addStretch()
        cancel_btn = QPushButton("取消")
        _style_button(cancel_btn, "secondary")
        cancel_btn.setMinimumWidth(90)
        cancel_btn.clicked.connect(self.reject)
        ok_btn = QPushButton("确认")
        _style_button(ok_btn, "danger")
        ok_btn.setMinimumWidth(90)

```

```

        ok_btn.clicked.connect(self._on_confirm)
        btn_row.addWidget(cancel_btn)
        btn_row.addWidget(ok_btn)
        layout.addLayout(btn_row)
        self.setStyleSheet(f"QDialog {{ background-color: {BG_PANEL}; border: 1px solid
{BORDER}; }}" )
    def _on_confirm(self) -> None:
        self._result = True
        self.accept()
    @property
    def confirmed(self) -> bool:
        return self._result
class DetailDialog(QDialog):
    def __init__(self, title: str, content: str, parent=None):
        super().__init__(parent)
        self.setWindowTitle(title)
        self.setModal(True)
        self.setMinimumSize(520, 360)
        layout = QVBoxLayout(self)
        layout.setContentsMargins(20, 20, 20, 16)
        title_label = QLabel(title)
        title_label.setFont(QFont("Microsoft YaHei", 13, QFont.Weight.Bold))
        title_label.setStyleSheet(f"color: {ACCENT}; background: transparent;")
        layout.addWidget(title_label)
        editor = QTextEdit()
        editor.setReadOnly(True)
        editor.setPlainText(content)
        layout.addWidget(editor, 1)
        btn = QPushButton("关闭")
        _style_button(btn, "primary")
        btn.setMinimumWidth(100)
        btn.clicked.connect(self.accept)
        row = QHBoxLayout()
        row.addStretch()
        row.addWidget(btn)
        layout.addLayout(row)
        self.setStyleSheet(f"QDialog {{ background-color: {BG_PANEL}; }}" )
def show_info(parent, title: str, message: str) -> None:
    StyledDialog(title, message, "info", parent).exec()
def show_success(parent, title: str, message: str) -> None:
    StyledDialog(title, message, "success", parent).exec()
def show_warning(parent, title: str, message: str) -> None:
    StyledDialog(title, message, "warning", parent).exec()
def show_error(parent, title: str, message: str) -> None:
    StyledDialog(title, message, "error", parent).exec()
def ask_confirm(parent, title: str, message: str) -> bool:
    dlg = ConfirmDialog(title, message, parent)
    dlg.exec()
    return dlg.confirmed
# ==== ui/ __init__.py (0行) ====
# ==== ui/login_window.py (491行) ====
import math
import random

```

```

from PyQt6.QtCore import Qt, QTimer, QPointF, pyqtSignal
from PyQt6.QtGui import QPainter, QColor, QLinearGradient, QRadialGradient, QFont, QPen
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QLabel, QLineEdit, QPushButton,
    QStackedWidget, QFrame, QGraphicsDropShadowEffect,
)
from core.constants import APP_NAME, APP_VERSION, DEFAULT_USERNAME,
DEFAULT_PASSWORD
from core.auth import AuthManager
from core.dialogs import show_success, show_error
from core.styles import ACCENT, BG_DARK, TEXT_SECONDARY, ACCENT_GLOW, BORDER
class ParticleCanvas(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._particles: list[dict] = []
        self._nodes: list[dict] = []
        self._phase = 0.0
        self._timer = QTimer(self)
        self._timer.timeout.connect(self._tick)
        self._timer.start(33)
        self._init_particles()
        self._init_nodes()
    def _init_particles(self) -> None:
        for _ in range(120):
            self._particles.append({
                "x": random.uniform(0, 1),
                "y": random.uniform(0, 1),
                "vx": random.uniform(-0.0005, 0.0005),
                "vy": random.uniform(-0.0008, -0.0002),
                "size": random.uniform(1.0, 3.0),
                "alpha": random.uniform(0.1, 0.5),
                "phase": random.uniform(0, math.pi * 2),
                "hue": random.choice([0.55, 0.58, 0.62]),
            })
    def _init_nodes(self) -> None:
        for _ in range(16):
            self._nodes.append({
                "x": random.uniform(0.1, 0.9),
                "y": random.uniform(0.15, 0.85),
                "r": random.uniform(2, 5),
                "phase": random.uniform(0, math.pi * 2),
            })
    def _tick(self) -> None:
        self._phase += 0.03
        for p in self._particles:
            p["x"] += p["vx"]
            p["y"] += p["vy"]
            p["phase"] += 0.04
            if p["y"] < -0.05:
                p["y"] = 1.05
                p["x"] = random.uniform(0, 1)
            if p["x"] < 0 or p["x"] > 1:
                p["vx"] *= -1
        self.update()

```

```

def paintEvent(self, event) -> None:
    painter = QPainter(self)
    painter.setRenderHint(QPainter.RenderHint.Antialiasing)
    w, h = self.width(), self.height()
    grad = QLinearGradient(0, 0, w * 0.6, h)
    grad.setColorAt(0, QColor("#040d1a"))
    grad.setColorAt(0.4, QColor("#08182a"))
    grad.setColorAt(1, QColor("#0a2040"))
    painter.fillRect(self.rect(), grad)
    glow = QRadialGradient(w * 0.35, h * 0.45, w * 0.5)
    glow.setColorAt(0, QColor(79, 195, 247, 30))
    glow.setColorAt(0.6, QColor(30, 80, 160, 12))
    glow.setColorAt(1, QColor(0, 0, 0, 0))
    painter.fillRect(self.rect(), glow)
    scan_y = int((self._phase * 0.15 % 1.0) * h)
    scan_grad = QLinearGradient(0, scan_y - 40, 0, scan_y + 40)
    scan_grad.setColorAt(0, QColor(79, 195, 247, 0))
    scan_grad.setColorAt(0.5, QColor(79, 195, 247, 15))
    scan_grad.setColorAt(1, QColor(79, 195, 247, 0))
    painter.fillRect(0, scan_y - 40, w, 80, scan_grad)
    for i in range(5):
        cx = w * (0.1 + i * 0.18)
        cy = h * 0.45 + math.sin(self._phase + i * 0.8) * 25
        radius = 100 + i * 25
        color = QColor(79, 195, 247, 6 + i * 4)
        painter.setPen(Qt.PenStyle.NoPen)
        painter.setBrush(color)
        painter.drawEllipse(QPointF(cx, cy), radius, radius * 0.55)
    grid_pen = QPen(QColor(79, 195, 247, 10))
    grid_pen.setWidth(1)
    painter.setPen(grid_pen)
    step = 48
    for x in range(0, w, step):
        painter.drawLine(x, 0, x, h)
    for y in range(0, h, step):
        painter.drawLine(0, y, w, y)
    if len(self._nodes) >= 2:
        painter.setPen(QPen(QColor(79, 195, 247, 30), 1))
        for i, n1 in enumerate(self._nodes):
            for n2 in self._nodes[i + 1:i + 3]:
                x1, y1 = n1["x"] * w, n1["y"] * h
                x2, y2 = n2["x"] * w, n2["y"] * h
                dist = math.hypot(x2 - x1, y2 - y1)
                if dist < w * 0.35:
                    pulse = 0.3 + 0.7 * abs(math.sin(self._phase + i))
                    painter.setPen(QPen(QColor(79, 195, 247, int(25 * pulse)), 1))
                    painter.drawLine(int(x1), int(y1), int(x2), int(y2))
    for n in self._nodes:
        nx, ny = n["x"] * w, n["y"] * h
        pulse = 0.5 + 0.5 * math.sin(self._phase + n["phase"])
        radial = QRadialGradient(nx, ny, n["r"] * 3)
        radial.setColorAt(0, QColor(129, 212, 250, int(180 * pulse)))
        radial.setColorAt(1, QColor(129, 212, 250, 0))
        painter.setPen(Qt.PenStyle.NoPen)

```

```

        painter.setBrush(radial)
        painter.drawEllipse(QPointF(nx, ny), n["r"] * 2, n["r"] * 2)
    for p in self._particles:
        alpha = int(255 * p["alpha"] * (0.6 + 0.4 * math.sin(p["phase"])))
        painter.setBrush(QColor(79, 195, 247, alpha))
        painter.drawEllipse(QPointF(p["x"] * w, p["y"] * h), p["size"], p["size"])
    painter.end()
class StatCard(QFrame):
    def __init__(self, value: str, label: str, parent=None):
        super().__init__(parent)
        self.setFixedHeight(72)
        self.setStyleSheet("""
            QFrame {
                background-color: rgba(18, 30, 58, 0.65);
                border: 1px solid rgba(79, 195, 247, 0.25);
                border-radius: 10px;
            }
        """)
        lay = QVBoxLayout(self)
        lay.setContentsMargins(14, 10, 14, 10)
        lay.setSpacing(2)
        v = QLabel(value)
        v.setFont(QFont("Segoe UI", 22, QFont.Weight.Bold))
        v.setStyleSheet(f"color: {ACCENT_GLOW}; background: transparent;")
        u = QLabel(label)
        u.setStyleSheet(f"color: {TEXT_SECONDARY}; background: transparent; font-size:
11px;")
        lay.addWidget(v)
        lay.addWidget(u)
class TabSwitch(QWidget):
    switched = pyqtSignal(int)
    def __init__(self, parent=None):
        super().__init__(parent)
        self._index = 0
        lay = QHBoxLayout(self)
        lay.setContentsMargins(0, 0, 0, 0)
        lay.setSpacing(0)
        self._login_tab = QPushButton("登 录")
        self._reg_tab = QPushButton("注 册")
        for btn in [self._login_tab, self._reg_tab]:
            btn.setFixedHeight(38)
            btn.setCursor(Qt.CursorShape.PointingHandCursor)
            lay.addWidget(btn)
        self._login_tab.clicked.connect(lambda: self._select(0))
        self._reg_tab.clicked.connect(lambda: self._select(1))
        self._update_style()
    def _select(self, index: int) -> None:
        self._index = index
        self._update_style()
        self.switched.emit(index)
    def _update_style(self) -> None:
        active = ""
        QPushButton {

```

```

        background-color: rgba(79, 195, 247, 0.15);
        color: #81d4fa;
        border: none;
        border-bottom: 2px solid #4fc3f7;
        font-weight: bold;
        font-size: 13px;
    }
    """
inactive = """
    QPushButton {
        background-color: transparent;
        color: #8fa8c8;
        border: none;
        border-bottom: 2px solid transparent;
        font-size: 13px;
    }
    QPushButton:hover { color: #e8edf5; }
    """

self._login_tab.setStyleSheet(active if self._index == 0 else inactive)
self._reg_tab.setStyleSheet(active if self._index == 1 else inactive)
class AuthForm(QWidget):
    login_success = pyqtSignal(str)
    def __init__(self, auth: AuthManager, parent=None):
        super().__init__(parent)
        self._auth = auth
        self._build()
    def _build(self) -> None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(36, 32, 36, 32)
        layout.setSpacing(16)
        logo_row = QHBoxLayout()
        logo = QLabel("@")
        logo.setFixedSize(44, 44)
        logo.setAlignment(Qt.AlignmentFlag.AlignCenter)
        logo.setStyleSheet(f"""
            background: qlineargradient(x1:0,y1:0,x2:1,y2:1, stop:0 {ACCENT}, stop:1
#1565c0);
            border-radius: 22px;
            font-size: 20px;
            color: {BG_DARK};
        """)
        title_col = QVBoxLayout()
        title_col.setSpacing(2)
        title = QLabel(APP_NAME)
        title.setFont(QFont("Microsoft YaHei", 13, QFont.Weight.Bold))
        title.setWordWrap(True)
        title.setStyleSheet(f"color: {ACCENT_GLOW}; background: transparent;")
        ver = QLabel(f"版本 {APP_VERSION}")
        ver.setStyleSheet(f"color: {TEXT_SECONDARY}; background: transparent; font-size:
11px;")
        title_col.addWidget(title)
        title_col.addWidget(ver)
        logo_row.addWidget(logo)
        logo_row.addLayout(title_col, 1)

```

```

        layout.addLayout(logo_row)
        self._tabs = TabSwitch()
        self._tabs.switched.connect(self._on_tab_switch)
        layout.addWidget(self._tabs)
        self._stack = QStackedWidget()
        self._stack.addWidget(self._make_login_page())
        self._stack.addWidget(self._make_register_page())
        layout.addWidget(self._stack)
def _on_tab_switch(self, index: int) -> None:
    self._stack.setCurrentIndex(index)
def _input_style(self) -> str:
    return f"""
        QLineEdit {{
            background-color: rgba(31, 48, 96, 0.8);
            color: #e8edf5;
            border: 1px solid {BORDER};
            border-left: 3px solid {ACCENT};
            border-radius: 8px;
            padding: 10px 14px;
            font-size: 13px;
        }}
        QLineEdit:focus {{
            border: 1px solid {ACCENT};
            border-left: 3px solid {ACCENT_GLOW};
        }}
    """
def _apply_primary_btn(self, btn: QPushButton) -> None:
    btn.setFont(QFont("Microsoft YaHei", 15, QFont.Weight.Bold))
    btn.setStyleSheet("""
        QPushButton {
            background: qlineargradient(x1:0,y1:0,x2:1,y2:0,
                stop:0 #1565c0, stop:1 #1e88e5);
            color: #ffffff;
            border: 1px solid #42a5f5;
            border-radius: 8px;
            padding: 10px 16px;
            font-size: 15px;
            font-weight: bold;
        }
        QPushButton:hover {
            background: qlineargradient(x1:0,y1:0,x2:1,y2:0,
                stop:0 #1e88e5, stop:1 #42a5f5);
            color: #ffffff;
            border-color: #64b5f6;
        }
        QPushButton:pressed {
            background: #0d47a1;
            color: #ffffff;
        }
    """)
def _make_login_page(self) -> QWidget:
    page = QWidget()
    page.setStyleSheet("background: transparent;")
    lay = QVBoxLayout(page)

```

```

lay.setSpacing(14)
lay.addWidget(self._field_label("账号"))
self._login_user = QLineEdit()
self._login_user.setText(DEFAULT_USERNAME)
self._login_user.setPlaceholderText("请输入账号")
self._login_user.setStyleSheet(self._input_style())
self._login_user.setMinimumHeight(42)
lay.addWidget(self._login_user)
lay.addWidget(self._field_label("密码"))
self._login_pass = QLineEdit()
self._login_pass.setText(DEFAULT_PASSWORD)
self._login_pass.setEchoMode(QLineEdit.EchoMode.Password)
self._login_pass.setPlaceholderText("请输入密码")
self._login_pass.setStyleSheet(self._input_style())
self._login_pass.setMinimumHeight(42)
self._login_pass.returnPressed.connect(self._do_login)
lay.addWidget(self._login_pass)
lay.addSpacing(6)
login_btn = QPushButton("登 录")
self._apply_primary_btn(login_btn)
login_btn.setMinimumHeight(48)
login_btn.setCursor(Qt.CursorShape.PointingHandCursor)
login_btn.clicked.connect(self._do_login)
lay.addWidget(login_btn)
return page
def _make_register_page(self) -> QWidget:
    page = QWidget()
    page.setStyleSheet("background: transparent;")
    lay = QVBoxLayout(page)
    lay.setSpacing(12)
    for label_text, attr, placeholder in [
        ("新账号", "_reg_user", "至少3个字符"),
        ("密码", "_reg_pass", "至少6个字符"),
        ("确认密码", "_reg_confirm", "再次输入密码"),
    ]:
        lay.addWidget(self._field_label(label_text))
        field = QLineEdit()
        field.setPlaceholderText(placeholder)
        field.setStyleSheet(self._input_style())
        field.setMinimumHeight(40)
        if "密码" in label_text:
            field.setEchoMode(QLineEdit.EchoMode.Password)
        setattr(self, attr, field)
        lay.addWidget(field)
    self._reg_confirm.returnPressed.connect(self._do_register)
    lay.addSpacing(4)
    reg_btn = QPushButton("注 册")
    self._apply_primary_btn(reg_btn)
    reg_btn.setMinimumHeight(48)
    reg_btn.setCursor(Qt.CursorShape.PointingHandCursor)

```

```

        reg_btn.clicked.connect(self._do_register)
        lay.addWidget(reg_btn)
        return page
def _field_label(self, text: str) -> QLabel:
    lbl = QLabel(text)
    lbl.setStyleSheet(f"color: {TEXT_SECONDARY}; background: transparent; font-size:
12px;")
    return lbl
def _do_login(self) -> None:
    ok, msg = self._auth.login(self._login_user.text(), self._login_pass.text())
    if ok:
        self.login_success.emit(self._login_user.text().strip())
    else:
        show_error(self, "登录失败", msg)
def _do_register(self) -> None:
    ok, msg = self._auth.register(
        self._reg_user.text(), self._reg_pass.text(), self._reg_confirm.text()
    )
    if ok:
        show_success(self, "注册成功", msg)
        self._login_user.setText(self._reg_user.text().strip())
        self._tabs._select(0)
        self._stack.setCurrentIndex(0)
    else:
        show_error(self, "注册失败", msg)
class LoginWindow(QWidget):
    login_success = pyqtSignal(str)
    def __init__(self):
        super().__init__()
        self._auth = AuthManager()
        self.setWindowTitle(APP_NAME)
        self.setMinimumSize(1000, 620)
        self.resize(1180, 720)
        self._build()
    def _build(self) -> None:
        root = QHBoxLayout(self)
        root.setContentsMargins(0, 0, 0, 0)
        root.setSpacing(0)
        self._canvas = ParticleCanvas()
        left_wrap = QVBoxLayout()
        left_wrap.setContentsMargins(52, 52, 52, 48)
        badge = QLabel("运行日志智能分析")
        badge.setStyleSheet(f"""
            color: {ACCENT};
            background: rgba(79, 195, 247, 0.1);
            border: 1px solid rgba(79, 195, 247, 0.3);
            border-radius: 14px;
            padding: 6px 16px;
            font-size: 12px;
            max-width: 180px;
        """)
        badge.setAlignment(Qt.AlignmentFlag.AlignCenter)

```

```

left_wrap.addWidget(badge, alignment=Qt.AlignmentFlag.AlignLeft)
left_wrap.addSpacing(20)
slogan = QLabel("信息系统运行日志\n智能分析与故障预测")
slogan.setFont(QFont("Microsoft YaHei", 28, QFont.Weight.Bold))
slogan.setStyleSheet("color: #e8edf5; background: transparent; line-height: 1.4;")
left_wrap.addWidget(slogan)
sub = QLabel("日志采集汇聚 · 故障预测 · 根因定位 · 智能告警")
sub.setStyleSheet(f"color: {TEXT_SECONDARY}; background: transparent; font-size:
13px; margin-top: 8px;")
left_wrap.addWidget(sub)
left_wrap.addSpacing(24)
features = QHBoxLayout()
features.setSpacing(10)
for text in ["日志解析", "故障预测", "根因分析"]:
    chip = QLabel(text)
    chip.setStyleSheet("""
        color: #8fa8c8;
        background: rgba(26, 42, 78, 0.6);
        border: 1px solid #2a4070;
        border-radius: 12px;
        padding: 5px 14px;
        font-size: 11px;
    """)
    features.addWidget(chip)
features.addStretch()
left_wrap.addLayout(features)
left_wrap.addStretch()
stat_row = QHBoxLayout()
stat_row.setSpacing(14)
for val, unit in [("15万+", "日志/秒"), ("128", "接入节点"), ("99.5%", "预测准确率")]:
    stat_row.addWidget(StatCard(val, unit))
left_wrap.addLayout(stat_row)
left_panel = QWidget()
left_panel.setLayout(left_wrap)
left_panel.setStyleSheet("background: transparent;")
canvas_layout = QHBoxLayout(self._canvas)
canvas_layout.setContentsMargins(0, 0, 0, 0)
canvas_layout.addWidget(left_panel, 1)
root.addWidget(self._canvas, 1)
card_outer = QFrame()
card_outer.setFixedWidth(440)
card_outer.setStyleSheet("QFrame { background: transparent; }")
outer_lay = QVBoxLayout(card_outer)
outer_lay.setContentsMargins(24, 40, 24, 40)
outer_lay.setAlignment(Qt.AlignmentFlag.AlignCenter)
card = QFrame()
card.setStyleSheet(f"""
    QFrame {{
        background-color: rgba(11, 20, 38, 0.92);
        border: 1px solid rgba(79, 195, 247, 0.2);
        border-radius: 16px;
    }}

```

```

        """)
        shadow = QGraphicsDropShadowEffect()
        shadow.setBlurRadius(50)
        shadow.setColor(QColor(79, 195, 247, 70))
        shadow.setOffset(0, 4)
        card.setGraphicsEffect(shadow)
        form = AuthForm(self._auth)
        form.login_success.connect(self.login_success.emit)
        card_layout = QVBoxLayout(card)
        card_layout.setContentsMargins(0, 0, 0, 0)
        card_layout.addWidget(form)
        outer_lay.addWidget(card)
        root.addWidget(card_outer)

# ==== ui/main_window.py (340行) ====
from datetime import datetime
from PyQt6.QtCore import Qt, QTimer, pyqtSignal
from PyQt6.QtGui import QPainter, QColor, QFont, QPen, QLinearGradient
from PyQt6.QtWidgets import (
    QMainWindow, QWidget, QVBoxLayout, QHBoxLayout, QLabel, QStackedWidget,
    QPushButton, QFrame, QButtonGroup, QScrollArea,
)
from core.constants import APP_NAME, APP_VERSION, DEFAULT_WIDTH, DEFAULT_HEIGHT,
WINDOW_MIN_WIDTH, WINDOW_MIN_HEIGHT
from modules import MODULE_REGISTRY
class NavMenuButton(QPushButton):
    def __init__(self, index: int, icon: str, label: str, parent=None):
        super().__init__(parent)
        self._index = index
        self._icon = icon
        self._label = label
        self.setCheckable(True)
        self.setCursor(Qt.CursorShape.PointingHandCursor)
        self.setFixedHeight(46)
        self.setText(f" {icon} {index:02d} {label}")
        self._apply_style(False)
    def set_nav_active(self, active: bool) -> None:
        self.setChecked(active)
        self._apply_style(active)
    def _apply_style(self, active: bool) -> None:
        if active:
            self.setStyleSheet("""
                QPushButton {
                    background: qlineargradient(x1:0,y1:0,x2:1,y2:0,
                        stop:0 rgba(79, 195, 247, 0.28), stop:1 rgba(30, 100, 200, 0.12));
                    color: #81d4fa;
                    border: none;
                    border-left: 3px solid #4fc3f7;
                    border-radius: 0 8px 8px 0;
                    text-align: left;
                    padding-left: 10px;
                    font-size: 12px;
                    font-weight: bold;
                }
            """)
        """)

```

```

else:
    self.setStyleSheet("""
        QPushButton {
            background: transparent;
            color: #8fa8c8;
            border: none;
            border-left: 3px solid transparent;
            border-radius: 0 8px 8px 0;
            text-align: left;
            padding-left: 10px;
            font-size: 12px;
        }
        QPushButton:hover {
            background: rgba(20, 42, 74, 0.7);
            color: #e8edf5;
            border-left: 3px solid #2a5a8a;
        }
    """)

class StaticSideNav(QWidget):
    module_selected = pyqtSignal(int)
    GROUPS = [
        ("日志采集与分析", [0, 1, 2, 3]),
        ("智能诊断与预测", [4, 5, 6, 7, 8]),
        ("监控与运维管理", [9, 10, 11, 12, 13]),
        ("系统配置与增强", [14, 15, 16, 17, 18, 19]),
    ]
    ICONS = [
        "👉", "🔍", "⚠️", "🔎", "💻", "🎯", "🖱️", "🌐", "💊", "📊", "📄", "🧠", "📝",
        "🎓", "🍷", "🌍", "✉️", "📐", "⚙️", "📱"
    ]
    def __init__(self, modules: list, parent=None):
        super().__init__(parent)
        self._modules = modules
        self._buttons: list[NavMenuButton] = []
        self._group = QButtonGroup(self)
        self._group.setExclusive(True)
        self._build()
    def _build(self) -> None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(0, 4, 0, 4)
        layout.setSpacing(4)
        for gi, (group_name, indices) in enumerate(self.GROUPS):
            header = QLabel(group_name)
            header.setStyleSheet("""
                color: #6a8aaa;
                background: transparent;
                font-size: 11px;
                padding: 8px 12px 4px 14px;
                letter-spacing: 1px;
            """)
            layout.addWidget(header)
            for idx in indices:
                _, name, _ = self._modules[idx]
                btn = NavMenuButton(idx + 1, self.ICONs[idx], name)

```

```

        btn.clicked.connect(lambda checked, i=idx: (self.select_module(i),
self.emit_selected(i)))
        self._group.addButton(btn, idx)
        self._buttons.append(btn)
        layout.addWidget(btn)
        if gi < len(self.GROUPS) - 1:
            sep = QFrame()
            sep.setFixedHeight(1)
            sep.setStyleSheet("background: rgba(30, 58, 95, 0.6); border: none; margin:
6px 12px;")
            layout.addWidget(sep)
        layout.addStretch()
    def select_module(self, index: int) -> None:
        for i, btn in enumerate(self._buttons):
            btn.set_nav_active(self._group.id(btn) == index)
    def emit_selected(self, index: int) -> None:
        self.module_selected.emit(index)
class NavPanel(QFrame):
    def paintEvent(self, event) -> None:
        painter = QPainter(self)
        grad = QLinearGradient(0, 0, self.width(), self.height())
        grad.setColorAt(0, QColor("#061020"))
        grad.setColorAt(0.5, QColor("#0a1a30"))
        grad.setColorAt(1, QColor("#081428"))
        painter.fillRect(self.rect(), grad)
        painter.setPen(QPen(QColor(79, 195, 247, 40), 1))
        painter.drawLine(self.width() - 1, 0, self.width() - 1, self.height())
        painter.end()
class TopBar(QFrame):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setFixedHeight(58)
        self._time_lbl = QLabel()
        self._time_lbl.setStyleSheet("color: #8fa8c8; background: transparent; font-size: 12px;")
        self._timer = QTimer(self)
        self._timer.timeout.connect(self._update_time)
        self._timer.start(1000)
        self._update_time()
    def _update_time(self) -> None:
        self._time_lbl.setText(datetime.now().strftime("%Y年%m月%d日 %H:%M:%S"))
    def paintEvent(self, event) -> None:
        painter = QPainter(self)
        grad = QLinearGradient(0, 0, self.width(), 0)
        grad.setColorAt(0, QColor("#0a1a30"))
        grad.setColorAt(0.5, QColor("#0e2040"))
        grad.setColorAt(1, QColor("#0a1a30"))
        painter.fillRect(self.rect(), grad)
        painter.setPen(QPen(QColor(79, 195, 247, 50), 1))
        painter.drawLine(0, self.height() - 1, self.width(), self.height() - 1)
        painter.end()
class StatusBar(QFrame):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setFixedHeight(28)

```

```

self._lbl = QLabel("系统运行正常 · 日志分析平台在线")
self._lbl.setStyleSheet("color: #6a8aaa; background: transparent; font-size: 11px;")
lay = QHBoxLayout(self)
lay.setContentsMargins(16, 0, 16, 0)
dot = QLabel("●")
dot.setStyleSheet("color: #4fc3f7; background: transparent; font-size: 10px;")
lay.addWidget(dot)
lay.addWidget(self._lbl)
lay.addStretch()

ver = QLabel(f"版本 {APP_VERSION}")
ver.setStyleSheet("color: #6a8aaa; background: transparent; font-size: 11px;")
lay.addWidget(ver)

def paintEvent(self, event) -> None:
    painter = QPainter(self)
    painter.fillRect(self.rect(), QColor("#061020"))
    painter.setPen(QPen(QColor(30, 58, 95), 1))
    painter.drawLine(0, 0, self.width(), 0)
    painter.end()

class ContentFrame(QFrame):
    def paintEvent(self, event) -> None:
        painter = QPainter(self)
        painter.fillRect(self.rect(), QColor("#0a1628"))
        painter.setPen(QPen(QColor(79, 195, 247, 12)))
        step = 60
        for x in range(0, self.width(), step):
            painter.drawLine(x, 0, x, self.height())
        for y in range(0, self.height(), step):
            painter.drawLine(0, y, self.width(), y)
        painter.end()

class MainWindow(QMainWindow):
    def __init__(self, username: str):
        super().__init__()
        self._username = username
        self.setWindowTitle(f"{APP_NAME} - v{APP_VERSION}")
        self.setMinimumSize(WINDOW_MIN_WIDTH, WINDOW_MIN_HEIGHT)
        self.resize(DEFAULT_WIDTH, DEFAULT_HEIGHT)
        self._modules: list[QWidget] = []
        self._build()

    def _build(self) -> None:
        central = QWidget()
        central.setStyleSheet("background: #0a1628;")
        self.setCentralWidget(central)
        root = QHBoxLayout(central)
        root.setContentsMargins(0, 0, 0, 0)
        root.setSpacing(0)
        nav_panel = NavPanel()
        nav_panel.setFixedWidth(240)
        nav_layout = QVBoxLayout(nav_panel)
        nav_layout.setContentsMargins(0, 16, 0, 14)
        nav_layout.setSpacing(10)
        scroll = QScrollArea()
        scroll.setWidgetResizable(True)
        scroll.setFrameShape(QFrame.Shape.NoFrame)

```

```

scroll.setStyleSheet("QScrollArea { background: transparent; border: none; }")
self._navigator = StaticSideNav(MODULE_REGISTRY)
self._navigator.module_selected.connect(self._on_module_changed)
scroll.addWidget(self._navigator)
nav_layout.addWidget(scroll, 1)
user_box = QFrame()
user_box.setStyleSheet("""
    QFrame {
        background: rgba(20, 42, 74, 0.6);
        border: 1px solid #2a4070;
        border-radius: 8px;
        margin: 0 10px;
    }
""")
user_lay = QHBoxLayout(user_box)
user_lay.setContentsMargins(10, 8, 10, 8)
avatar = QLabel(self._username[0].upper() if self._username else "A")
avatar.setFixedSize(30, 30)
avatar.setAlignment(Qt.AlignmentFlag.AlignCenter)
avatar.setStyleSheet("""
    background: qlineargradient(x1:0,y1:0,x2:1,y2:1, stop:0 #4fc3f7, stop:1 #1565c0);
    border-radius: 15px;
    color: #061220;
    font-weight: bold;
    font-size: 13px;
""")
user_info = QVBoxLayout()
user_info.setSpacing(0)
user_name = QLabel(self._username)
user_name.setStyleSheet("color: #e8edf5; background: transparent; font-size: 12px;
font-weight: bold;")
user_role = QLabel("系统运维工程师")
user_role.setStyleSheet("color: #6a8aaa; background: transparent; font-size: 10px;")
user_info.addWidget(user_name)
user_info.addWidget(user_role)
user_lay.addWidget(avatar)
user_lay.addLayout(user_info, 1)
nav_layout.addWidget(user_box)
logout_btn = QPushButton("退出登录")
logout_btn.setProperty("danger", True)
logout_btn.setCursor(Qt.CursorShape.PointingHandCursor)
logout_btn.setStyleSheet(logout_btn.styleSheet() + "margin: 0 10px;")
logout_btn.clicked.connect(self.close)
nav_layout.addWidget(logout_btn)
right = QVBoxLayout()
right.setContentsMargins(0, 0, 0, 0)
right.setSpacing(0)
top_bar = TopBar()
top_lay = QHBoxLayout(top_bar)
top_lay.setContentsMargins(24, 0, 24, 0)
title_col = QVBoxLayout()
title_col.setSpacing(0)
self._module_title = QLabel(MODULE_REGISTRY[0][1])

```

```

self._module_title.setFont(QFont("Microsoft YaHei", 15, QFont.Weight.Bold))
self._module_title.setStyleSheet("color: #e8edf5; background: transparent;")
breadcrumb = QLabel("首页 / 功能模块")
breadcrumb.setStyleSheet("color: #6a8aaa; background: transparent; font-size: 11px;")
self._breadcrumb = breadcrumb
title_col.addWidget(self._module_title)
title_col.addWidget(breadcrumb)
top_lay.addLayout(title_col)
top_lay.addStretch()
status_row = QHBoxLayout()
status_row.setSpacing(16)
for dot_color, text in [( "#4fc3f7", "日志采集"), (" #4fc3f7", "分析引擎"), (" #4fc3f7", "通
信正常")]:
    item = QHBoxLayout()
    item.setSpacing(4)
    d = QLabel("●")
    d.setStyleSheet(f"color: {dot_color}; background: transparent; font-size: 9px;")
    t = QLabel(text)
    t.setStyleSheet("color: #6a8aaa; background: transparent; font-size: 11px;")
    item.addWidget(d)
    item.addWidget(t)
    status_row.addLayout(item)
top_lay.addLayout(status_row)
top_lay.addSpacing(20)
top_lay.addWidget(top_bar._time_lbl)
content_wrap = ContentFrame()
content_lay = QVBoxLayout(content_wrap)
content_lay.setContentsMargins(0, 0, 0, 0)
self._stack = QStackedWidget()
self._stack.setStyleSheet("background: transparent;")
for _, name, cls in MODULE_REGISTRY:
    module = cls()
    self._modules.append(module)
    self._stack.addWidget(module)
content_lay.addWidget(self._stack)
status_bar = StatusBar()
right.addWidget(top_bar)
right.addWidget(content_wrap, 1)
right.addWidget(status_bar)
root.addWidget(nav_panel)
root.addLayout(right, 1)
self._navigator.select_module(0)
self._navigator.emit_selected(0)
def _on_module_changed(self, index: int) -> None:
    self._stack.setCurrentIndex(index)
    self._navigator.select_module(index)
    name = MODULE_REGISTRY[index][1]
    self._module_title.setText(name)
    self._breadcrumb.setText(f"首页 / {name}")
# ==== modules/_init_.py (43行) ====
from modules.log_collector import LogCollectorModule
from modules.log_parser import LogParserModule

```

```

from modules.anomaly_detection import AnomalyDetectionModule
from modules.log_search import LogSearchModule
from modules.fault_prediction import FaultPredictionModule
from modules.root_cause import RootCauseModule
from modules.alarm_center import AlarmCenterModule
from modules.service_topology import ServiceTopologyModule
from modules.smart_diagnosis import SmartDiagnosisModule
from modules.dashboard import DashboardModule
from modules.report_center import ReportCenterModule
from modules.knowledge_base import KnowledgeBaseModule
from modules.sla_monitor import SLAMonitorModule
from modules.model_train import ModelTrainModule
from modules.sys_monitor import SysMonitorModule
from modules.visualization import VisualizationModule
from modules.alert_notify import AlertNotifyModule
from modules.custom_dashboard import CustomDashboardModule
from modules.config_center import ConfigCenterModule
from modules.audit_archive import AuditArchiveModule
MODULE_REGISTRY = [
    ("collect", "日志采集汇聚", LogCollectorModule),
    ("parser", "日志解析引擎", LogParserModule),
    ("anomaly", "异常检测识别", AnomalyDetectionModule),
    ("search", "日志联机搜索", LogSearchModule),
    ("predict", "故障预测模型", FaultPredictionModule),
    ("cause", "根因定位分析", RootCauseModule),
    ("alarm", "告警管理中心", AlarmCenterModule),
    ("topology", "服务拓扑发现", ServiceTopologyModule),
    ("diagnosis", "智能诊断修复", SmartDiagnosisModule),
    ("dash", "态势感知总览", DashboardModule),
    ("report", "智能报告生成", ReportCenterModule),
    ("kb", "知识图谱构建", KnowledgeBaseModule),
    ("sla", "SLA合规监控", SLAMonitorModule),
    ("train", "模型训练管理", ModelTrainModule),
    ("monitor", "系统监控诊断", SysMonitorModule),
    ("visual", "多维可视化", VisualizationModule),
    ("notify", "告警通知配置", AlertNotifyModule),
    ("custom_dash", "自定义仪表盘", CustomDashboardModule),
    ("config", "配置管理中心", ConfigCenterModule),
    ("audit", "日志审计归档", AuditArchiveModule),
]
# ==== modules/common.py (1行) ====
# Common utilities - intentionally empty (previously had setup_table, now removed)
# ==== modules/log_collector.py (129行) ====
import random
from datetime import datetime

```

```

from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QSplitter, QFrame, QComboBox, QSlider, QCheckBox,
)
from PyQt6.QtGui import QPainter, QColor, QPen, QFont, QLinearGradient
from core.dialogs import show_info, show_success, show_warning, DetailDialog
class GaugeWidget(QWidget):
    def __init__(self, label="", max_val=100, color="#4fc3f7"):
        super().__init__()
        self._label = label; self._max = max_val; self._color = color; self._val = 0
        self.setMinimumSize(100, 80)
    def set_value(self, v): self._val = v; self.update()
    def paintEvent(self, e):
        painter = QPainter(self)
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)
        w,h = self.width(), self.height()
        if w < 20 or h < 20: painter.end(); return
        painter.fillRect(self.rect(), QColor("#0b1426"))
        cx, cy, r = w//2, h//2-10, min(w,h)//2-15
        painter.setPen(QPen(QColor("#1a3060"), 6))
        painter.drawArc(int(cx-r), int(cy-r), int(r*2), int(r*2), 180*16, 180*16)
        angle = int(180 * self._val / self._max)
        painter.setPen(QPen(QColor(self._color), 6))
        painter.drawArc(int(cx-r), int(cy-r), int(r*2), int(r*2), 180*16, angle*16)
        painter.setPen(QColor(self._color))
        painter.setFont(QFont("Microsoft YaHei", 18, QFont.Weight.Bold))
        painter.drawText(int(cx-30), int(cy+8), f"{self._val:.0f}")
        painter.setPen(QColor("#8fa8c8"))
        painter.setFont(QFont("Microsoft YaHei", 8))
        painter.drawText(int(cx-30), int(cy-15), self._label)
        painter.end()
class LogCollectorModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._collecting = True; self._count = 0; self._errors = 0; self._rate = 0
        self._sources = {s:True for s in ["Syslog(514)", "应用日志", "数据库审计", "网络设备", "
安全日志"]}
        self._build_ui()
        self._timer = QTimer(self); self._timer.timeout.connect(self._tick);
self._timer.start(1500)
    def _build_ui(self):
        layout = QVBoxLayout(self); layout.setContentsMargins(16,12,16,12)
        # Title bar
        title_row = QHBoxLayout()
        title_row.addWidget(QLabel("🔍 采集控制台", styleSheet="font-size:16px;font-
weight:bold;color:#4fc3f7;"))
        self._status_btn = QPushButton("● 运行中");
self._status_btn.setStyleSheet("color:#81c784;font-weight:bold;padding:6px 18px;")
        self._status_btn.clicked.connect(self._toggle)
        title_row.addStretch(); title_row.addWidget(self._status_btn)
        layout.addLayout(title_row)

```

```

        # Gauges row
        gauge_row = QHBoxLayout()
        self._gauge_rate = GaugeWidget("条/秒", 50, "#4fc3f7");
        gauge_row.addWidget(self._gauge_rate)
        self._gauge_total = GaugeWidget("总采集(万)", 100, "#81c784");
        gauge_row.addWidget(self._gauge_total)
        self._gauge_err = GaugeWidget("错误率%", 10, "#ff5252");
        gauge_row.addWidget(self._gauge_err)
        self._gauge_storage = GaugeWidget("存储MB", 200, "#ffab40");
        gauge_row.addWidget(self._gauge_storage)
        layout.addLayout(gauge_row)
        # Source toggles
        src_box = QGroupBox("采集源控制")
        src_grid = QHBoxLayout()
        self._src_toggles = {}
        for name in self._sources:
            cb = QCheckBox(name); cb.setChecked(True);
            cb.setStyleSheet("color:#e8edf5;padding:8px;")
            cb.stateChanged.connect(lambda st, n=name: self._toggle_source(n, st))
            self._src_toggles[name] = cb; src_grid.addWidget(cb)
        src_box.setLayout(src_grid); layout.addWidget(src_box)
        # Control buttons
        ctrl_row = QHBoxLayout()
        for t,s in [("⏏ 一键启动全部", self._start_all), ("■ 停止全部", self._stop_all),
                  ("📊 采集统计", self._stats), ("⚙ 采集配置", self._config),
                  ("🔄 重置计数器", self._reset)]:
            b = QPushButton(t); b.clicked.connect(s)
            if "启动" in t: b.setProperty("primary", True)
            ctrl_row.addWidget(b)
        ctrl_row.addStretch()
        layout.addLayout(ctrl_row)
        # Status cards
        stat_row = QHBoxLayout()
        self._status_labels = {}
        for name in self._sources:
            card = QFrame(); card.setStyleSheet("QFrame{background:#121e3a;border:1px
            solid #2a4070;border-radius:8px;}")
            cl = QVBoxLayout(card); cl.setContentsMargins(10,8,10,8)
            cl.addWidget(QLabel(name,styleSheet="color:#8fa8c8;font-size:10px;"))
            lbl = QLabel(" 正常"); lbl.setStyleSheet("color:#81c784;font-weight:bold;")
            cl.addWidget(lbl); self._status_labels[name] = lbl; stat_row.addWidget(card)
        layout.addLayout(stat_row)
        def _toggle(self):
            self._collecting = not self._collecting
            self._status_btn.setText("● 运行中" if self._collecting else "○ 已暂停")
            self._status_btn.setStyleSheet("color:#81c784;font-weight:bold;padding:6px 18px;" if
            self._collecting else "color:#ffab40;font-weight:bold;padding:6px 18px;")
        def _toggle_source(self, name, state):
            self._sources[name] = state == 2

```

```

        self._status_labels[name].setText(" 正常" if state == 2 else "● 已关闭")
        self._status_labels[name].setStyleSheet("color:#81c784;font-weight:bold;" if state == 2
else "color:#ff5252;font-weight:bold;")
    def _start_all(self):
        for cb in self._src_toggles.values(): cb.setChecked(True)
        self._collecting = True; show_success(self,"启动","全部采集源已启动")
    def _stop_all(self):
        for cb in self._src_toggles.values(): cb.setChecked(False)
        self._collecting = False; show_warning(self,"停止","全部采集源已停止")
    def _stats(self):
        lines=[f"采集总量: {self._count}条",f"错误数: {self._errors}",f"采集速率: {self._rate}条
/秒",
                f"运行时间: 2天3小时",f"活跃源: {sum(1 for v in self._sources.values() if
v)}/5"]
        DetailDialog("统计","\n".join(lines),self).exec()
    def _config(self):
        show_info(self,"采集配置","缓冲大小: 1000条\n超时时间: 30秒\n重试次数: 3次\n编
码: UTF-8")
    def _reset(self):
        self._count=0; self._errors=0; self._rate=0; show_success(self,"重置","计数器已清零")
    def _tick(self):
        if not self._collecting: return
        self._rate = random.randint(10,45); self._count += self._rate
        if random.random()<0.15: self._errors += 1
        self._gauge_rate.set_value(self._rate)
        self._gauge_total.set_value(min(100, self._count/1000))
        self._gauge_err.set_value(min(10, self._errors/max(1,self._count)*1000))
        self._gauge_storage.set_value(min(200, self._count*0.15))
# ==== modules/log_parser.py (109行) ====
import random
from datetime import datetime
from PyQt6.QtCore import Qt
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QSplitter, QFrame, QComboBox, QTextEdit,
    QListWidget, QListWidgetItem, QSlider, QFormLayout, QCheckBox,
)
from PyQt6.QtGui import QColor, QFont
from core.dialogs import show_info, show_success, DetailDialog
class LogParserModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._build_ui()
    def _build_ui(self):
        layout = QVBoxLayout(self); layout.setContentsMargins(16,12,16,12)
        title_row = QHBoxLayout()
        title_row.addWidget(QLabel("🔍 解析配置台", styleSheet="font-size:16px;font-
weight:bold;color:#4fc3f7;"))
        self._status = QLabel("✅ 引擎就绪"); self._status.setStyleSheet("color:#81c784;font-

```

```

weight:bold;")
    title_row.addStretch(); title_row.addWidget(self._status)
    layout.addLayout(title_row)
    # Template & config
    config_row = QHBoxLayout()
    tpl_box = QGroupBox("解析模板"); tf = QFormLayout(tpl_box)

    self._tpl = QComboBox(); self._tpl.addItem("Syslog (RFC 5424)","JSON日志",
    "Apache访问日志","Nginx错误日志","自定义正则"])

    tf.addRow("选择模板", self._tpl)

    self._auto_parse = QCheckBox("采集后自动解析"); self._auto_parse.setChecked(True)
    tf.addRow("", self._auto_parse)
    config_row.addWidget(tpl_box)

    field_box = QGroupBox("字段映射"); fl = QVBoxLayout(field_box)

    fields = [("timestamp","时间戳","✓"),("level","级别","✓"),("module","模块","✓"),
              ("thread","线程","✓"),("request_id","请求ID","✓"),("message","消息",
    "✓")]

    for f, label, status in fields:
        row = QHBoxLayout()
        row.addWidget(QLabel(f"{status} {label}", styleSheet="color:#e8edf5;"))
        row.addWidget(QLabel(f, styleSheet="color:#8fa8c8;font-size:9px;"))
        row.addStretch(); fl.addLayout(row)
    config_row.addWidget(field_box)
    layout.addLayout(config_row)

    # Action panel
    action_panel = QFrame()
    action_panel.setStyleSheet("QFrame{background:#121e3a;border:1px solid
    #2a4070;border-radius:10px;}")
    ap = QHBoxLayout(action_panel)

    for t,s in [(▶ 执行解析, self._parse), (🔍 测试解析, self._test),
              (📊 解析统计, self._stats), (🔍 错误模式, self._patterns),
              (📄 导出结果, self._export)]:
        b = QPushButton(t); b.clicked.connect(s)
        if "执行" in t: b.setProperty("primary", True)
        ap.addWidget(b)
    layout.addWidget(action_panel)

    # Pipeline visualization
    pipe_box = QGroupBox("解析流水线")
    pipe = QHBoxLayout()
    stages = [
        (📄\n原始日志, "#4fc3f7"), (🔍\n分词提取, "#81c784"),
        (📄\n字段映射, "#ffab40"), (✓\n格式校验, "#ce93d8"),
        (📦\n结构化输出, "#4dd0e1"),
    ]

    for i, (label, color) in enumerate(stages):
        stage = QFrame(); stage.setStyleSheet(f"QFrame{{background:{color}22;border:2px
    solid {color};border-radius:10px;}}")
        sl = QVBoxLayout(stage); sl.setContentsMargins(14,12,14,12)
        sl.addWidget(QLabel(label,styleSheet=f"color:{color};font-weight:bold;font-

```

```

size:11px;",alignment=Qt.AlignmentFlag.AlignCenter))
        if i < len(stages)-1:
            arrow = QLabel(" → ", styleSheet="color:#4fc3f7;font-size:20px;")
            pipe.addWidget(arrow)
        pipe.addWidget(stage, 1)
    pipe_box.setLayout(pipe); layout.addWidget(pipe_box)
    # Output preview
    out_box = QGroupBox("解析输出预览")
    self._output = QTextEdit()
    self._output.setReadOnly(True); self._output.setPlaceholderText("解析结果将在此显示...")

    self._output.setMinimumHeight(100)
    out_box.setLayout(QVBoxLayout()); out_box.layout().addWidget(self._output)
    layout.addWidget(out_box, 1)
    def _parse(self):
        lines = ["=== 批量解析完成 ===", f"模板: {self._tpl.currentText()}", f"时间: {datetime.now().strftime('%H:%M:%S')}",
            "", "成功解析: 156条", "解析失败: 2条", "提取字段: 6个", "平均耗时: 0.3ms/条"]
        self._output.setText("\n".join(lines))
        self._status.setText("✓ 解析完成")
        show_success(self,"解析","156条日志已解析完成")
    def _test(self):
        lines = ["=== 测试解析 ===", f"原始: 2026-06-23 10:15:23 [auth] ERROR - 认证失败",
            "", "→ 时间戳: 2026-06-23 10:15:23", "→ 级别: ERROR", "→ 模块: auth",
            "→ 消息: 认证失败"]
        self._output.setText("\n".join(lines))
        show_info(self,"测试","单条解析测试通过")
    def _stats(self):
        lines = ["解析统计", f"总处理: 12,580条", f"成功率: 98.7%", f"平均吞吐: 850条/秒",
            f"字段覆盖率: 92%", f"异常率: 1.2%"]
        DetailDialog("统计","\n".join(lines),self).exec()
    def _patterns(self):
        lines = ["错误模式 TOP5:", "", "超时: 45次 ██████████", "拒绝连接: 32次 ██████████",
            "空指针: 18次 ████████", "权限不足: 12次 █████", "OOM: 5次 █"]
        DetailDialog("错误模式","\n".join(lines),self).exec()
    def _export(self):
        show_success(self,"导出","解析结果已导出为JSON格式")

# ==== modules/anomaly_detection.py (142行) ====
import random
from datetime import datetime
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QFrame, QComboBox, QSlider, QTextEdit, QProgressBar,

```

```

)
from PyQt6.QtGui import QColor, QFont
from core.dialogs import show_info, show_success, DetailDialog
class AnomalyDetectionModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._detecting = True; self._count = 0; self._anomaly_count = 0
        self._build_ui()
        self._timer = QTimer(self)
        self._timer.timeout.connect(self._tick)
        self._timer.start(2000)
    def _build_ui(self):
        layout = QVBoxLayout(self); layout.setContentsMargins(16,12,16,12)
        title_row = QHBoxLayout()
        title_row.addWidget(QLabel("🔍 检测控制台", styleSheet="font-size:16px;font-weight:bold;color:#4fc3f7;"))
        self._run_btn = QPushButton("■ 停止检测")
        self._run_btn.clicked.connect(self._toggle)
        self._run_btn.setStyleSheet("color:#ffab40;font-weight:bold;padding:6px 18px;")
        title_row.addStretch(); title_row.addWidget(self._run_btn)
        layout.addLayout(title_row)
        # Score display using progress bar
        score_box = QGroupBox("实时异常分数")
        score_layout = QVBoxLayout(score_box)
        self._score_bar = QProgressBar()
        self._score_bar.setRange(0, 100)
        self._score_bar.setValue(0)
        self._score_bar.setFormat("当前分数: %p/100")
        self._score_bar.setFixedHeight(30)
        score_layout.addWidget(self._score_bar)
        score_sub = QHBoxLayout()
        score_sub.addWidget(QLabel("阈值线: 65", styleSheet="color:#ffab40;font-weight:bold;"))
        score_sub.addStretch()
        score_sub.addWidget(QLabel("正常 ←", styleSheet="color:#4fc3f7;"))
        score_sub.addWidget(QLabel("|", styleSheet="color:#ffab40;"))
        score_sub.addWidget(QLabel("→ 异常", styleSheet="color:#ff5252;"))
        score_layout.addLayout(score_sub)
        layout.addWidget(score_box)
        # Control row
        ctrl_row = QHBoxLayout()
        param_panel = QFrame()
        param_panel.setStyleSheet("QFrame{background:#121e3a;border:1px solid #2a4070;border-radius:10px;}")
        pp = QVBoxLayout(param_panel); pp.setContentsMargins(14,12,14,12)
        pp.addWidget(QLabel("检测参数", styleSheet="color:#4fc3f7;font-weight:bold;font-size:13px;"))
        pp.addWidget(QLabel("检测算法:"))
        self._algo_cb = QComboBox()
        self._algo_cb.addItem("阈值检测","Z-Score","滑动窗口","集成检测")

```

```

pp.addWidget(self._algo_cb)
pp.addWidget(QLabel("灵敏度阈值:"))
self._th_slider = QSlider(Qt.Orientation.Horizontal)
self._th_slider.setRange(10, 100); self._th_slider.setValue(65)
self._th_lbl = QLabel("阈值: 65", styleSheet="color:#4fc3f7;font-weight:bold;")
self._th_slider.valueChanged.connect(lambda v: self._th_lbl.setText(f"阈值: {v}"))
pp.addWidget(self._th_slider); pp.addWidget(self._th_lbl)
pp.addWidget(QLabel("窗口大小:"))
self._win_slider = QSlider(Qt.Orientation.Horizontal)
self._win_slider.setRange(10, 200); self._win_slider.setValue(60)
pp.addWidget(self._win_slider)
ctrl_row.addWidget(param_panel)
# Status cards
stat_panel = QFrame()
stat_panel.setStyleSheet("QFrame{background:#121e3a;border:1px solid
#2a4070;border-radius:10px;}")
sp = QVBoxLayout(stat_panel); sp.setContentsMargins(14,12,14,12)
sp.addWidget(QLabel("检测统计", styleSheet="color:#4fc3f7;font-weight:bold;font-
size:13px;"))
for label, init, color in [("已检测", "0次", "#4fc3f7"), ("异常发现", "0", "#ff5252"), ("误报率", "0%", "#ffab40")]:
    row = QHBoxLayout()
    row.addWidget(QLabel(label, styleSheet="color:#8fa8c8;"))
    v = QLabel(init); v.setStyleSheet(f"color:{color};font-size:16px;font-weight:bold;")
    row.addStretch(); row.addWidget(v)
    sp.addLayout(row)
    self.__setattr__(f"_lbl_{label[:2]}", v)
ctrl_row.addWidget(stat_panel)
layout.addLayout(ctrl_row)
# Buttons
for buttons in [
    [("▶ 立即检测", self._detect_now), ("📄 检测报告", self._report),
     ("⚙️ 高级配置", self._config), ("📤 导出结果", self._export)],
]:
    r = QHBoxLayout()
    for t, s in buttons:
        b = QPushButton(t); b.clicked.connect(s)
        if "立即" in t: b.setProperty("primary", True)
        r.addWidget(b)
    layout.addLayout(r)
self._log = QTextEdit()
self._log.setReadOnly(True); self._log.setMaximumHeight(70)
self._log.setText("[就绪] 异常检测引擎已加载")
layout.addWidget(self._log)
def _toggle(self):
    self._detecting = not self._detecting
    self._run_btn.setText("■ 停止检测" if self._detecting else "▶ 启动检测")
    style = "color:#ffab40;font-weight:bold;" if self._detecting else "color:#81c784;font-
weight:bold;"
    self._run_btn.setStyleSheet(style)

```

```

def _detect_now(self):
    score = random.randint(0, 100)
    self._score_bar.setValue(score)
    self._count += 1
    th = self._th_slider.value()
    is_anomaly = score > th
    if hasattr(self, '_lbl_已检') and self._lbl_已检:
        self._lbl_已检.setText(f"{self._count}次")
    if is_anomaly:
        self._anomaly_count += 1
        if hasattr(self, '_lbl_异常') and self._lbl_异常:
            self._lbl_异常.setText(f"{self._anomaly_count}次")
        self._log.append(f"[{datetime.now().strftime('%H:%M:%S')}]      △      异常
score={score} th={th}")
    else:
        self._log.append(f"[{datetime.now().strftime('%H:%M:%S')}]      ✓      正常
score={score}")
def _report(self):
    atext = str(self._anomaly_count)
    lines = ["检测报告", f"总检测: {self._count}次", f"异常: {atext}次",
            "准确率: 92.3%", "误报率: 3.1%"]
    DetailDialog("报告","\n".join(lines),self).exec()
def _config(self):
    show_info(self,"高级配置","基线模型: 动态学习\n自适应阈值: 开启\n告警延迟: 5
秒\n抑制周期: 300秒")
def _export(self):
    show_success(self,"导出","检测结果已导出")
def _tick(self):
    if not self._detecting: return
    self._detect_now()
# ==== modules/log_search.py (257行) ====
import random, re, time
from datetime import datetime
from collections import deque
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QLineEdit, QTextEdit, QSplitter, QListWidget,
    QListWidgetItem, QComboBox, QSpinBox, QCheckBox,
)
from PyQt6.QtGui import QPainter, QColor, QPen, QFont
from core.dialogs import show_info, show_success, DetailDialog
class SearchTimeline(QWidget):
    def __init__(self, parent=None):
        super().__init__()
        self._data = deque([0]*24, maxlen=24)
        self.setMinimumHeight(80)
    def add_hit(self, count: int):
        self._data.append(count)

```

```

        self.update()
    def paintEvent(self, event):
        painter = QPainter(self)
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)
        w, h = self.width(), self.height()
        painter.fillRect(self.rect(), QColor("#121e3a"))
        if not self._data:
            painter.end(); return
        max_val = max(self._data) or 1
        bar_w = (w - 20) / len(self._data)
        painter.setPen(Qt.PenStyle.NoPen)
        for i, v in enumerate(self._data):
            bh = (v / max_val) * (h - 20)
            color = QColor(79, 195, 247, 180) if v < max_val * 0.8 else QColor("#ff5252")
            painter.setBrush(color)
            painter.drawRect(int(10 + i * bar_w), int(h - 10 - bh), int(bar_w - 2), int(bh))
        painter.setPen(QColor("#8fa8c8"))
        painter.setFont(QFont("Microsoft YaHei", 8))
        painter.drawText(2, h - 2, "过去24小时搜索结果分布")
        painter.end()
class LogSearchModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._results = []
        self._history = []
        self._build_ui()
    def _build_ui(self):
        layout = QVBoxLayout(self)
        layout.setContentsMargins(16, 12, 16, 12)
        # Search bar
        search_row = QHBoxLayout()
        self._search_input = QLineEdit()
        self._search_input.setPlaceholderText('输入搜索关键词, 支持 AND OR NOT 语法,
例如: error AND database NOT mysql')
        self._search_input.setMinimumHeight(40)
        self._search_input.returnPressed.connect(self._do_search)
        search_row.addWidget(self._search_input, 1)
        for text, slot in [("🔍 搜索", self._do_search), ("🔄 实时Tail", self._toggle_tail), ("🗑️
清空", self._clear)]:
            b = QPushButton(text)
            b.setMinimumHeight(40)
            if text == "🔍 搜索": b.setProperty("primary", True)
            b.clicked.connect(slot)
            search_row.addWidget(b)
        layout.addLayout(search_row)
        # Filters row
        filter_row = QHBoxLayout()
        filter_row.addWidget(QLabel("时间范围:"))
        self._time_combo = QComboBox()
        self._time_combo.addItem("过去15分钟", "过去1小时", "过去6小时", "过去24小时

```

```

", "过去7天", "全部时间"])
    filter_row.addWidget(self._time_combo)
    filter_row.addWidget(QLabel("级别:"))
    self._level_combo = QComboBox()
    self._level_combo.addItem("全部", "ERROR", "WARN", "INFO", "DEBUG", "FATAL")
    filter_row.addWidget(self._level_combo)
    filter_row.addWidget(QLabel("最多结果:"))
    self._max_spin = QSpinBox(); self._max_spin.setRange(10, 500);
self._max_spin.setValue(100)
    filter_row.addWidget(self._max_spin)
    filter_row.addStretch()
    layout.addLayout(filter_row)
    # Timeline
    self._timeline = SearchTimeline()
    layout.addWidget(self._timeline)
    # Results area
    splitter = QSplitter(Qt.Orientation.Horizontal)
    self._results_area = QTextEdit()
    self._results_area.setReadOnly(True)
    self._results_area.setPlaceholderText("搜索结果将显示在这里...")
    splitter.addWidget(self._results_area)
    splitter.setStretchFactor(0, 3)
    splitter.setSizes([600, 200])
    right_panel = QWidget()
    rl = QVBoxLayout(right_panel)
    hist_box = QGroupBox("搜索历史")
    self._history_list = QListWidget()
    hist_box.setLayout(QVBoxLayout())
    hist_box.layout().addWidget(self._history_list)
    rl.addWidget(hist_box)
    stat_box = QGroupBox("结果统计")
    sl = QVBoxLayout(stat_box)
    self._lbl_total = QLabel("总结果数: 0")
    self._lbl_time = QLabel("搜索耗时: --")
    self._lbl_sources = QLabel("涉及来源: --")
    for lb in [self._lbl_total, self._lbl_time, self._lbl_sources]:
        lb.setStyleSheet("color: #4fc3f7; background: transparent;")
        sl.addWidget(lb)
    rl.addWidget(stat_box)
    splitter.addWidget(right_panel)
    layout.addWidget(splitter, 1)
    # Toolbar
    tool_row = QHBoxLayout()
    for text, slot in [("导出结果", self._export), ("加入知识库", self._add_to_kb),
                      ("生成搜索报表", self._gen_report), ("语法帮助",
self._syntax_help)]:
        b = QPushButton(text); b.clicked.connect(slot); tool_row.addWidget(b)
    tool_row.addStretch()
    layout.addLayout(tool_row)

```

```

self._tail_timer = QTimer(self)
self._tail_timer.timeout.connect(self._tail_tick)
self._is_tailing = False
self._log_msg("日志联机搜索已就绪, 请输入关键词开始搜索")
def _log_msg(self, msg):
    self._results_area.append(f"// {msg}")
def _do_search(self):
    query = self._search_input.text().strip()
    if not query:
        show_info(self, "提示", "请输入搜索关键词")
        return
    if query not in self._history:
        self._history.insert(0, query)
        self._history_list.insertItem(0, query)
        if self._history_list.count() > 20:
            self._history_list.takeItem(20)
    start = time.time()
    count = self._max_spin.value()
    results = []
    levels = {"全部": ["ERROR", "WARN", "INFO", "DEBUG", "FATAL"],
              "ERROR": ["ERROR"], "WARN": ["WARN"], "INFO": ["INFO"],
              "DEBUG": ["DEBUG"], "FATAL": ["FATAL"]}
    selected = levels.get(self._level_combo.currentText(), ["ERROR"])
    sources = ["auth-svc", "api-gw", "db-cluster", "cache-svc", "web-srv", "mq-consumer"]
    for i in range(min(count, 80)):
        ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        lv = random.choice(selected)
        src = random.choice(sources)
        msg_templates = {
            "ERROR": [f"数据库连接超时 (timeout={random.randint(5,30)}s)", f"请求处
理异常: {random.choice(['NullPointer', 'IndexOutOfBounds', 'IllegalArgument'])}",
                      f"上游服务返回{random.randint(500,503)}状态码", f"消息队列消
费失败 offset={random.randint(100,9999)}"],
            "WARN": [f"连接池使用率超过{random.randint(75,95)}%", f"响应时间超过
{random.randint(200,2000)}ms",
                     f"磁盘剩余空间不足{random.randint(5,20)}%", f"重试次数已达
{random.randint(3,5)}次"],
            "INFO": [f"请求处理完成 duration={random.randint(10,500)}ms", f"缓存刷新
成功 entries={random.randint(100,9999)}",
                     f"定时任务执行完毕耗时{random.randint(1,10)}s", f"健康检查通
过"],
            "DEBUG": [f"SQL执行计划: index_scan cost={random.randint(100,9999)}", f"
变量值解析 result={random.choice([True,False])}"],
            "FATAL": [f"系统内存溢出 OOM: {random.choice(['Java heap
space', 'Metaspace', 'Direct buffer'])}",
                      f"核心服务进程崩溃 signal={random.randint(6,11)}"],
        }

```

```

        msg = random.choice(msg_templates.get(lv, ["日志记录"]))
        line = f"[{ts}] [{src}] {lv} - {msg}"
        if query.lower() in line.lower():
            results.append({"时间": ts, "级别": lv, "来源": src, "内容": msg, "原始行":
line})

        elapsed = time.time() - start
        self._results = results
        self._timeline.add_hit(len(results))
        self._lbl_total.setText(f"总结果数: {len(results)}")
        self._lbl_time.setText(f"搜索耗时: {elapsed*1000:.1f}ms")
        self._lbl_sources.setText(f"涉及来源: {len(set(r['来源'] for r in results))} 个")
        self._results_area.clear()
        if results:
            self._results_area.append(f"===== 找到 {len(results)} 条结果 (耗时
{elapsed*1000:.1f}ms) =====\n")
            for r in results:
                tag =
{"ERROR": "✖", "WARN": "⚠", "INFO": "ℹ", "DEBUG": "🔍", "FATAL": "💀"}.get(r["级别"], "•")
                self._results_area.append(f"{tag} [{r['时间']}] [{r['来源']}] {r['级别']}")
                self._results_area.append(f"    {r['内容']}")
                self._results_area.append("")
            else:
                self._results_area.append("未找到匹配结果, 请尝试更宽松的关键词")
            show_success(self, "搜索完成", f"找到 {len(results)} 条匹配结果, 耗时
{elapsed*1000:.1f}ms")
            def _toggle_tail(self):
                if self._is_tailing:
                    self._tail_timer.stop()
                    self._is_tailing = False
                    self._log_msg("实时Tail已停止")
                else:
                    self._tail_timer.start(3000)
                    self._is_tailing = True
                    self._log_msg("实时Tail已启动, 每3秒刷新...")
                    show_info(self, "实时Tail", "正在实时追踪最新日志, 每3秒自动刷新")
            def _tail_tick(self):
                ts = datetime.now().strftime("%H:%M:%S")
                lv = random.choice(["INFO", "INFO", "INFO", "WARN", "ERROR"])
                src = random.choice(["api-gw", "web-srv", "db-cluster"])
                msg = {"INFO": "请求正常处理完成", "WARN": "响应延迟偏高", "ERROR": "调用远程服
务失败"}
                line = f"[{ts}] [{src}] {lv} - {msg.get(lv, '')}"
                self._results_area.insertPlainText(f">> {line}\n")
                scrollbar = self._results_area.verticalScrollBar()
                scrollbar.setValue(scrollbar.maximum())
            def _clear(self):
                self._results.clear()
                self._results_area.clear()

```

```

        self._log_msg("搜索结果已清空")
    def _export(self):
        lines = ["日志搜索结果导出", f"搜索关键词: {self._search_input.text()}", f"导出时间: {datetime.now()}", ""]
        for r in self._results:
            lines.append(r["原始行"])
        DetailDialog("导出结果", "\n".join(lines), self).exec()
        self._log_msg(f"已导出 {len(self._results)} 条结果")
    def _add_to_kb(self):
        show_info(self, "加入知识库", "可将搜索到的重要结果加入知识图谱模块作为知识条目")
        self._log_msg("已添加到知识库")
    def _gen_report(self):
        lines = ["搜索分析报告", f"关键词: {self._search_input.text()}", f"结果数: {len(self._results)}",
                f"搜索时间: {datetime.now()}", "", "级别分布:"]
        for lv in ["FATAL", "ERROR", "WARN", "INFO", "DEBUG"]:
            cnt = sum(1 for r in self._results if r["级别"]==lv)
            if cnt: lines.append(f" {lv}: {cnt}")
        DetailDialog("搜索报告", "\n".join(lines), self).exec()
    def _syntax_help(self):
        help_text = """搜索语法帮助:
AND 同时匹配多个词      error AND database
OR  匹配任意一个词      timeout OR timeout
NOT 排除某个词          error NOT mysql
精确匹配                  "connection timeout"
通配符                    db_*
字段筛选                  source:api-gw level:ERROR
示例:
(error OR fatal) AND database NOT mysql
level:WARN AND timeout
source:api-gw AND 500"""
        DetailDialog("搜索语法帮助", help_text, self).exec()
# ==== modules/fault_prediction.py (120行) ====
import random
from datetime import datetime, timedelta
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QFormLayout, QDoubleSpinBox, QComboBox,
    QTextEdit, QSplitter, QListWidget, QListWidgetItem, QProgressBar, QSpinBox, QFrame,
)
from PyQt6.QtGui import QColor, QFont
from core.dialogs import show_info, show_success, DetailDialog
class FaultPredictionModule(QWidget):

```

```

def __init__(self, parent=None):
    super().__init__(parent)
    self._preds = []
    self._build_ui()
    self._run_prediction()
    self._timer = QTimer(self)
    self._timer.timeout.connect(self._run_prediction)
    self._timer.start(5000)
def _build_ui(self):
    layout = QVBoxLayout(self); layout.setContentsMargins(16,12,16,12)
    splitter = QSplitter(Qt.Orientation.Horizontal)
    left = QWidget(); ll = QVBoxLayout(left)
    # Risk cards
    self._card_row = QHBoxLayout()
    self._risk_cards = []
    for label in ["高风险", "中风险", "低风险"]:
        card = QFrame()
        card.setStyleSheet("QFrame{background:#121e3a;border:1px solid
#2a4070;border-radius:10px;}")
        cl = QVBoxLayout(card); cl.setContentsMargins(12,10,12,10)
        cl.addWidget(QLabel(label, styleSheet="color:#8fa8c8;font-size:10px;"))
        val = QLabel("0"); val.setFont(QFont("Microsoft YaHei",22,QFont.Weight.Bold))
        val.setStyleSheet(f"color:{'#ff5252' if '高' in label else '#ffab40' if '中' in label else
'#4fc3f7'};")
        cl.addWidget(val, alignment=Qt.AlignmentFlag.AlignCenter)
        self._risk_cards.append((label,val))
        self._card_row.addWidget(card)
    ll.addLayout(self._card_row)
    # Prediction list
    self._pred_list = QListWidget()
    self._pred_list.setStyleSheet("QListWidget{background:#121e3a;border:1px solid
#2a4070;border-radius:8px;}")
    "QListWidget::item{padding:8px 12px;border-bottom:1px solid #1a3060;}")
    "QListWidget::item:hover{background:#1a3060;}")
    ll.addWidget(self._pred_list, 1)
    for buttons in [
        [("执行预测", self._run_prediction), ("趋势分析", self._trend), ("风险评分",
self._risk), ("导出", self._export)],
    ]:
        r = QHBoxLayout()
        for t, s in buttons:
            b = QPushButton(t); b.clicked.connect(s)
            if "执行" in t: b.setProperty("primary", True)
            r.addWidget(b)
        ll.addLayout(r)
    splitter.addWidget(left)
    right = QWidget(); rl = QVBoxLayout(right); rl.setContentsMargins(0,0,0,0)
    param_box = QGroupBox("预测参数")
    form = QFormLayout(param_box)
    self._th_spin = QDoubleSpinBox(); self._th_spin.setRange(50,100);
    self._th_spin.setValue(85); self._th_spin.setSuffix("%")

```

```

form.addRow("故障阈值", self._th_spin)
self._model_combo = QComboBox(); self._model_combo.addItem("线性回归","指数平滑","ARIMA","LSTM","集成")
form.addRow("模型", self._model_combo)
rl.addWidget(param_box)
stat_box = QGroupBox("状态")
sl = QVBoxLayout(stat_box)
self._lbl_acc = QLabel("准确率: --"); self._lbl_acc.setStyleSheet("color:#4fc3f7;")
self._lbl_last = QLabel("最后预测: --"); self._lbl_last.setStyleSheet("color:#8fa8c8;")
sl.addWidget(self._lbl_acc); sl.addWidget(self._lbl_last)
rl.addWidget(stat_box)
self._log = QTextEdit(); self._log.setReadOnly(True); self._log.setMaximumHeight(120)
rl.addWidget(self._log); rl.addStretch()
splitter.addWidget(right); layout.addWidget(splitter, 1)
def _run_prediction(self):
    self._pred_list.clear(); self._preds.clear()
    resources = [
        ("CPU使用率",85), ("内存使用率",85), ("磁盘IO等待",70), ("网络延迟",150),
        ("连接池使用率",90), ("错误率",5), ("JVM堆内存",90), ("GC暂停",500),
        ("线程阻塞数",20), ("缓存命中率",75), ("数据库QPS",6000), ("磁盘使用率",90),
    ]
    high = mid = low = 0
    for name, threshold in resources:
        cur = round(random.uniform(30,95),1)
        trend = random.uniform(-0.5, 1.5)
        risk = "低"
        if trend > 0.8: risk = "高"; high += 1
        elif trend > 0.3: risk = "中"; mid += 1
        else: low += 1
        self._preds.append((name, cur, trend, threshold, risk))
        icon = {"高": "●", "中": "○", "低": "□"}
        item = QListWidgetItem(f" {icon[risk]} {name} 当前:{cur}% 趋势:{trend:+.1f}/h 阈值:{threshold}")
        item.setForeground(QColor({"高": "#ff5252", "中": "#ffab40", "低": "#81c784"}[risk]))
        if risk == "高": item.setBackground(QColor("#2a0000"))
        self._pred_list.addItem(item)
    self._risk_cards[0][1].setText(str(high))
    self._risk_cards[1][1].setText(str(mid))
    self._risk_cards[2][1].setText(str(low))
    self._lbl_acc.setText(f"模型: {self._model_combo.currentText()} 置信度: {random.randint(82,95)}%")
    self._lbl_last.setText(f"最后预测: {datetime.now().strftime('%H:%M:%S')}")
    self._log.append(f"[预测] {len(resources)}项指标分析完成, {high}项高风险")
def _trend(self):
    lines = ["趋势分析", f"数据点: 30", f"当前值: {random.uniform(50,90):.1f}", f"斜率:

```

```

{random.uniform(-0.5,1.5):+.2f}/h"]
    DetailDialog("趋势","\n".join(lines),self).exec()
def _risk(self):
    score = random.uniform(30,95)
    lines = [f"综合风险评分: {score:.0f}/100", f"等级: {'高' if score>70 else '中' if score>40
else '低'}"]
    DetailDialog("风险","\n".join(lines),self).exec()
def _export(self):
    lines = ["故障预测报告",f"时间:{datetime.now()}",f"模型:{self._model_combo.currentText()}",f""]
    for name,cur,trend,th,risk in self._preds:
        lines.append(f"[{risk}] {name}: {cur}% 趋势{trend:+.1f}")
    DetailDialog("报告","\n".join(lines),self).exec()
# ==== modules/root_cause.py (124行) ====
import random
from datetime import datetime
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QTextEdit, QSplitter, QTreeWidget, QTreeWidgetItem,
    QListWidget, QListWidgetItem,
)
from PyQt6.QtGui import QColor
from core.dialogs import show_info, show_success, show_warning, DetailDialog
class RootCauseModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._incidents = []
        self._init_data()
        self._build_ui()
    def _init_data(self):
        data = [
            ("10:22:15","web_server","502 Bad Gateway","严重"),
            ("10:22:18","api_gateway","上游连接超时","严重"),
            ("10:22:20","auth_service","认证服务无响应","一般"),
            ("10:22:25","order_service","订单查询失败","严重"),
            ("10:22:30","database_primary","主库连接池耗尽","严重"),
            ("10:22:35","payment_service","支付回调超时","一般"),
        ]
        for ts,comp,desc,sev in data:
            self._incidents.append({"时间":ts,"组件":comp,"描述":desc,"严重度":sev})
    def _build_ui(self):
        layout = QVBoxLayout(self); layout.setContentsMargins(16,12,16,12)
        splitter = QSplitter(Qt.Orientation.Horizontal)
        left = QWidget(); ll = QVBoxLayout(left); ll.setContentsMargins(0,0,0,0)
        # Topology tree
        topo_box = QGroupBox("依赖拓扑")

```

```

self._tree = QTreeWidget()
self._tree.setHeaderLabels(["组件"])
self._tree.setStyleSheet("QTreeWidget{background:#121e3a;border:1px solid
#2a4070;border-radius:4px;color:#e8edf5;}")
    "QTreeWidget::item:selected{background:#1a3a6a;}")
root = QTreeWidgetItem(self._tree,["web_server"])
gw = QTreeWidgetItem(root,["api_gateway"])
for s in ["auth_service","user_service","order_service","payment_service"]:
    QTreeWidgetItem(gw,[s])
db = QTreeWidgetItem(root,["database_primary"])
QTreeWidgetItem(db,["storage_system"])
root.setExpanded(True); gw.setExpanded(True)
topo_box.setLayout(QVBoxLayout()); topo_box.layout().addWidget(self._tree)
ll.addWidget(topo_box)
# Result
result_box = QGroupBox("根因分析结果")
self._result = QTextEdit()
self._result.setReadOnly(True); self._result.setPlaceholderText("点击「根因追溯」分
析...")
result_box.setLayout(QVBoxLayout()); result_box.layout().addWidget(self._result)
ll.addWidget(result_box)
for buttons in [
    [("根因追溯", self._trace), ("关联分析", self._correlate), ("故障注入", self._inject),
("影响域", self._impact)],
    [("导出报告", self._export), ("生成因果图", self._causal), ("刷新", self._refresh), ("
构建拓扑", self._build_topo)],
]:
    r = QHBoxLayout()
    for t, s in buttons:
        b = QPushButton(t); b.clicked.connect(s)
        if "根因" in t: b.setProperty("primary", True)
        r.addWidget(b)
    ll.addLayout(r)
splitter.addWidget(left)
right = QWidget(); rl = QVBoxLayout(right); rl.setContentsMargins(0,0,0,0)
inc_box = QGroupBox("当前事件")
self._inc_list = QListWidget()
self._inc_list.setStyleSheet("QListWidget{background:#121e3a;border:1px solid
#2a4070;border-radius:4px;}")
    "QListWidget::item{padding:6px 10px;border-bottom:1px solid #1a3060;}")
for inc in self._incidents:
    item = QListWidgetItem(f"    [{inc['严重度']}] {inc['时间']} {inc['组件']}: {inc['描述
']}]")
    if inc["严重度"]=="严重": item.setForeground(QColor("#ff5252"));
item.setBackground(QColor("#2a0000"))
    self._inc_list.addItem(item)
inc_box.setLayout(QVBoxLayout()); inc_box.layout().addWidget(self._inc_list)
rl.addWidget(inc_box)
self._log = QTextEdit(); self._log.setReadOnly(True); self._log.setMaximumHeight(100)

```

```

        rl.addWidget(self._log); rl.addStretch()
        splitter.addWidget(right); layout.addWidget(splitter, 1)
    def _trace(self):
        lines = ["根因定位结果:", "", "● [根因] database_primary (影响分数: 1.65)",
                "    [可能原因] api_gateway (影响分数: 0.85)",
                "    ● [关联因素] auth_service (影响分数: 0.45)"]
        self._result.setText("\n".join(lines))
        self._log.append(f"[{datetime.now().strftime('%H:%M:%S')}] 根因追溯完成")
        show_info(self, "根因", "定位到3个根因候选, 最高置信度92%")
    def _correlate(self):
        lines = ["事件关联:", "    关联度 95%: 主库连接池耗尽 <- 订单查询失败",
                "    关联度 82%: 上游连接超时 <- 502 Bad Gateway"]
        DetailDialog("关联", "\n".join(lines), self).exec()
    def _inject(self):
        target = random.choice(["database_primary", "api_gateway", "auth_service"])
        self._incidents.insert(0, {"时间": datetime.now().strftime("%H:%M:%S"), "组件": target,
                                   "描述": f"故障注入: {target}异常", "严重度": "严重"})
        self._refresh(); self._log.append(f"[故障注入] {target}")
        show_warning(self, "注入", f"已向{target}注入模拟故障")
    def _impact(self):
        lines = ["影响域分析:", "    database_primary 故障影响:",
                "        auth_service - 认证服务无响应",
                "        order_service - 订单查询失败",
                "        payment_service - 支付回调超时"]
        DetailDialog("影响域", "\n".join(lines), self).exec()
    def _export(self):
        lines = ["根因分析报告"] + [f"[{'严重度'}] [{'时间'}] [{'组件'}]: [{'描述'}]" for i in
self._incidents]
        DetailDialog("报告", "\n".join(lines), self).exec()
    def _causal(self):
        lines = ["因果图:", "    database_primary 连接池耗尽", "        └─ auth_service 无响应",
                "    └─ order_service 失败", "        └─ api_gateway 超时", "            └─ web_server",
                "502"]
        DetailDialog("因果图", "\n".join(lines), self).exec()
    def _refresh(self):
        self._inc_list.clear()
        for inc in self._incidents:
            item = QListWidgetItem(f"    [{inc['严重度']}] [{inc['时间']}] [{inc['组件'}]: {inc['描述'}]")
            if inc["严重度"]=="严重":
                item.setForeground(QColor("#ff5252"));
            item.setBackground(QColor("#2a0000"))
            self._inc_list.addItem(item)
    def _build_topo(self):
        self._log.append("[拓扑] 依赖关系已更新")

```

```

# ==== modules/alarm_center.py (118行) ====
import random
from datetime import datetime
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QSplitter, QFrame, QComboBox, QSlider, QTextEdit, QCheckBox,
)
from PyQt6.QtGui import QColor, QFont
from core.dialogs import show_info, show_success, show_warning, DetailDialog
class AlarmCenterModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._p0=self._p1=self._p2=self._p3=0
        self._suppression = True; self._auto_resolve = False
        self._build_ui()
    def _build_ui(self):
        layout = QVBoxLayout(self); layout.setContentsMargins(16,12,16,12)
        title_row = QHBoxLayout()
        title_row.addWidget(QLabel("🔔 告警控制台", styleSheet="font-size:16px;font-weight:bold;color:#4fc3f7;"))
        self._status = QLabel("● 监控中"); self._status.setStyleSheet("color:#81c784;font-weight:bold;")
        title_row.addStretch(); title_row.addWidget(self._status)
        layout.addLayout(title_row)
        # Severity matrix
        sev_row = QHBoxLayout()
        sev_data = [
            ("P0 灾难", self._p0, "#ff1744", "#4a0000"),
            ("P1 严重", self._p1, "#ff5252", "#2a0000"),
            ("P2 一般", self._p2, "#ffab40", "#2a2000"),
            ("P3 提示", self._p3, "#4fc3f7", "#002040"),
        ]
        self._sev_labels = {}
        for label, val, color, bg in sev_data:
            card = QFrame(); card.setStyleSheet(f"QFrame{{background:{bg};border:2px solid {color};border-radius:12px;}}")
            cl = QVBoxLayout(card); cl.setContentsMargins(16,14,16,14)
            cl.addWidget(QLabel(label,styleSheet=f"color:{color};font-weight:bold;font-size:11px;",alignment=Qt.AlignmentFlag.AlignCenter))
            v = QLabel("0"); v.setStyleSheet(f"color:{color};font-size:32px;font-weight:bold;")
            v.setAlignment(Qt.AlignmentFlag.AlignCenter)
            self._sev_labels[label[:2]] = v; cl.addWidget(v)
            sev_row.addWidget(card, 1)
        layout.addLayout(sev_row)
        # Controls
        ctrl_panel = QFrame();
        ctrl_panel.setStyleSheet("QFrame{background:#121e3a;border:1px solid #2a4070;border-radius:10px;}")
        cp = QHBoxLayout(ctrl_panel)
        left_ctrl = QVBoxLayout()

```

```

        left_ctrl.addWidget(QLabel("告警策略",styleSheet="color:#4fc3f7;font-weight:bold;"))
        self._sup_cb = QCheckBox("启用告警抑制"); self._sup_cb.setChecked(True);
left_ctrl.addWidget(self._sup_cb)
        self._auto_cb = QCheckBox("自动解决已恢复告警");
left_ctrl.addWidget(self._auto_cb)
        cp.addLayout(left_ctrl)
        mid_ctrl = QVBoxLayout()
        mid_ctrl.addWidget(QLabel("抑制窗口:",styleSheet="color:#8fa8c8;"))
        self._win_slider = QSlider(Qt.Orientation.Horizontal);
self._win_slider.setRange(30,600); self._win_slider.setValue(300)
        self._win_lbl = QLabel("300秒"); self._win_slider.valueChanged.connect(lambda v:
self._win_lbl.setText(f"{v}秒"))
        mid_ctrl.addWidget(self._win_slider); mid_ctrl.addWidget(self._win_lbl)
        cp.addLayout(mid_ctrl)
        right_ctrl = QVBoxLayout()
        right_ctrl.addWidget(QLabel("通知方式:",styleSheet="color:#8fa8c8;"))
        for t,ck in [("邮件",True),("企业微信",True),("钉钉",False),("短信",False)]:
            cb = QCheckBox(t); cb.setChecked(ck); right_ctrl.addWidget(cb)
        cp.addLayout(right_ctrl)
        layout.addWidget(ctrl_panel)
        # Action buttons
        for buttons in [
            [("🔄 模拟告警", self._sim_alarm), ("✅ 全部确认", self._ack_all),
            ("🧹 清理已处理", self._clear), ("📊 告警统计", self._stats)],
            [("🔇 静默模式", self._silence), ("📤 导出告警", self._export),
            ("⚙️ 规则配置", self._rules), ("📖 告警日志", self._audit)],
        ]:
            r = QHBoxLayout()
            for t, s in buttons:
                b = QPushButton(t); b.clicked.connect(s)
                if "模拟" in t: b.setProperty("primary", True)
                r.addWidget(b)
            layout.addLayout(r)
        self._log = QTextEdit(); self._log.setReadOnly(True); self._log.setMaximumHeight(80)
        self._log.setText("[就绪] 告警管理中心已加载")
        layout.addWidget(self._log)
def _sim_alarm(self):
    import random as rnd
    levels = ["P0",self._p0,"#ff1744"),("P1",self._p1,"#ff5252"),("P2",self._p2,"#ffab40"),("P3",self._p3,"#4fc3f7"))]
    lv, lbl, color = rnd.choice(levels)
    setattr(self, f"_{lv}", getattr(self, f"_{lv}")+1)
    self._sev_labels[lv].setText(str(getattr(self, f"_{lv}")))
    self._log.append(f"[{datetime.now().strftime('%H:%M:%S')}] {lv}级告警已接收 {rnd.choice(['连接超时','CPU过载','内存告警','磁盘告警'])}")
    show_info(self,"告警",f"已注入{lv}级模拟告警")

```

```

def _ack_all(self):
    self._log.append(f"[{datetime.now().strftime('%H:%M:%S')}] ✓ 全部告警已确认")
def _clear(self):
    for lv in ["p0", "p1", "p2", "p3"]:
        setattr(self, f"_{lv}", 0); self._sev_labels[lv].setText("0")
    self._log.append("[已清理]")
    show_success(self, "清理", "告警计数器已清零")
def _stats(self):
    lines = ["当前告警统计:", "", f"P0灾难: {self._p0} 次", "P1严重: {self._p1} 次",
            f"P2一般: {self._p2} 次", f"P3提示: {self._p3} 次", "", "抑制率: 85%", "平均响
应: 2.5分钟"]
    DetailDialog("统计", "\n".join(lines), self).exec()
def _silence(self):
    show_info(self, "静默", "已开启静默模式, 期间告警仅记录不通知")
def _export(self):
    show_success(self, "导出", "告警数据已导出")
def _rules(self):
    show_info(self, "规则", "1. P0/P1 即时通知\n2. P2 5分钟聚合\n3. P3 日报汇总\n4. 同
源告警30分钟内合并")
def _audit(self):
    lines = ["告警审计日志:", "10:23 P1 数据库连接超时 已确认",
            "10:25 P0 核心服务宕机 已升级", "10:30 P2 磁盘告警 已处理"]
    DetailDialog("审计", "\n".join(lines), self).exec()
# ==== modules/service_topology.py (179行) ====
import random, math
from datetime import datetime
from PyQt6.QtCore import Qt, QTimer, QPointF, QRectF
from PyQt6.QtGui import QPainter, QColor, QPen, QBrush, QRadialGradient, QFont
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QTextEdit, QSplitter, QTreeWidget, QTreeWidgetItem,
    QListWidget, QListWidgetItem,
)
from core.dialogs import show_info, show_success, show_warning, DetailDialog
class TopoCanvas(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setMinimumHeight(350)
        self._phase = 0.0
        self._nodes = [
            {"name": "负载均衡", "x": 0.5, "y": 0.08, "status": 1.0, "type": "lb"},
            {"name": "Web服务器", "x": 0.2, "y": 0.25, "status": 0.95, "type": "web"},
            {"name": "API网关", "x": 0.5, "y": 0.25, "status": 0.98, "type": "gw"},
            {"name": "Web服务器2", "x": 0.8, "y": 0.25, "status": 0.90, "type": "web"},
            {"name": "用户服务", "x": 0.1, "y": 0.5, "status": 0.92, "type": "svc"},
            {"name": "订单服务", "x": 0.35, "y": 0.5, "status": 0.85, "type": "svc"},

```

```

        {"name": "支付服务", "x": 0.6, "y": 0.5, "status": 0.78, "type": "svc"},
        {"name": "认证服务", "x": 0.85, "y": 0.5, "status": 0.95, "type": "svc"},
        {"name": "主数据库", "x": 0.25, "y": 0.75, "status": 0.88, "type": "db"},
        {"name": "缓存集群", "x": 0.55, "y": 0.75, "status": 0.92, "type": "cache"},
        {"name": "消息队列", "x": 0.8, "y": 0.75, "status": 0.96, "type": "mq"},
        {"name": "对象存储", "x": 0.5, "y": 0.92, "status": 0.99, "type": "storage"},
    ]
    self._edges = [(0,1),(0,2),(0,3),(2,4),(2,5),(2,6),(2,7),(4,8),(5,8),(6,9),(5,9),(7,8),(8,10),(9,10),(10,11)]
    self._timer = QTimer(self)
    self._timer.timeout.connect(lambda: setattr(self, '_phase', self._phase+0.03) or self.update())
    self._timer.start(40)
    def paintEvent(self, event):
        painter = QPainter(self)
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)
        w, h = self.width(), self.height()
        painter.fillRect(self.rect(), QColor("#0a1628"))
        # Draw edges
        for i, j in self._edges:
            n1, n2 = self._nodes[i], self._nodes[j]
            x1, y1 = n1["x"]*w, n1["y"]*h
            x2, y2 = n2["x"]*w, n2["y"]*h
            status = (n1["status"] + n2["status"]) / 2
            alpha = int(80 + 120 * status)
            painter.setPen(QPen(QColor(79, 195, 247, alpha), 2))
            painter.drawLine(int(x1), int(y1), int(x2), int(y2))
            t = (self._phase * 0.5) % 1.0
            px = x1 + (x2-x1)*t; py = y1 + (y2-y1)*t
            painter.setBrush(QColor(129, 212, 250, 200))
            painter.setPen(Qt.PenStyle.NoPen)
            painter.drawEllipse(QPointF(px, py), 4, 4)
        # Draw nodes
        for n in self._nodes:
            cx, cy = int(n["x"]*w), int(n["y"]*h)
            r = 24 if n["type"] in ("db", "lb") else 20
            color_map = {"lb": "#4fc3f7", "web": "#81c784", "gw": "#ffab40", "svc": "#ce93d8",
                "db": "#ef5350", "cache": "#ffd740", "mq": "#4dd0e1", "storage": "#8d6e63"}
            base_color = QColor(color_map.get(n["type"], "#666"))
            pulse = 0.7 + 0.3 * n["status"]
            painter.setBrush(QColor(base_color.red(), base_color.green(), base_color.blue(),
                int(180*pulse)))
            painter.setPen(QPen(base_color.lighter(130), 2))
            painter.drawEllipse(cx-r, cy-r, r*2, r*2)
            painter.setPen(QColor("#e8edf5"))
            painter.setFont(QFont("Microsoft YaHei", 8))
            painter.drawText(QRectF(cx-r-5, cy+8, r*2+10, 16), Qt.AlignmentFlag.AlignCenter,
                n["name"])
            painter.drawText(QRectF(cx-r-5, cy-12, r*2+10, 16), Qt.AlignmentFlag.AlignCenter,
                f"{n['status']*100:.0f}%")
            painter.end()

```

```

class ServiceTopologyModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._build_ui()
    def _build_ui(self):
        layout = QVBoxLayout(self)
        layout.setContentsMargins(16, 12, 16, 12)
        splitter = QSplitter(Qt.Orientation.Horizontal)
        self._canvas = TopoCanvas()
        splitter.addWidget(self._canvas)
        splitter.setStretchFactor(0, 3)
        splitter.setSizes([600, 200])
        right = QWidget()
        rl = QVBoxLayout(right)
        info_box = QGroupBox("节点详情")
        self._detail_text = QTextEdit()
        self._detail_text.setReadOnly(True)
        self._detail_text.setPlaceholderText("点击拓扑图中的节点查看详情...")
        info_box.setLayout(QVBoxLayout())
        info_box.layout().addWidget(self._detail_text)
        rl.addWidget(info_box)
        tree_box = QGroupBox("依赖层级")
        self._dep_tree = QTreeWidget()
        self._dep_tree.setHeaderLabels(["服务组件", "状态"])
        self._build_tree()
        tree_box.setLayout(QVBoxLayout())
        tree_box.layout().addWidget(self._dep_tree)
        rl.addWidget(tree_box)
        splitter.addWidget(right)
        layout.addWidget(splitter, 1)
        for buttons in [
            [("自动发现拓扑", self._auto_discover), ("刷新状态", self._refresh_status),
             ("导出拓扑数据", self._export), ("拓扑分析", self._analyze)],
            [("放大(+)", self._zoom_in), ("缩小(-)", self._zoom_out),
             ("重置视图", self._reset_view), ("全屏展示", self._fullscreen)],
        ]:
            r = QHBoxLayout()
            for text, slot in buttons:
                b = QPushButton(text)
                if text == "自动发现拓扑": b.setProperty("primary", True)
                b.clicked.connect(slot)
                r.addWidget(b)
            layout.addLayout(r)
    def _build_tree(self):
        self._dep_tree.clear()
        layers = [
            ("入口层", [("负载均衡 (LB)", " 正常")]),
            ("应用层", [("Web服务器集群", " 正常"), ("API网关", " 正常")]),
            ("服务层", [("用户服务", " 正常"), ("订单服务", " 延迟"), ("支付服务", "● 异

```

```

常"), ("认证服务", " 正常")),
        ("数据层", [("主数据库", " 连接池高"), ("缓存集群", " 正常"), ("消息队列", "
正常"))],
    ]
    for layer, items in layers:
        layer_item = QTreeWidgetItem(self._dep_tree, [layer, ""])
        for name, status in items:
            child = QTreeWidgetItem(layer_item, [name, status])
            if "●" in status: child.setForeground(1, QColor("#ff5252"))
            elif " " in status: child.setForeground(1, QColor("#ffab40"))
        layer_item.setExpanded(True)
    def _auto_discover(self):
        self._detail_text.setText(f"[{datetime.now().strftime('%H:%M:%S')}] 拓扑自动发现完
成\n\n发现 12 个服务节点\n检测到 15 条服务依赖关系\n服务健康度: 91.2%")
        show_success(self, "自动发现", "服务拓扑自动发现完成\n发现12个节点, 15条依赖
关系")
    def _refresh_status(self):
        for n in self._canvas._nodes:
            n["status"] = max(0.5, min(1.0, n["status"] + random.uniform(-0.05, 0.05)))
        self._canvas.update()
        self._detail_text.setText(f"[{datetime.now().strftime('%H:%M:%S')}] 状态已刷新")
    def _export(self):
        lines = ["服务拓扑数据", f"导出时间: {datetime.now()}", ""]
        for n in self._canvas._nodes:
            lines.append(f"{n['name']} (type={n['type']}) status={n['status']*100:.0f}%")
        DetailDialog("拓扑导出", "\n".join(lines), self).exec()
    def _analyze(self):
        unhealthy = [n for n in self._canvas._nodes if n["status"] < 0.8]
        lines = ["拓扑分析报告:", f"\n总节点数: {len(self._canvas._nodes)}",
                f"异常节点: {len(unhealthy)}", f"平均健康度: {sum(n['status'] for n in
self._canvas._nodes)/len(self._canvas._nodes)*100:.1f}%"]
        if unhealthy:
            lines.append("\n需关注节点:")
            for n in unhealthy: lines.append(f"  △ {n['name']} ({n['status']*100:.0f}%")
        DetailDialog("拓扑分析", "\n".join(lines), self).exec()
    def _zoom_in(self):
        self._detail_text.append("[缩放] 已放大")
    def _zoom_out(self):
        self._detail_text.append("[缩放] 已缩小")
    def _reset_view(self):
        self._detail_text.append("[视图] 已重置为默认视角")
    def _fullscreen(self):
        show_info(self, "全屏", "请使用窗口最大化按钮进入全屏展示模式")
# ==== modules/smart_diagnosis.py (207行) ====
import random
from datetime import datetime

```

```

from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QTextEdit, QSplitter, QListWidget, QListWidgetItem,
    QProgressBar, QFrame, QComboBox,
)
from PyQt6.QtGui import QColor
from core.dialogs import show_info, show_success, show_warning, DetailDialog
class DiagnosisEngine:
    @staticmethod
    def analyze(issue: str) -> list[dict]:
        issues_db = {
            "数据库连接超时": {"root_cause": "连接池耗尽/网络延迟", "confidence": 0.92,
                                "steps": ["检查连接池配置 maxActive=100", "查看数据库服务器
                                负载", "检查网络延迟 ping db-host",
                                "优化慢查询SQL", "考虑增加连接池上限到200"],
                                "severity": "严重", "fix_time": "15-30分钟"},
            "服务响应缓慢": {"root_cause": "CPU负载过高/内存不足", "confidence": 0.85,
                                "steps": ["检查CPU使用率 top -H", "检查内存使用情况 free -m", "
                                查看GC日志",
                                "分析线程堆栈 jstack", "考虑水平扩展实例数"],
                                "severity": "一般", "fix_time": "20-60分钟"},
            "磁盘空间告警": {"root_cause": "日志文件未轮转/数据增长", "confidence":
0.95,
                                "steps": ["检查磁盘使用率 df -h", "清理过期日志文件", "配置
                                logrotate按天切割",
                                "压缩归档历史数据", "设置磁盘告警阈值80%"],
                                "severity": "一般", "fix_time": "10-20分钟"},
            "内存泄漏": {"root_cause": "对象未正确释放/ThreadLocal", "confidence": 0.78,
                                "steps": ["获取heap dump分析", "检查ThreadLocal使用", "审查代码中
                                Map缓存使用",
                                "启用内存泄漏检测", "计划重启临时恢复"],
                                "severity": "严重", "fix_time": "1-4小时"},
            "网络分区": {"root_cause": "交换机故障/网线松动", "confidence": 0.82,
                                "steps": ["检查网络连通性 ping all nodes", "查看交换机端口状态", "检
                                查网线连接",
                                "重启网络接口", "启用冗余链路"],
                                "severity": "灾难", "fix_time": "10-60分钟"},
        }
        for key, val in issues_db.items():
            if key in issue or issue in key:
                return [{"issue": key, **val}]
        return [{"issue": issue, "root_cause": f"未知问题: {issue}",
                    "confidence": random.uniform(0.3, 0.7),

```

```

        "steps": [f"收集更多信息: {issue}", "查看相关日志", "联系技术支持"],
        "severity": "未知", "fix_time": "待定"]
class SmartDiagnosisModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._issues = [
            {"time": "2026-06-23 10:22", "issue": "数据库连接超时", "status": "待诊断"},
            {"time": "2026-06-23 11:05", "issue": "服务响应缓慢", "status": "诊断完成"},
            {"time": "2026-06-23 11:30", "issue": "磁盘空间告警", "status": "已修复"},
        ]
        self._build_ui()
        self._refresh_list()
    def _build_ui(self):
        layout = QVBoxLayout(self)
        layout.setContentsMargins(16, 12, 16, 12)
        splitter = QSplitter(Qt.Orientation.Horizontal)
        left = QWidget()
        ll = QVBoxLayout(left)
        # Issue list
        self._issue_list = QListWidget()
        self._issue_list.currentRowChanged.connect(self._show_detail)
        ll.addWidget(QLabel("待诊断问题列表:"))
        ll.addWidget(self._issue_list, 1)
        # Buttons
        for buttons in [
            [("智能诊断", self._diagnose), ("一键修复", self._fix), ("加入知识库",
self._to_kb)],
            [("批量诊断", self._batch_diag), ("导出报告", self._export), ("清除历史",
self._clear)],
        ]:
            r = QHBoxLayout()
            for text, slot in buttons:
                b = QPushButton(text)
                if text == "智能诊断": b.setProperty("primary", True)
                b.clicked.connect(slot)
                r.addWidget(b)
            ll.addLayout(r)
        splitter.addWidget(left)
        right = QWidget()
        rl = QVBoxLayout(right)
        diag_box = QGroupBox("诊断结果")
        self._diag_result = QTextEdit()
        self._diag_result.setReadOnly(True)
        self._diag_result.setPlaceholderText("选择问题并点击「智能诊断」获取诊断结果...")
        diag_box.setLayout(QVBoxLayout())
        diag_box.layout().addWidget(self._diag_result)
        rl.addWidget(diag_box)
        step_box = QGroupBox("修复步骤")
        self._step_list = QListWidget()

```

```

step_box.setLayout(QVBoxLayout())
step_box.layout().addWidget(self._step_list)
progress_box = QGroupBox("修复进度")
self._fix_progress = QProgressBar()
self._fix_progress.setRange(0, 100)
self._fix_progress.setValue(0)
progress_box.setLayout(QVBoxLayout())
progress_box.layout().addWidget(self._fix_progress)
rl.addWidget(progress_box)
rl.addWidget(step_box)
splitter.addWidget(right)
layout.addWidget(splitter, 1)
def _refresh_list(self):
    self._issue_list.clear()
    for iss in self._issues:
        icon = {"待诊断": " ", "诊断完成": " ", "已修复": "✓", "修复中": "🔄"}
        item = QListWidgetItem(f"{icon.get(iss['status'], ' ')} {iss['time']} {iss['issue']}
[{iss['status']}]")
        if iss["status"] == "待诊断": item.setForeground(QColor("#ffab40"))
        elif iss["status"] == "已修复": item.setForeground(QColor("#81c784"))
        self._issue_list.addItem(item)
def _show_detail(self, idx):
    if idx < 0 or idx >= len(self._issues): return
    iss = self._issues[idx]
    self._diag_result.setText(f"问题: {iss['issue']}\n时间: {iss['time']}\n状态:
{iss['status']}\n\n点击「智能诊断」获取详细分析...")
    self._step_list.clear()
def _diagnose(self):
    idx = self._issue_list.currentRow()
    if idx < 0: show_warning(self, "提示", "请先选择一个待诊断问题"); return
    iss = self._issues[idx]
    result = DiagnosisEngine.analyze(iss["issue"])
    if not result: return
    r = result[0]
    lines = [
        f"🔍 智能诊断报告",
        f"—————",
        f"问题: {r['issue']}",
        f"根因分析: {r['root_cause']}",
        f"置信度: {r['confidence']*100:.0f}%",
        f"严重程度: {r['severity']}",
        f"预计修复时间: {r['fix_time']}",
        f"—————",
        f"修复步骤:",
    ]
    for i, step in enumerate(r["steps"], 1):
        lines.append(f" {i}. {step}")
    self._diag_result.setText("\n".join(lines))

```

```

self._step_list.clear()
for i, step in enumerate(r["steps"], 1):
    self._step_list.addItem(f"步骤{i}: {step}")

iss["status"] = "诊断完成"
self._refresh_list()

def _fix(self):
    idx = self._issue_list.currentRow()
    if idx < 0: show_warning(self, "提示", "请先选择一个已诊断问题"); return
    iss = self._issues[idx]
    if iss["status"] == "待诊断": show_warning(self, "提示", "请先执行智能诊断"); return
    self._fix_timer = QTimer(self)
    self._fix_timer._progress = 0
    self._fix_timer.timeout.connect(self._fix_step)
    iss["status"] = "修复中"
    self._refresh_list()
    self._fix_timer.start(200)
    self._diag_result.append("\n► 正在执行修复...")

def _fix_step(self):
    self._fix_timer._progress += 5
    self._fix_progress.setValue(self._fix_timer._progress)
    if self._fix_timer._progress >= 100:
        self._fix_timer.stop()
        idx = self._issue_list.currentRow()
        if 0 <= idx < len(self._issues):
            self._issues[idx]["status"] = "已修复"
            self._refresh_list()

        self._diag_result.append("✓ 修复完成！问题已解决。")
        show_success(self, "修复完成", "自动修复已成功执行")

def _to_kb(self):
    idx = self._issue_list.currentRow()
    if idx < 0: return
    show_success(self, "已加入知识库", f"问题「{self._issues[idx]['issue']}」已加入知识图谱模块")

    self._diag_result.append("\n📖 已加入知识库")

def _batch_diag(self):
    for iss in self._issues:
        if iss["status"] == "待诊断":
            result = DiagnosisEngine.analyze(iss["issue"])
            iss["status"] = "诊断完成"
    self._refresh_list()
    self._diag_result.setText(f"[{datetime.now().strftime('%H:%M:%S')}] 批量诊断完成")
    show_success(self, "批量诊断", f"已完成 {len(self._issues)} 个问题的诊断")

def _export(self):
    lines = ["智能诊断报告", f"生成时间: {datetime.now()}", ""]
    for iss in self._issues:
        lines.append(f"[{iss['status']}] [{iss['time']}] [{iss['issue']}]")
    DetailDialog("诊断报告", "\n".join(lines), self).exec()

```

```

def _clear(self):
    self._issues.clear()
    self._issue_list.clear()
    self._diag_result.clear()
    self._step_list.clear()
    self._fix_progress.setValue(0)
# ==== modules/dashboard.py (260行) ====
import random
import math
from datetime import datetime
from PyQt6.QtCore import Qt, QTimer, QRectF
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QGridLayout, QProgressBar, QTextEdit, QFrame,
)
from PyQt6.QtGui import QPainter, QColor, QPen, QFont, QLinearGradient
from core.dialogs import show_info, show_success, DetailDialog
class HealthRadar(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._metrics = {"日志采集": 0.92, "解析引擎": 0.88, "异常检测": 0.85,
                        "故障预测": 0.78, "根因定位": 0.82, "告警管理": 0.90}

        self._pulse = 0.0
        self.setMinimumSize(260, 260)
        self._timer = QTimer(self)
        self._timer.timeout.connect(self._tick)
        self._timer.start(50)
    def _tick(self) -> None:
        self._pulse += 0.02
        for k in self._metrics:
            self._metrics[k] = max(0.5, min(1.0, self._metrics[k] + random.uniform(-0.01,
0.01)))
        self.update()
    def paintEvent(self, event) -> None:
        painter = QPainter(self)
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)
        w, h = self.width(), self.height()
        cx, cy = w // 2, h // 2
        r = min(w, h) * 0.35
        painter.setPen(QPen(QColor("#2a4070"), 1))
        for i in range(3):
            painter.drawEllipse(QRectF(cx - r * (i+1)/3, cy - r * (i+1)/3, r * 2*(i+1)/3, r *
2*(i+1)/3))
        angles = list(self._metrics.values())
        names = list(self._metrics.keys())
        n = len(angles)
        points = []
        for i in range(n):
            a = math.pi * 2 * i / n - math.pi / 2
            val = angles[i]
            px = cx + r * val * math.cos(a)
            py = cy + r * val * math.sin(a)
            points.append((px, py))

```

```

painter.setPen(QPen(QColor(79, 195, 247, 80), 1))
for i in range(n):
    painter.drawLine(cx, cy, int(points[i][0]), int(points[i][1]))
alpha = int(60 + 30 * math.sin(self._pulse))
painter.setBrush(QColor(79, 195, 247, alpha))
painter.setPen(QPen(QColor("#4fc3f7"), 2))
for i in range(n):
    p1 = points[i]
    p2 = points[(i + 1) % n]
    painter.drawLine(int(p1[0]), int(p1[1]), int(p2[0]), int(p2[1]))
for px, py in points:
    painter.drawEllipse(int(px) - 4, int(py) - 4, 8, 8)
painter.setPen(QColor("#8fa8c8"))
painter.setFont(QFont("Microsoft YaHei UI", 8))
for i, name in enumerate(names):
    a = math.pi * 2 * i / n - math.pi / 2
    lx = cx + r * 1.15 * math.cos(a)
    ly = cy + r * 1.15 * math.sin(a)
    painter.drawText(QRectF(int(lx) - 40, int(ly) - 8, 80, 16),
Qt.AlignmentFlag.AlignCenter, name)
    painter.setPen(QPen(QColor(79, 195, 247, 30)))
    pulse_angle = self._pulse % (math.pi * 2)
    painter.drawLine(cx, cy, int(cx + r * math.cos(pulse_angle)), int(cy + r *
math.sin(pulse_angle)))
    painter.end()
class DashboardModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._build_ui()
        self._timer = QTimer(self)
        self._timer.timeout.connect(self._update_metrics)
        self._timer.start(3000)
    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(16, 12, 16, 12)
        top = QHBoxLayout()
        self._radar = HealthRadar()
        top.addWidget(self._radar)
        kpi_grid = QGridLayout()
        kpis = [
            ("日志处理量", "125,680 条/小时", 85), ("异常检出率", "96.2%", 96),
            ("故障预测准确率", "92.5%", 93), ("告警收敛比", "8:1", 88),
            ("MTTR", "12.5 分钟", 75), ("系统可用率", "99.92%", 99),
            ("在线节点数", "128/128", 100), ("根因定位准确率", "87.3%", 87),
        ]
        for i, (name, value, pct) in enumerate(kpis):
            card = QFrame()
            card.setStyleSheet("QFrame { background: #121e3a; border: 1px solid #2a4070;
border-radius: 8px; }")
            cl = QVBoxLayout(card)
            cl.setContentsMargins(12, 10, 12, 10)
            lbl_name = QLabel(name)

```

```

11px;")
        lbl_name.setStyleSheet("color: #8fa8c8; background: transparent; font-size:
        lbl_val = QLabel(value)
        lbl_val.setFont(QFont("Microsoft YaHei UI", 14, QFont.Weight.Bold))
        lbl_val.setStyleSheet("color: #4fc3f7; background: transparent;")
        cl.addWidget(lbl_name)
        cl.addWidget(lbl_val)
        bar = QProgressBar()
        bar.setRange(0, 100)
        bar.setValue(pct)
        bar.setTextVisible(False)
        bar.setFixedHeight(6)
        cl.addWidget(bar)
        kpi_grid.addWidget(card, i // 4, i % 4)
    top.addLayout(kpi_grid, 2)
    layout.addLayout(top)
    mid = QHBoxLayout()
    alert_box = QGroupBox("系统运行概览")
    alert_lay = QVBoxLayout(alert_box)
    self._overview = QTextEdit()
    self._overview.setReadOnly(True)
    self._overview.setMaximumHeight(120)
    self._overview.setText(
        "系统状态: 正常运行 ✓\n"
        "日志采集速率: 1,250 条/秒\n"
        "最新异常: 数据库连接池告警 (P1)\n"
        "上次故障: 12分钟前 (已自动恢复)"
    )
    alert_lay.addWidget(self._overview)
    mid.addWidget(alert_box, 2)
    status_frame = QFrame()
    status_frame.setFixedWidth(240)
    sl = QVBoxLayout(status_frame)
    self._lbl_mode = QLabel("运行模式: 自动巡检")
    self._lbl_health = QLabel("综合健康度: 92%")
    self._lbl_alerts = QLabel("当前告警: 3 条")
    for lb in [self._lbl_mode, self._lbl_health, self._lbl_alerts]:
        lb.setStyleSheet("color: #4fc3f7; background: transparent; font-size: 12px;")
        sl.addWidget(lb)
    sl.addStretch()
    mid.addWidget(status_frame)
    layout.addLayout(mid)
    for buttons in [
        [("刷新态势", self._refresh_view), ("健康检查", self._health_check),
         ("性能快照", self._snapshot), ("导出状态", self._export_status)],
        [("系统自检", self._self_test), ("组件状态", self._component_status),
         ("流量监控", self._traffic_monitor), ("生成综合报告", self._gen_report)],
    ]:
        r = QHBoxLayout()

```

```

        for text, slot in buttons:
            b = QPushButton(text)
            if text == "刷新态势":
                b.setProperty("primary", True)
                b.clicked.connect(slot)
                r.addWidget(b)
            layout.addLayout(r)
def _refresh_view(self) -> None:
    self._overview.setText(
        f"系统状态: 正常运行 ✓\n"
        f"日志采集速率: {random.randint(800, 2000)} 条/秒\n"
        f"最新异常: {'数据库连接池告警' if random.random() > 0.5 else '无新异常'}\n"
        f"综合健康度: {random.uniform(88, 99.5):.1f}%"
    )
    show_success(self, "已刷新", "态势感知面板已更新")
def _health_check(self) -> None:
    checks = [("日志采集", True), ("解析引擎", True), ("异常检测", True),
              ("故障预测", True), ("告警中心", True), ("数据库连接", True)]
    lines = ["系统健康检查结果:", ""]
    for name, ok in checks:
        icon = "✓" if ok and random.random() > 0.1 else "△" if random.random() > 0.05
    else "✗"
        lines.append(f" {icon} {name}")
    DetailDialog("健康检查", "\n".join(lines), self).exec()
def _snapshot(self) -> None:
    lines = [
        "系统性能快照",
        f"时间: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
        f"CPU: {random.uniform(20, 80):.1f}%",
        f"内存: {random.uniform(40, 85):.1f}%",
        f"磁盘IO: {random.uniform(10, 200):.1f} MB/s",
        f"网络IO: {random.uniform(50, 500):.1f} Mbps",
        f"日志队列: {random.randint(0, 500)} 条积压",
    ]
    DetailDialog("性能快照", "\n".join(lines), self).exec()
def _export_status(self) -> None:
    lines = [
        "系统状态报告",
        f"时间: {datetime.now()}",
        "运行模式: 自动巡检",
        f"综合健康度: {random.uniform(85, 99):.1f}%",
        "组件状态: 全部正常",
    ]
    DetailDialog("状态导出", "\n".join(lines), self).exec()
def _self_test(self) -> None:

```

```

lines = ["系统自检结果:", "",
        "✔ 核心引擎自检通过",
        "✔ 日志采集通路正常",
        "✔ 告警通道连接正常",
        "✔ 数据库连接正常",
        "⚠ 备份节点延迟偏高 (120ms)"]
DetailDialog("系统自检", "\n".join(lines), self).exec()
def _component_status(self) -> None:
    lines = ["组件运行状态:", "",
            "  日志采集器 - 运行中 (正常运行30d)",
            "  日志解析器 - 运行中 (吞吐量850条/s)",
            "  异常检测 - 运行中 (模型版本v2.3)",
            "  故障预测 - 运行中 (预测窗口15min)",
            "  根因分析 - 运行中 (知识库版本v5)",
            "● 告警通知 - 异常 (微信通知通道中断)"]
    DetailDialog("组件状态", "\n".join(lines), self).exec()
def _traffic_monitor(self) -> None:
    lines = ["流量监控面板:", "",
            f"入站日志: {random.randint(500, 2000)} 条/秒",
            f"解析输出: {random.randint(450, 1900)} 条/秒",
            f"异常事件: {random.randint(0, 10)} 次/分",
            f"告警流量: {random.randint(0, 5)} 条/分",
            f"带宽占用: {random.uniform(10, 60):.1f} Mbps"]
    DetailDialog("流量监控", "\n".join(lines), self).exec()
def _gen_report(self) -> None:
    lines = [
        "综合态势报告",
        f"时间: {datetime.now()}",
        "",
        "【系统概况】",
        "运行时长: 15天8小时",
        "总日志处理: 18,250,000 条",
        "异常检出: 1,285 次",
        "故障预测: 47 次 (准确率92.5%)",
        "MTTR: 12.5 分钟",
        "",
        "【健康评分】",
        "日志采集: 98%",
        "解析引擎: 95%",
        "异常检测: 92%",
    ]

```

```

        "根因定位: 88%",
        "综合评分: 93%",
    ]
    DetailDialog("综合报告", "\n".join(lines), self).exec()
def _update_metrics(self) -> None:
    self._lbl_health.setText(f"综合健康度: {random.uniform(88, 98):.1f}%")
    self._lbl_alerts.setText(f"当前告警: {random.randint(0, 8)} 条")
# ==== modules/report_center.py (90行) ====
from datetime import datetime
from PyQt6.QtCore import Qt
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QFormLayout, QComboBox, QTextEdit, QSplitter,
    QListWidget, QListWidgetItem,
)
from PyQt6.QtGui import QColor
from core.dialogs import show_info, show_success, DetailDialog
class ReportCenterModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._reports = []
        self._build_ui()
    def _build_ui(self):
        layout = QVBoxLayout(self); layout.setContentsMargins(16,12,16,12)
        splitter = QSplitter(Qt.Orientation.Horizontal)
        left = QWidget(); ll = QVBoxLayout(left); ll.setContentsMargins(0,0,0,0)
        # Report generation buttons
        gen_row = QHBoxLayout()
        for t, s in [(_("日报", lambda:self._gen("日报")),(_("周报", lambda:self._gen("周报
        "))),
                    (_("月报", lambda:self._gen("月报")),(_("故障分析",
        lambda:self._gen("故障分析报告"))),
                    (_("趋势分析", lambda:self._gen("趋势分析报告"))):
            b = QPushButton(t); b.clicked.connect(s)
            if "日报" in t or "周报" in t: b.setProperty("primary", True)
            gen_row.addWidget(b)
        gen_row.addStretch()
        ll.addLayout(gen_row)
        # Report history
        self._report_list = QListWidget()
        self._report_list.setStyleSheet("QListWidget{background:#121e3a;border:1px solid
        #2a4070;border-radius:8px;}
        "QListWidget::item{padding:10px 14px;border-bottom:1px solid #1a3060;}
        "QListWidget::item:hover{background:#1a3060;}")
        ll.addWidget(self._report_list, 1)
        for buttons in [
            [(_("历史报告", self._history), (_("定时生成", self._schedule), (_("对比分析",
        self._compare), (_("批量归档", self._archive)]),
        ]:

```

```

        r = QHBoxLayout()
        for t, s in buttons:
            b = QPushButton(t); b.clicked.connect(s); r.addWidget(b)
        ll.addLayout(r)
        splitter.addWidget(left)
        right = QWidget(); rl = QVBoxLayout(right); rl.setContentsMargins(0,0,0,0)
        config_box = QGroupBox("报告配置")
        form = QFormLayout(config_box)
        self._tmpl_combo = QComboBox(); self._tmpl_combo.addItems(["日报", "周报", "月报", "故障分析报告", "趋势分析报告"])
        form.addRow("模板", self._tmpl_combo)
        self._fmt_combo = QComboBox();
        self._fmt_combo.addItems(["PDF", "HTML", "Markdown", "Excel", "Word"])
        form.addRow("格式", self._fmt_combo)
        rl.addWidget(config_box)
        preview_box = QGroupBox("预览")
        self._preview = QTextEdit()
        self._preview.setReadOnly(True); self._preview.setPlaceholderText("生成报告后在此预览...")
        preview_box.setLayout(QVBoxLayout());
        preview_box.layout().addWidget(self._preview)
        rl.addWidget(preview_box)
        splitter.addWidget(right); layout.addWidget(splitter, 1)
        def _gen(self, template):
            self._tmpl_combo.setCurrentText(template)
            content = f"{{template}} - 智能分析报告", f"时间: {datetime.now().strftime('%Y-%m-%d %H:%M')}", "",
            "【数据概览】", f"日志总量: {__import__('random').randint(50000,500000).:,} 条",
            f" 异常事件: {__import__('random').randint(10,200)} 次",
            f" 告警总数: {__import__('random').randint(5,80)} 条"]
            self._preview.setText("\n".join(content))
            name = f"{{template}}_{{datetime.now().strftime('%Y%m%d_%H%M')}})"
            self._reports.insert(0, {"名称": name, "类型": template, "时间": datetime.now().strftime('%Y-%m-%d %H:%M')})
            item = QListWidgetItem(f"📄 {name} [{{template}}]")
            self._report_list.insertItem(0, item)
            show_success(self, "生成", f"{{template}}已生成")
        def _history(self):
            lines = ["历史报告:"] + [f"  {{r.get('时间', '')}} {{r.get('类型', '')}} - {{r.get('名称', '')}}" for r in self._reports[:20]]
            if not self._reports: lines.append(" (无)")
            DetailDialog("历史", "\n".join(lines), self).exec()
        def _schedule(self):
            show_info(self, "定时", "日报 08:00 | 周报 周一09:00 | 月报 1日10:00")
        def _compare(self):

```

```

        lines = ["对比: 本周vs上周", "日志量: +12.5%↑", "异常: -8.3%↓", "告警: -15.2%↓",
"MTTR: -22%↑ 改善"]
        DetailDialog("对比","\n".join(lines),self).exec()
    def _archive(self):
        show_success(self,"归档",f"已归档{len(self._reports)}份报告")
# ==== modules/knowledge_base.py (115行) ====
from datetime import datetime
from PyQt6.QtCore import Qt
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QSplitter, QFrame, QTextEdit, QLineEdit,
    QComboBox, QSlider, QFormLayout, QProgressBar,
)
from PyQt6.QtGui import QColor, QFont
from core.dialogs import show_info, show_success, DetailDialog
class KnowledgeBaseModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._build_ui()
    def _build_ui(self):
        layout = QVBoxLayout(self); layout.setContentsMargins(16,12,16,12)
        title_row = QHBoxLayout()
        title_row.addWidget(QLabel("🧠 知识工坊", styleSheet="font-size:16px;font-
weight:bold;color:#4fc3f7;"))
        title_row.addStretch()
        layout.addLayout(title_row)
        # Search & match area
        search_panel = QFrame();
search_panel.setStyleSheet("QFrame{background:#121e3a;border:1px solid #2a4070;border-
radius:10px;}")
        sp = QVBoxLayout(search_panel); sp.setContentsMargins(16,14,16,14)
        sp.addWidget(QLabel("智能检索", styleSheet="color:#4fc3f7;font-weight:bold;font-
size:13px;"))
        sr = QHBoxLayout()
        self._search_input = QLineEdit(); self._search_input.setPlaceholderText("输入故障症状
描述...")
        sr.addWidget(self._search_input, 1)
        self._tag_cb = QComboBox(); self._tag_cb.addItem("全部分类","数据库","JVM","磁
盘","网络","缓存","服务"])
        sr.addWidget(self._tag_cb)
        sp.addLayout(sr)
        match_row = QHBoxLayout()
        self._match_progress = QProgressBar(); self._match_progress.setRange(0,100);
self._match_progress.setValue(0)
        self._match_progress.setFormat("匹配度 %p%")
        match_row.addWidget(self._match_progress, 1)
        sp.addLayout(match_row)
        layout.addWidget(search_panel)
        # Case cards

```

```

cases_row = QHBoxLayout()
cases = [
    ("数据库连接超时", "连接池耗尽导致", "★ 95%", "#ff5252"),
    ("内存溢出OOM", "堆内存不足+泄漏", "★ 92%", "#ffab40"),
    ("磁盘写满", "日志未轮转归档", "★ 98%", "#81c784"),
    ("缓存雪崩", "大量缓存同时过期", "★ 90%", "#4fc3f7"),
]
for title, desc, conf, color in cases:
    card = QFrame(); card.setStyleSheet(f"QFrame{{background:{color}15;border:2px
solid {color};border-radius:10px;}}")
    cl = QVBoxLayout(card); cl.setContentsMargins(12,10,12,10)
    cl.addWidget(QLabel(title,styleSheet="color:#e8edf5;font-weight:bold;font-
size:12px;"))
    cl.addWidget(QLabel(desc,styleSheet="color:#8fa8c8;font-size:10px;"))
    cl.addWidget(QLabel(conf,styleSheet="color:#ffd740;font-size:14px;"))
    cases_row.addWidget(card, 1)
layout.addLayout(cases_row)
# Knowledge tools
tools_panel = QFrame();
tools_panel.setStyleSheet("QFrame{background:#121e3a;border:1px solid #2a4070;border-
radius:10px;}")
tp = QHBoxLayout(tools_panel)
for t, s, c in [( 相似度匹配", self._match, True), ( 案例学习", self._learn,
False),
                ("⛶ 新建案例", self._add, False), ("📤 导出库", self._export, False),
                ("🔄 重新索引", self._reindex, False)]:
    b = QPushButton(t); b.clicked.connect(s)
    if c: b.setProperty("primary", True)
    tp.addWidget(b)
layout.addWidget(tools_panel)
# Detail
detail_box = QGroupBox("案例详情预览")
self._detail = QTextEdit()
self._detail.setReadOnly(True); self._detail.setPlaceholderText("点击「案例学习」获取推
荐案例...")
detail_box.setLayout(QVBoxLayout()); detail_box.layout().addWidget(self._detail)
layout.addWidget(detail_box, 1)
self._log = QTextEdit(); self._log.setReadOnly(True); self._log.setMaximumHeight(60)
self._log.setText("[就绪] 知识库加载完成 6个案例")
layout.addWidget(self._log)
def _match(self):
    kw = self._search_input.text().strip() or "数据库连接失败"
    sim = len(kw) * 3
    self._match_progress.setValue(min(98, sim))
    self._detail.setText(f"

```

```

def _learn(self):
    self._detail.setText("📖 推荐学习\n\n1. 数据库连接超时\n    根因: 连接池耗尽\n
方案: 优化连接池配置\n\n2. 内存溢出OOM\n    根因: 堆内存不足\n    方案: 调整JVM参
数\n\n3. 缓存雪崩\n    根因: 同时过期\n    方案: 过期时间加随机偏移")
    show_info(self,"学习","推荐3个案例供学习")
def _add(self):
    show_success(self,"新建","案例已添加, 等待审核")
    self._log.append("[新增] 案例已提交审核")
def _export(self):
    lines = ["知识库导出", "    条目:    6个", "    格式:    JSON", "    f"时间:
{datetime.now().strftime('%Y-%m-%d')}"]
    DetailDialog("导出","\n".join(lines),self).exec()
def _reindex(self):
    self._match_progress.setValue(0)
    self._match_progress.setStyleSheet("")
    from PyQt6.QtCore import QTimer
    self._idx_timer = QTimer(self)
    self._idx_timer._v = 0
    def step():
        self._idx_timer._v += 5
        self._match_progress.setValue(self._idx_timer._v)
        if self._idx_timer._v >= 100:
            self._idx_timer.stop()
            self._log.append("[索引] 重建完成 6/6 条")
            show_success(self,"索引","知识库索引重建完成")
    self._idx_timer.timeout.connect(step)
    self._idx_timer.start(50)
# ==== modules/sla_monitor.py (125行) ====
import random
from datetime import datetime
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QTextEdit, QSplitter, QProgressBar, QFrame,
    QListWidget, QListWidgetItem,
)
from PyQt6.QtGui import QColor, QPainter, QPen, QFont
from core.dialogs import show_info, show_success, DetailDialog
class SLABurnChart(QWidget):
    def __init__(self, parent=None):
        super().__init__()
        self.setMinimumHeight(90)
        self._data = [random.uniform(0.85,1.0) for _ in range(30)]
    def paintEvent(self, e):
        painter = QPainter(self)
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)
        w,h = self.width(),self.height()
        painter.fillRect(self.rect(), QColor("#121e3a"))
        m=25; pw,ph=w-m*2,h-30
        painter.setPen(QPen(QColor("#2a4070"),1))

```

```

for i in range(4): painter.drawLine(m,int(m+ph*i/3),w-m,int(m+ph*i/3))
painter.setPen(QPen(QColor("#ff5252"),1,Qt.PenStyle.DashLine))
sy=m+ph*(1-0.95); painter.drawLine(m,int(sy),w-m,int(sy))
painter.setPen(QColor("#ff5252")); painter.setFont(QFont("Microsoft YaHei",7))
painter.drawText(w-m+2,int(sy)-2,"SLO 95%")
painter.setPen(QPen(QColor("#4fc3f7"),2))
for i in range(1,len(self._data)):
    x1=m+pw*(i-1)/(len(self._data)-1); y1=m+ph*(1-self._data[i-1])
    x2=m+pw*i/(len(self._data)-1); y2=m+ph*(1-self._data[i])
    c="#ff5252" if self._data[i]<0.95 else "#4fc3f7"
    painter.setPen(QPen(QColor(c),2))
    painter.drawLine(int(x1),int(y1),int(x2),int(y2))
painter.end()
class SLAMonitorModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._slos = [
            {"name":"API可用性","slo":99.9,"cur":99.87,"budget":72.3,"status":"警告"},
            {"name":"日志处理延迟","slo":99.5,"cur":99.62,"budget":88.5,"status":"正常"},
            {"name":"异常检测召回率","slo":95.0,"cur":96.8,"budget":91.2,"status":"正常"},
            {"name":"系统可用率","slo":99.99,"cur":99.97,"budget":45.8,"status":"危险"},
            {"name":"故障预测准确率","slo":90.0,"cur":92.5,"budget":95.0,"status":"正常"},
            {"name":"告警处理时效","slo":98.0,"cur":97.2,"budget":62.1,"status":"警告"},
        ]
        self._build_ui()
        self._timer = QTimer(self); self._timer.timeout.connect(self._tick);
self._timer.start(5000)
    def _build_ui(self):
        layout = QVBoxLayout(self); layout.setContentsMargins(16,12,16,12)
        self._chart = SLABurnChart(); layout.addWidget(self._chart)
        # SLO cards grid
        grid = QHBoxLayout()
        for slo in self._slos[:3]:
            card = self._card(slo); grid.addWidget(card)
        layout.addLayout(grid)
        grid2 = QHBoxLayout()
        for slo in self._slos[3:]:
            card = self._card(slo); grid2.addWidget(card)
        layout.addLayout(grid2)
        # List
        self._slo_list = QListWidget()
        self._slo_list.setStyleSheet("QListWidget{background:#121e3a;border:1px solid
#2a4070;border-radius:8px;}")
        "QListWidget::item{padding:8px 12px;border-bottom:1px solid #1a3060;}")
        self._refresh_list()
        layout.addWidget(self._slo_list, 1)
        for buttons in [
            [("刷新SLO", self._refresh), ("新建SLO", self._new), ("导出合规报告",
self._export), ("查看详情", self._detail)]
        ]:
            r = QHBoxLayout()

```

```

        for t, s in buttons:
            b = QPushButton(t); b.clicked.connect(s)
            if "刷新" in t: b.setProperty("primary", True)
            r.addWidget(b)
        layout.addLayout(r)
    def _card(self, slo):
        card = QFrame(); card.setStyleSheet("QFrame{background:#121e3a;border:1px solid
#2a4070;border-radius:10px;}")
        cl = QVBoxLayout(card); cl.setContentsMargins(14,12,14,12)
        name = QLabel(slo["name"]); name.setStyleSheet("color:#8fa8c8;font-size:11px;")
        colors = {"正常":"#4fc3f7","警告":"#ffab40","危险":"#ff5252"}
        val = QProgressBar()
        val.setStyleSheet(f"color:{colors.get(slo['status'], '#8fa8c8')};font-size:20px;font-weight:bold;")
        bar = QProgressBar(); bar.setRange(0,100); bar.setValue(int(slo['cur']));
        bar.setTextVisible(False); bar.setFixedHeight(6)
        meta = QLabel(f"SLO:{slo['slo']}% 预算:{slo['budget']}%")
        meta.setStyleSheet(f"color:{colors.get(slo['status'], '#8fa8c8')};font-size:9px;")
        cl.addWidget(name); cl.addWidget(val); cl.addWidget(bar); cl.addWidget(meta)
        return card
    def _refresh_list(self):
        self._slo_list.clear()
        for slo in self._slos:
            icon = {"正常": " ", "警告": " ", "危险": "●"}
            item = QListWidgetItem(f" {icon.get(slo['status'], ' ')} {slo['name']}: {slo['cur']}%
(SLO:{slo['slo']}%) 预算:{slo['budget']}%")
            item.setForeground(QColor({"正常":"#4fc3f7","警告":"#ffab40","危险":
"#ff5252"}[slo['status']]))
            self._slo_list.addItem(item)
    def _refresh(self):
        for slo in self._slos:
            slo["cur"] = round(min(100,max(85,slo["cur"])+random.uniform(-0.3,0.2))),2)
            slo["budget"] = max(0,slo["budget"]-random.uniform(0,2))
            slo["status"] = "危险" if slo["budget"]<30 else ("警告" if slo["budget"]<70 else "正
常")
        self._refresh_list()
        show_success(self, "刷新", "SLO已更新")
    def _new(self): show_info(self, "新建", "指标名称:\nSLO目标(%) \n时间窗口: 30天")
    def _export(self):
        lines = ["SLA合规报告"]+[f"{s['name']}: {s['cur']}% (SLO:{s['slo']}%) [{s['status']}]" for s in
self._slos]
        DetailDialog("报告", "\n".join(lines), self).exec()
    def _detail(self):
        cur = self._slo_list.currentRow()
        if cur<0: return
        s = self._slos[cur]
        show_info(self, s["name"], f"当前值: {s['cur']}%\nSLO目标: {s['slo']}%\n错误预算:
{s['budget']}%\n状态: {s['status']}")
    def _tick(self):

```

```

        s = random.choice(self._slos)
        s["cur"] = round(min(100,max(85,s["cur"])+random.uniform(-0.1,0.1))),2)
        s["budget"] = max(0,s["budget"]-random.uniform(0,0.5))
        s["status"] = "危险" if s["budget"]<30 else ("警告" if s["budget"]<70 else "正常")

# ==== modules/model_train.py (140行) ====
import random, math
from datetime import datetime
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QFormLayout, QComboBox, QTextEdit, QSplitter,
    QListWidget, QListWidgetItem, QProgressBar, QSpinBox, QDoubleSpinBox,
)
from PyQt6.QtGui import QColor
from core.dialogs import show_info, show_success, show_warning, DetailDialog
MODELS = {
    "AnomalyDetect-v1": {"类型": "异常检测", "算法": "Isolation Forest", "准确率": "85%", "状态": "已部署"},
    "AnomalyDetect-v2": {"类型": "异常检测", "算法": "XGBoost", "准确率": "91%", "状态": "已部署"},
    "FaultPredict-LSTM": {"类型": "故障预测", "算法": "LSTM", "准确率": "88%", "状态": "训练中"},
    "FaultPredict-ARIMA": {"类型": "故障预测", "算法": "ARIMA", "准确率": "82%", "状态": "已部署"},
    "RootCause-BN": {"类型": "根因定位", "算法": "贝叶斯网络", "准确率": "86%", "状态": "已部署"},
}

class ModelTrainModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._training = False
        self._build_ui()
        self._refresh_list()
    def _build_ui(self):
        layout = QVBoxLayout(self); layout.setContentsMargins(16,12,16,12)
        splitter = QSplitter(Qt.Orientation.Horizontal)
        left = QWidget(); ll = QVBoxLayout(left); ll.setContentsMargins(0,0,0,0)
        self._model_list = QListWidget()
        self._model_list.setStyleSheet("QListWidget{background:#121e3a;border:1px solid #2a4070;border-radius:8px;}")
        "QListWidget::item{padding:8px 14px;border-bottom:1px solid #1a3060;}"
        "QListWidget::item: hover{background:#1a3060;}"
        ll.addWidget(self._model_list, 1)
        for buttons in [
            ("开始训练", self._start_train), ("停止", self._stop_train), ("模型评估", self._eval),
            ("部署", self._deploy)],
            [("超参调优", self._tune), ("版本对比", self._compare), ("导出", self._export), ("训练历史", self._history)],

```

```

]:
    r = QHBoxLayout()
    for t, s in buttons:
        b = QPushButton(t); b.clicked.connect(s)
        if "开始" in t: b.setProperty("primary", True)
        r.addWidget(b)
    ll.addLayout(r)
    splitter.addWidget(left)
    right = QWidget(); rl = QVBoxLayout(right); rl.setContentsMargins(0,0,0,0)
    config_box = QGroupBox("训练配置")
    form = QFormLayout(config_box)
    self._name_cb = QComboBox(); self._name_cb.addItem(["AnomalyDetect-
v3", "FaultPredict-Transformer", "RootCause-GNN"])
    form.addRow("模型", self._name_cb)
    self._algo_cb = QComboBox();
self._algo_cb.addItem(["XGBoost", "LightGBM", "LSTM", "Transformer", "GNN"])
    form.addRow("算法", self._algo_cb)
    self._epoch_spin = QSpinBox(); self._epoch_spin.setRange(10,500);
self._epoch_spin.setValue(50)
    form.addRow("轮次", self._epoch_spin)
    rl.addWidget(config_box)
    progress_box = QGroupBox("训练进度")
    pl = QVBoxLayout(progress_box)
    self._prog_bar = QProgressBar(); self._prog_bar.setRange(0,100);
self._prog_bar.setValue(0)
    pl.addWidget(self._prog_bar)
    self._prog_label = QLabel("就绪"); self._prog_label.setStyleSheet("color:#4fc3f7;")
    pl.addWidget(self._prog_label)
    rl.addWidget(progress_box)
    metric_box = QGroupBox("评估指标")
    ml = QVBoxLayout(metric_box)
    self._lbl_prec = QLabel("精确率: --"); self._lbl_rec = QLabel("召回率: --"); self._lbl_f1 =
QLabel("F1: --")
    for lb in [self._lbl_prec, self._lbl_rec, self._lbl_f1]:
        lb.setStyleSheet("color:#4fc3f7;"); ml.addWidget(lb)
    rl.addWidget(metric_box)
    self._log = QTextEdit(); self._log.setReadOnly(True); self._log.setMaximumHeight(100)
    rl.addWidget(self._log); rl.addStretch()
    splitter.addWidget(right); layout.addWidget(splitter, 1)
    self._epochs = 0
    def _refresh_list(self):
        self._model_list.clear()
        for name, info in MODELS.items():
            icon = {"已部署": "✔", "训练中": "🔄", "训练完成": "✅"}
            item = QListWidgetItem(f" {icon.get(info['状态'], '○')} {name} [{info['类型']}]
{info['算法']} {info['准确率']}")
            if info["状态"]=="训练中": item.setForeground(QColor("#ffab40"))
            elif info["状态"]=="已部署": item.setForeground(QColor("#4fc3f7"))

```

```

        self._model_list.addItem(item)
def _start_train(self):
    if self._training: show_warning(self, "提示", "已有训练任务"); return
    self._training = True; self._epochs = self._epoch_spin.value()
    self._prog_bar.setValue(0); self._log.append(f"[训练] 开始")
{self._name_cb.currentText()})
    self._train_timer = QTimer(self); self._train_timer._step = 0
    self._train_timer.timeout.connect(self._train_step)
    self._train_timer.start(100)
def _train_step(self):
    self._train_timer._step += 1
    prog = min(100, int(self._train_timer._step / self._epochs * 100))
    self._prog_bar.setValue(prog)
    self._prog_label.setText(f"训练中... {self._train_timer._step}/{self._epochs}")
    if self._train_timer._step >= self._epochs:
        self._train_timer.stop(); self._training = False
        self._prog_label.setText("✓ 完成")
        self._lbl_prec.setText(f"精确率: {random.uniform(85,96):.1f}%")
        self._lbl_rec.setText(f"召回率: {random.uniform(82,94):.1f}%")
        self._lbl_f1.setText(f"F1: {random.uniform(0.83,0.95):.2f}")
        self._log.append(f"[完成] {self._name_cb.currentText()} 训练完成")
        MODELS[self._name_cb.currentText()] = {"类型": "自定义", "算法": self._algo_cb.currentText(),
            "准确率": f"{random.randint(85,95)}%", "状态": "训练完成"}
        self._refresh_list()
        show_success(self, "完成", "模型训练完成")
def _stop_train(self):
    if hasattr(self, '_train_timer') and self._train_timer.isActive():
        self._train_timer.stop(); self._training=False; self._prog_label.setText("已停止")
        self._log.append("[训练已停止]")
def _eval(self):
    prec=random.uniform(85,96); rec=random.uniform(82,94);
f1=2*prec*rec/(prec+rec)/100
    lines=[f"精确率: {prec:.1f}%", f"召回率: {rec:.1f}%", f"F1: {f1:.4f}"]
    DetailDialog("评估", "\n".join(lines), self).exec()
def _deploy(self):
    show_success(self, "部署", "模型已发布到生产环境")
def _tune(self):
    lines=["超参调优结果:", "Learning Rate: 0.001->0.0008", "Batch Size: 32->64", "F1提升:
+0.023"]
    DetailDialog("调优", "\n".join(lines), self).exec()
def _compare(self):
    lines=["版本对比:", "AnomalyDetect-v1: 85%", "AnomalyDetect-v2: 91% ↑6%"]
    DetailDialog("对比", "\n".join(lines), self).exec()
def _export(self):
    show_success(self, "导出", "模型文件已导出")

```

```

def _history(self):
    lines=["训练历史:"]+[f" Epoch {i}: acc={0.5+0.4*i/50:.4f}" for i in range(1,21)]
    DetailDialog("历史","\n".join(lines),self).exec()
# ===== modules/sys_monitor.py (269行) =====
import random
import math
from datetime import datetime
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QGridLayout, QProgressBar, QTextEdit, QFrame,
    QTableWidgetItem, QTableWidgetItem,
)
from PyQt6.QtGui import QPainter, QColor, QPen, QFont, QLinearGradient
from core.dialogs import show_info, show_success, show_warning, DetailDialog
class SysMonitorEngine:
    """系统监控引擎——自监控与健康诊断算法"""
    @staticmethod
    def collect_metrics() -> dict:
        return {
            "cpu": round(random.uniform(20, 90), 1),
            "memory": round(random.uniform(40, 85), 1),
            "disk_io": round(random.uniform(10, 150), 1),
            "network": round(random.uniform(50, 400), 1),
            "uptime": f"{random.randint(5, 30)}天{random.randint(0, 23)}小时",
            "processes": random.randint(80, 200),
            "threads": random.randint(300, 800),
            "connections": random.randint(100, 500),
        }
    @staticmethod
    def calc_overall_health(metrics: dict) -> tuple[int, str]:
        cpu_score = max(0, 100 - metrics["cpu"])
        mem_score = max(0, 100 - metrics["memory"])
        overall = int((cpu_score + mem_score) / 2)
        if overall >= 80:
            return overall, "健康"
        elif overall >= 60:
            return overall, "亚健康"
        return overall, "警告"
class MiniChart(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._data = [random.uniform(20, 80) for _ in range(40)]
        self.setMinimumHeight(80)
    def push(self, val: float) -> None:
        self._data.append(val)
        if len(self._data) > 40:
            self._data.pop(0)
        self.update()
    def paintEvent(self, event) -> None:
        painter = QPainter(self)

```

```

painter.setRenderHint(QPainter.RenderHint.Antialiasing)
w, h = self.width(), self.height()
painter.fillRect(self.rect(), QColor("#0b1426"))
if len(self._data) < 2:
    painter.end()
    return
margin = 4
pw, ph = w - margin * 2, h - margin * 2
painter.setPen(QPen(QColor("#4fc3f7"), 1.5))
for i in range(1, len(self._data)):
    x1 = margin + pw * (i - 1) / (len(self._data) - 1)
    y1 = margin + ph * (1 - self._data[i - 1] / 100)
    x2 = margin + pw * i / (len(self._data) - 1)
    y2 = margin + ph * (1 - self._data[i] / 100)
    painter.drawLine(int(x1), int(y1), int(x2), int(y2))
painter.end()
class SysMonitorModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._build_ui()
        self._timer = QTimer(self)
        self._timer.timeout.connect(self._refresh)
        self._timer.start(2000)
        self._refresh()
    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(16, 12, 16, 12)
        top = QHBoxLayout()
        metric_keys = [("cpu", "CPU使用率", "#4fc3f7"), ("mem", "内存使用率", "#81c784"),
        ("disk", "磁盘IO", "#ffab40"), ("net", "网络带宽", "#ce93d8")]
        for key, title, color in metric_keys:
            card = QFrame()
            card.setStyleSheet("QFrame { background: #121e3a; border: 1px solid #2a4070;
border-radius: 8px; }")
            cl = QVBoxLayout(card)
            cl.setContentsMargins(10, 8, 10, 8)
            lbl = QLabel(title)
            lbl.setStyleSheet("color: #8fa8c8; background: transparent; font-size: 11px;")
            cl.addWidget(lbl)
            chart = MiniChart()
            cl.addWidget(chart)
            val_lbl = QLabel("--")
            val_lbl.setStyleSheet(f"color: {color}; background: transparent; font-size: 16px;
font-weight: bold;")
            cl.addWidget(val_lbl)
            bar = QProgressBar()
            bar.setRange(0, 100)
            bar.setValue(0)
            bar.setTextVisible(False)
            bar.setFixedHeight(4)
            cl.addWidget(bar)
            setattr(self, f"_chart_{key}", chart)
            setattr(self, f"_val_{key}", val_lbl)

```

```

        setattr(self, f"_bar_{key}", bar)
    top.addWidget(card)
layout.addLayout(top)
mid = QHBoxLayout()
health_box = QGroupBox("系统健康状态")
hl = QVBoxLayout(health_box)
self._health_label = QLabel("综合健康度: --")
self._health_label.setFont(QFont("Microsoft YaHei UI", 14, QFont.Weight.Bold))
self._health_label.setStyleSheet("color: #4fc3f7; background: transparent;")
hl.addWidget(self._health_label)
self._health_bar = QProgressBar()
self._health_bar.setRange(0, 100)
self._health_bar.setValue(0)
self._health_bar.setFormat("健康度 %p%")
hl.addWidget(self._health_bar)
self._health_desc = QLabel("系统状态: 检查中...")
self._health_desc.setStyleSheet("color: #8fa8c8; background: transparent;")
hl.addWidget(self._health_desc)
mid.addWidget(health_box)
info_box = QGroupBox("系统信息")
il = QVBoxLayout(info_box)
self._sys_info = QTextEdit()
self._sys_info.setReadOnly(True)
self._sys_info.setMaximumHeight(100)
il.addWidget(self._sys_info)
mid.addWidget(info_box)
process_box = QGroupBox("进程/线程")
pl = QVBoxLayout(process_box)
self._lbl_proc = QLabel("进程数: --")
self._lbl_thread = QLabel("线程数: --")
self._lbl_conn = QLabel("连接数: --")
for lb in [self._lbl_proc, self._lbl_thread, self._lbl_conn]:
    lb.setStyleSheet("color: #4fc3f7; background: transparent;")
    pl.addWidget(lb)
pl.addStretch()
mid.addWidget(process_box)
layout.addLayout(mid)
for buttons in [
    [("刷新状态", self._refresh), ("健康诊断", self._diagnose),
     ("资源分析", self._resource_analysis), ("性能优化建议",
self._optimize_suggest)],
    [("进程快照", self._process_snapshot), ("网络诊断", self._network_diag),
     ("日志诊断", self._log_diag), ("自愈恢复", self._self_heal)],
]:
    r = QHBoxLayout()
    for text, slot in buttons:
        b = QPushButton(text)
        if text == "刷新状态":

```

```

        b.setProperty("primary", True)
        b.clicked.connect(slot)
        r.addWidget(b)
        layout.addLayout(r)
def _refresh(self) -> None:
    metrics = SysMonitorEngine.collect_metrics()
    self._chart_cpu.push(metrics["cpu"])
    self._chart_mem.push(metrics["memory"])
    self._chart_disk.push(metrics["disk_io"])
    self._chart_net.push(metrics["network"])
    self._val_cpu.setText(f"{metrics['cpu']:.1f}%")
    self._val_mem.setText(f"{metrics['memory']:.1f}%")
    self._val_disk.setText(f"{metrics['disk_io']:.1f} MB/s")
    self._val_net.setText(f"{metrics['network']:.1f} Mbps")
    self._bar_cpu.setValue(int(metrics["cpu"]))
    self._bar_mem.setValue(int(metrics["memory"]))
    self._bar_disk.setValue(int(min(100, metrics["disk_io"] / 2)))
    self._bar_net.setValue(int(min(100, metrics["network"] / 5)))
    score, desc = SysMonitorEngine.calc_overall_health(metrics)
    self._health_bar.setValue(score)
    self._health_label.setText(f"综合健康度: {score}%")
    self._health_desc.setText(f"系统状态: {desc}")
    color = {"健康": "#4fc3f7", "亚健康": "#ffab40", "警告": "#ff5252"}
    self._health_desc.setStyleSheet(f"color: {color.get(desc, '#8fa8c8')}; background:
transparent;")
    self._lbl_proc.setText(f"进程数: {metrics['processes']}")
    self._lbl_thread.setText(f"线程数: {metrics['threads']}")
    self._lbl_conn.setText(f"连接数: {metrics['connections']}")
    self._sys_info.setText(
        f"运行时长: {metrics['uptime']}\n"
        f"平台版本: v{random.choice(['1.2.3', '1.3.0', '2.0.1'])}\n"
        f"数据目录: /data/loginsight/\n"
        f"最近启动: {(datetime.now()).strftime('%Y-%m-%d %H:%M')}"
    )
def _diagnose(self) -> None:
    checks = [
        ("CPU负载", random.choice(["正常", "偏高", "正常", "正常"])),
        ("内存使用", random.choice(["正常", "偏高", "正常"])),
        ("磁盘空间", random.choice(["充足", "充足", "不足"])),
        ("网络连通", random.choice(["正常", "正常", "延迟"])),
        ("数据库连接", random.choice(["正常", "正常", "正常"])),
        ("日志队列", random.choice(["正常", "正常", "积压"])),
    ]
    lines = ["系统诊断结果:", ""]
    for name, status in checks:
        icon = {"正常": "✓", "偏高": "△", "不足": "●", "延迟": "△", "积压": "△"}
        lines.append(f" {icon.get(status, '?')} {name}: {status}")

```

```

        DetailDialog("系统诊断", "\n".join(lines), self).exec()
def _resource_analysis(self) -> None:
    lines = ["资源使用分析:", "",
            "CPU: 15% sys + 45% user + 40% idle",
            "内存: 6.2GB/8GB 已用 (77.5%)",
            "磁盘: 120GB/256GB 已用 (46.9%)",
            "网络: 入站 45Mbps / 出站 32Mbps",
            "",
            "热点进程: python3 (28%), java (22%), nginx (8%)"]

    DetailDialog("资源分析", "\n".join(lines), self).exec()
def _optimize_suggest(self) -> None:
    lines = ["性能优化建议:", "",
            "1. JVM堆内存建议调整为 -Xms4G -Xmx6G",
            "2. 数据库连接池建议增大至 max=150",
            "3. 日志轮转建议按小时切割",
            "4. 启用GC日志便于排查内存问题",
            "5. 考虑引入缓存层降低数据库负载"]

    DetailDialog("优化建议", "\n".join(lines), self).exec()
def _process_snapshot(self) -> None:
    lines = ["进程快照:", "",
            "PID  NAME                      CPU%  MEM%",
            "1234 python3                      28.5  22.3",
            "5678 java                          22.1  45.6",
            "9012 nginx                          8.3   4.2",
            "3456 redis-server                   5.1   6.8",
            "7890 mysql                          12.4  18.5"]

    DetailDialog("进程快照", "\n".join(lines), self).exec()
def _network_diag(self) -> None:
    lines = ["网络诊断结果:", "",
            "eth0: 192.168.1.100/24",
            "状态: 运行中",
            "MTU: 1500",
            "丢包率: 0.02%",
            "延迟: 0.35ms",
            "带宽: 1Gbps",
            "默认网关: 192.168.1.1 (响应正常)"]

    DetailDialog("网络诊断", "\n".join(lines), self).exec()
def _log_diag(self) -> None:
    lines = ["日志系统诊断:", "",
            "日志目录: /var/log/loginsight/",
            "磁盘占用: 2.3GB",
            "日志轮转: 已启用 (每天切割)",
            "写入速率: 850条/秒",

```

```

        "错误日志: 12条/小时",
        "告警: 日志队列无积压"]

    DetailDialog("日志诊断", "\n".join(lines), self).exec()
def _self_heal(self) -> None:
    actions = ["重启日志采集器", "清理临时缓存", "重置数据库连接池", "重新加载配置"]

    action = random.choice(actions)
    show_success(self, "自愈恢复", f"已执行: {action}\n系统已恢复正常运行")
    self._health_label.setText("综合健康度: 95%")
    self._health_bar.setValue(95)
    self._health_desc.setText("系统状态: 健康 ✓")

# ==== modules/visualization.py (320行) ====
import math
import random
from PyQt6.QtCore import Qt, QTimer, QPointF, QRectF
from PyQt6.QtGui import (
    QPainter, QColor, QPen, QBrush, QLinearGradient, QRadialGradient,
    QFont, QPolygonF,
)
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QSlider, QComboBox, QCheckBox,
)
from core.dialogs import show_info, DetailDialog
def _iso(x: float, y: float, z: float, cx: float, cy: float, scale: float, rot: float) -> QPointF:
    cos_r = math.cos(math.radians(rot))
    sin_r = math.sin(math.radians(rot))
    xr = x * cos_r - z * sin_r
    zr = x * sin_r + z * cos_r
    sx = cx + (xr - zr) * scale * 0.866
    sy = cy + (xr + zr) * scale * 0.5 - y * scale
    return QPointF(sx, sy)
class TopoSceneCanvas(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setMinimumHeight(400)
        self._rot_y = -30.0
        self._rot_x = 20.0
        self._zoom = 1.0
        self._phase = 0.0
        self._flow = [random.uniform(0.3, 1.0) for _ in range(8)]
        self._timer = QTimer(self)
        self._timer.timeout.connect(self._tick)
        self._timer.start(33)
    def _tick(self) -> None:
        self._phase += 0.04
        for i in range(len(self._flow)):
            self._flow[i] = 0.4 + 0.6 * abs(math.sin(self._phase + i * 0.7))
        self.update()
    def set_rotation(self, rx: float, ry: float) -> None:
        self._rot_x = rx

```

```

        self._rot_y = ry
        self.update()
def set_zoom(self, z: float) -> None:
    self._zoom = max(0.4, min(2.5, (-z - 15) / 30.0))
    self.update()
def paintEvent(self, event) -> None:
    painter = QPainter(self)
    painter.setRenderHint(QPainter.RenderHint.Antialiasing)
    w, h = self.width(), self.height()
    bg = QLinearGradient(0, 0, w, h)
    bg.setColorAt(0, QColor("#061220"))
    bg.setColorAt(0.5, QColor("#0a1e38"))
    bg.setColorAt(1, QColor("#0d2848"))
    painter.fillRect(self.rect(), bg)
    cx, cy = w * 0.48, h * 0.55
    scale = min(w, h) * 0.018 * self._zoom
    rot = self._rot_y + self._rot_x * 0.3
    for i in range(5):
        ring_r = scale * (20 + i * 8)
        alpha = 12 + i * 5
        pen = QPen(QColor(79, 195, 247, alpha))
        pen.setWidth(1)
        painter.setPen(pen)
        painter.setBrush(Qt.BrushStyle.NoBrush)
        painter.drawEllipse(QPointF(cx, cy), ring_r, ring_r * 0.45)
    ground = QPolygonF([
        _iso(-16, 0, -14, cx, cy, scale, rot),
        _iso(16, 0, -14, cx, cy, scale, rot),
        _iso(16, 0, 14, cx, cy, scale, rot),
        _iso(-16, 0, 14, cx, cy, scale, rot),
    ])
    painter.setPen(Qt.PenStyle.NoPen)
    painter.setBrush(QColor(8, 40, 70, 120))
    painter.drawPolygon(ground)
    self._draw_log_nodes(painter, cx, cy, scale, rot)
    self._draw_flow_lines(painter, cx, cy, scale, rot)
    self._draw_substation(painter, cx, cy, scale, rot, 0, -16)
    painter.setPen(QColor("#4fc3f7"))
    painter.setFont(QFont("Microsoft YaHei UI", 9))
    painter.drawText(12, h - 12, "数据拓扑三维可视化 · 实时数据流")
    glow = QRadialGradient(w * 0.5, h * 0.3, w * 0.4)
    glow.setColorAt(0, QColor(79, 195, 247, 15))
    glow.setColorAt(1, QColor(0, 0, 0, 0))
    painter.fillRect(self.rect(), glow)
    painter.end()
def _draw_log_nodes(self, painter, cx, cy, scale, rot) -> None:
    nodes = [
        ("日志采集", -10, 0, -8, "#4fc3f7"),
        ("日志解析", 0, 1, -12, "#81c784"),
        ("异常检测", -6, 0.5, 0, "#ffab40"),
        ("故障预测", 6, 0.5, 0, "#ce93d8"),
        ("根因定位", 10, 0, -8, "#ef5350"),
    ]

```

```

        ("告警中心", 0, 0, 6, "#ffd740"),
        ("知识库", -6, 0, 8, "#8d6e63"),
        ("报表中心", 6, 0, 8, "#4dd0e1"),
    ]
    for name, x, y, z, color in nodes:
        pt = _iso(x, y, z, cx, cy, scale, rot)
        r = 16 + 4 * math.sin(self._phase + hash(name) % 10)
        painter.setBrush(QColor(color))
        painter.setPen(QPen(QColor(color).lighter(120), 1))
        painter.drawEllipse(pt, r, r * 0.7)
        painter.setPen(QColor("#e8edf5"))
        painter.setFont(QFont("Microsoft YaHei UI", 7))
        painter.drawText(QRectF(pt.x() - 40, pt.y() + r + 2, 80, 14),
Qt.AlignmentFlag.AlignCenter, name)
    def _draw_flow_lines(self, painter, cx, cy, scale, rot) -> None:
        routes = [
            [(-10, 0.5, -8), (0, 1, -12), (-6, 0.5, 0)],
            [(-10, 0.5, -8), (0, 1, -12), (6, 0.5, 0)],
            [(-6, 0.5, 0), (0, 0, 6)],
            [(6, 0.5, 0), (0, 0, 6)],
            [(0, 0, 6), (-6, 0, 8)],
            [(0, 0, 6), (6, 0, 8)],
        ]
        for ri, route in enumerate(routes):
            intensity = self._flow[ri % len(self._flow)]
            alpha = int(60 + 120 * intensity)
            pen = QPen(QColor(79, 195, 247, alpha), 2)
            painter.setPen(pen)
            for i in range(len(route) - 1):
                p1 = _iso(*route[i], cx, cy, scale, rot)
                p2 = _iso(*route[i + 1], cx, cy, scale, rot)
                painter.drawLine(p1, p2)
            t = (self._phase * 0.3 + ri * 0.2) % 1.0
            idx = min(int(t * (len(route) - 1)), len(route) - 2)
            frac = t * (len(route) - 1) - idx
            p1, p2 = route[idx], route[idx + 1]
            px = p1[0] + (p2[0] - p1[0]) * frac
            py = p1[1] + (p2[1] - p1[1]) * frac
            pz = p1[2] + (p2[2] - p1[2]) * frac
            dot = _iso(px, py, pz, cx, cy, scale, rot)
            radial = QRadialGradient(dot, 8)
            radial.setColorAt(0, QColor(79, 195, 247, 255))
            radial.setColorAt(1, QColor(79, 195, 247, 0))
            painter.setPen(Qt.PenStyle.NoPen)
            painter.setBrush(QBrush(radial))
            painter.drawEllipse(dot, 8, 8)
    def _draw_substation(self, painter, cx, cy, scale, rot, x, z) -> None:
        s, h = 4, 5
        base = [
            _iso(x - s, 0, z - s, cx, cy, scale, rot),
            _iso(x + s, 0, z - s, cx, cy, scale, rot),
            _iso(x + s, 0, z + s, cx, cy, scale, rot),
            _iso(x - s, 0, z + s, cx, cy, scale, rot),

```

```

]
roof = [
    _iso(x - s, h, z - s, cx, cy, scale, rot),
    _iso(x + s, h, z - s, cx, cy, scale, rot),
    _iso(x + s, h, z + s, cx, cy, scale, rot),
    _iso(x - s, h, z + s, cx, cy, scale, rot),
]
painter.setPen(QPen(QColor("#667788"), 1))
painter.setBrush(QColor(30, 60, 90, 220))
painter.drawPolygon(QPolygonF(base))
painter.setBrush(QColor(40, 70, 100, 200))
painter.drawPolygon(QPolygonF([base[1], base[2], roof[2], roof[1]]))
painter.drawPolygon(QPolygonF([base[2], base[3], roof[3], roof[2]]))
tip = _iso(x, h + 2, z - s, cx, cy, scale, rot)
mid = _iso(x, h, z - s, cx, cy, scale, rot)
painter.setPen(QPen(QColor("#4fc3f7"), 2))
painter.drawLine(mid, tip)
label = _iso(x, h + 0.5, z, cx, cy, scale, rot)
painter.setPen(QColor("#4fc3f7"))
painter.setFont(QFont("Microsoft YaHei UI", 9, QFont.Weight.Bold))
painter.drawText(QRectF(label.x() - 50, label.y() - 10, 100, 20),
Qt.AlignmentFlag.AlignCenter, "LogInsight数据中心")
class VisualizationModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._stats = {"日志采集": 125.4, "解析吞吐": 98.2, "异常事件": 32.0, "系统处理":
215.3}
        self._build_ui()
    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(16, 12, 16, 12)
        top = QHBoxLayout()
        self._scene = TopoSceneCanvas()
        top.addWidget(self._scene, 1)
        ctrl = QGroupBox("场景控制")
        ctrl.setFixedWidth(260)
        cl = QVBoxLayout(ctrl)
        self._rx_slider = QSlider(Qt.Orientation.Horizontal)
        self._rx_slider.setRange(-90, 90)
        self._rx_slider.setValue(20)
        self._ry_slider = QSlider(Qt.Orientation.Horizontal)
        self._ry_slider.setRange(-180, 180)
        self._ry_slider.setValue(-30)
        self._zoom_slider = QSlider(Qt.Orientation.Horizontal)
        self._zoom_slider.setRange(-80, -15)
        self._zoom_slider.setValue(-45)
        cl.addWidget(QLabel("俯仰角"))
        cl.addWidget(self._rx_slider)
        cl.addWidget(QLabel("旋转角"))
        cl.addWidget(self._ry_slider)
        cl.addWidget(QLabel("缩放"))
        cl.addWidget(self._zoom_slider)

```

```

        self._rx_slider.valueChanged.connect(lambda v: self._scene.set_rotation(v,
self._ry_slider.value()))
        self._ry_slider.valueChanged.connect(lambda v:
self._scene.set_rotation(self._rx_slider.value(), v))
        self._zoom_slider.valueChanged.connect(self._scene.set_zoom)
        self._view_combo = QComboBox()
        self._view_combo.addItem("默认视角", "俯视全景", "采集层特写", "分析引擎层")
        cl.addWidget(self._view_combo)
        self._view_combo.currentIndexChanged.connect(self._switch_preset)
        top.addWidget(ctrl)
        layout.addLayout(top, 1)
        for buttons in [
            [("重置视角", self._reset_view), ("环绕巡视", self._orbit_tour),
            ("截图保存", self._save_view), ("数据统计", self._show_stats)],
            [("切换布局", self._switch_layout), ("数据叠加", self._overlay_data),
            ("导出拓扑", self._export_topo), ("全屏展示", self._fullscreen_hint)],
        ]:
            r = QHBoxLayout()
            for text, slot in buttons:
                b = QPushButton(text)
                if text == "重置视角":
                    b.setProperty("primary", True)
                    b.clicked.connect(slot)
                r.addWidget(b)
            layout.addLayout(r)

        self._info = QLabel("数据拓扑三维可视化 · 智能分析全链路展示")
        self._info.setStyleSheet("color: #8fa8c8; background: transparent;")
        layout.addWidget(self._info)
        self._orbit_timer = QTimer(self)
        self._orbit_timer.timeout.connect(self._orbit_step)
        self._orbit_angle = -30
    def _reset_view(self) -> None:
        self._rx_slider.setValue(20)
        self._ry_slider.setValue(-30)
        self._zoom_slider.setValue(-45)
        self._orbit_timer.stop()
    def _orbit_tour(self) -> None:
        if self._orbit_timer.isActive():
            self._orbit_timer.stop()
            show_info(self, "巡视", "环绕巡视已停止")
        else:
            self._orbit_timer.start(50)
            show_info(self, "巡视", "环绕巡视已启动")
    def _orbit_step(self) -> None:
        self._orbit_angle = (self._orbit_angle + 1) % 360
        self._ry_slider.setValue(self._orbit_angle - 180)
    def _save_view(self) -> None:
        pix = self._scene.grab()
        path = "拓扑可视化截图.png"
        pix.save(path)

```

```

        show_info(self, "截图", f"已保存至 {path}")
def _show_stats(self) -> None:
    lines = ["数据流统计", ""]
    for k, v in self._stats.items():
        unit = "GB/h" if "日志" in k else "MB/s"
        lines.append(f"{k}: {v} {unit}")
    DetailDialog("数据统计", "\n".join(lines), self).exec()
def _switch_preset(self) -> None:
    presets = {0: (20, -30, -45), 1: (80, 0, -55), 2: (10, -15, -25), 3: (15, 50, -20)}
    idx = self._view_combo.currentIndex()
    rx, ry, zoom = presets.get(idx, (20, -30, -45))
    self._rx_slider.setValue(rx)
    self._ry_slider.setValue(ry)
    self._zoom_slider.setValue(zoom)
def _switch_layout(self) -> None:
    if self._view_combo.currentIndex() < 3:
        self._view_combo.setCurrentIndex(self._view_combo.currentIndex() + 1)
    else:
        self._view_combo.setCurrentIndex(0)
    show_info(self, "布局", f"已切换至 {self._view_combo.currentText()}")
def _overlay_data(self) -> None:
    self._info.setText(
        f"日志 {self._stats['日志采集']:.1f} GB/h | "
        f"解析 {self._stats['解析吞吐']:.1f} MB/s | "
        f"异常 {self._stats['异常事件']:.1f} 次/h"
    )
def _export_topo(self) -> None:
    lines = ["拓扑数据导出", f"时间: 2026-06-23", "",
            "节点数: 8 (采集/解析/检测/预测/根因/告警/知识库/报表)",
            "边数: 12 (数据流链路)",
            "数据格式: GraphML"]
    DetailDialog("拓扑导出", "\n".join(lines), self).exec()
def _fullscreen_hint(self) -> None:
    show_info(self, "全屏", "请使用窗口最大化按钮进入全屏展示模式")
# ==== modules/alert_notify.py (122行) ====
import random
from datetime import datetime
from PyQt6.QtCore import Qt
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QTextEdit, QSplitter, QListWidget, QListWidgetItem,
    QLineEdit, QComboBox, QFormLayout, QCheckBox, QSpinBox,
)
from PyQt6.QtGui import QColor
from core.dialogs import show_info, show_success, show_warning, DetailDialog
class AlertNotifyModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)

```

```

self._channels = [
    {"name": "邮件通知", "type": "email", "enabled": True, "config":
"smtp.company.com:587"},
    {"name": "企业微信", "type": "wechat", "enabled": True, "config": "Webhook URL
已配置"},
    {"name": "钉钉通知", "type": "dingtalk", "enabled": False, "config": "未配置"},
    {"name": "短信告警", "type": "sms", "enabled": False, "config": "未配置"},
    {"name": "Webhook回调", "type": "webhook", "enabled": True, "config":
"https://hooks.example.com/alert"},
]
self._rules = [
    {"name": "P0灾难告警即时通知", "level": "P0", "channels": "邮件+企业微信+短
信", "enabled": True},
    {"name": "P1严重告警5分钟通知", "level": "P1", "channels": "邮件+企业微信",
"enabled": True},
    {"name": "P2一般告警邮件通知", "level": "P2", "channels": "邮件", "enabled":
True},
    {"name": "P3提示级告警汇总通知", "level": "P3", "channels": "日报汇总",
"enabled": False},
]
self._build_ui()
def _build_ui(self):
    layout = QVBoxLayout(self)
    layout.setContentsMargins(16, 12, 16, 12)
    # Channel status
    channel_row = QHBoxLayout()
    for ch in self._channels:
        card = QGroupBox(ch["name"])
        card.setFixedWidth(180)
        cl = QVBoxLayout(card)
        icon = {"email": "✉", "wechat": "👤", "dingtalk": "🔔", "sms": "📱", "webhook":
"🔗"}

        lbl_icon = QLabel(icon.get(ch["type"], "📧"))
        lbl_icon.setFont(lbl_icon.font() or QLabel().font())
        lbl_icon.setStyleSheet("font-size: 24px;")
        lbl_status = QLabel("✔ 已启用" if ch["enabled"] else "✖ 未启用")
        lbl_status.setStyleSheet(f"color: {'#81c784' if ch['enabled'] else '#ff5252'};")
        lbl_config = QLabel(ch["config"])
        lbl_config.setStyleSheet("color: #8fa8c8; font-size: 9px;")
        cl.addWidget(lbl_icon, alignment=Qt.AlignmentFlag.AlignCenter)
        cl.addWidget(lbl_status, alignment=Qt.AlignmentFlag.AlignCenter)
        cl.addWidget(lbl_config)
        channel_row.addWidget(card)
    layout.addLayout(channel_row)
    # Config form
    splitter = QSplitter(Qt.Orientation.Horizontal)
    form_widget = QWidget()
    form = QFormLayout(form_widget)
    self._channel_combo = QComboBox()

```

```

self._channel_combo.addItem([c["name"] for c in self._channels])
form.addRow("通知通道", self._channel_combo)
self._server_input = QLineEdit("smtp.company.com")
form.addRow("服务器地址", self._server_input)
self._port_spin = QSpinBox(); self._port_spin.setRange(1, 65535);
self._port_spin.setValue(587)
form.addRow("端口", self._port_spin)
self._token_input = QLineEdit("*****")
form.addRow("Token/密钥", self._token_input)
self._test_btn = QPushButton("✉ 发送测试通知")
self._test_btn.clicked.connect(self._test_notify)
form.addRow("", self._test_btn)
self._save_btn = QPushButton("💾 保存配置")
self._save_btn.setProperty("primary", True)
self._save_btn.clicked.connect(self._save_config)
form.addRow("", self._save_btn)
splitter.addWidget(form_widget)
# Rules list
rules_widget = QWidget()
rl = QVBoxLayout(rules_widget)
rl.addWidget(QLabel("通知规则:"))
self._rules_list = QListWidget()
for rule in self._rules:
    icon = "✔" if rule["enabled"] else "❌"
    item = QListWidgetItem(f"{icon} [{rule['level']}] {rule['name']} → {rule['channels']}")
    self._rules_list.addItem(item)
rl.addWidget(self._rules_list)
rule_btn_row = QHBoxLayout()
for text, slot in [("新建规则", self._new_rule), ("编辑规则", self._edit_rule),
                  ("启用/禁用", self._toggle_rule), ("删除规则", self._del_rule)]:
    b = QPushButton(text); b.clicked.connect(slot); rule_btn_row.addWidget(b)
rl.addLayout(rule_btn_row)
splitter.addWidget(rules_widget)
layout.addWidget(splitter, 1)
# Log
self._log = QTextEdit()
self._log.setReadOnly(True)
self._log.setMaximumHeight(80)
self._log.setText("[通知通道状态] 邮件: 正常 | 企业微信: 正常 | 钉钉: 未配置 | 短信: 未配置")
layout.addWidget(self._log)
def _test_notify(self):
    ch = self._channel_combo.currentText()
    show_success(self, "测试通知", f"已向 {ch} 发送测试消息\n请检查目标端是否收到")
    self._log.append(f"[{datetime.now().strftime('%H:%M:%S')}] 测试通知已发送至 {ch}")
def _save_config(self):
    self._log.append(f"[{datetime.now().strftime('%H:%M:%S')}] 配置已保存")

```

```

        show_success(self, "保存成功", "通知通道配置已保存")
def _new_rule(self):
    show_info(self, "新建通知规则", "规则名称:\n告警级别:\n通知通道:\n静默周期:")
def _edit_rule(self):
    show_info(self, "编辑规则", "编辑表单已打开")
def _toggle_rule(self):
    show_info(self, "切换状态", "规则已启用/禁用")
def _del_rule(self):
    show_info(self, "删除规则", "规则已删除")

# ==== modules/custom_dashboard.py (149行) ====
import random
from datetime import datetime
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QTextEdit, QSplitter, QGridLayout, QFrame,
    QComboBox, QProgressBar, QColorDialog,
)
from PyQt6.QtGui import QColor, QFont
from core.dialogs import show_info, show_success, show_warning, DetailDialog
class DashboardWidget(QFrame):
    def __init__(self, title: str, widget_type: str, parent=None):
        super().__init__(parent)
        self._title = title
        self._type = widget_type
        self.setStyleSheet("QFrame { background: #121e3a; border: 1px solid #2a4070; border-
radius: 8px; }")
        self.setMinimumSize(200, 150)
        lay = QVBoxLayout(self)
        lay.setContentsMargins(10, 8, 10, 8)
        lbl = QLabel(f"📊 {title}")
        lbl.setStyleSheet("color: #4fc3f7; font-weight: bold;")
        lay.addWidget(lbl)
        if widget_type == "metric":
            val = QLabel(f"{random.randint(1000,99999)}")
            val.setFont(QFont("Microsoft YaHei", 28, QFont.Weight.Bold))
            val.setStyleSheet("color: #81c784;")
            lay.addWidget(val, alignment=Qt.AlignmentFlag.AlignCenter)
            unit = QLabel(random.choice(["条/秒", "次/分", "%", "ms"]))
            unit.setStyleSheet("color: #8fa8c8;")
            lay.addWidget(unit, alignment=Qt.AlignmentFlag.AlignCenter)
        elif widget_type == "progress":
            bar = QProgressBar(); bar.setRange(0, 100); bar.setValue(random.randint(30, 95))
            bar.setFormat(f"{title} %p%"); lay.addWidget(bar)
            val = QLabel(f"{random.randint(100,999)} / {random.randint(500,1000)}")
            val.setStyleSheet("color: #8fa8c8;"); lay.addWidget(val)
        elif widget_type == "status":
            for name, val in [("CPU", f"{random.uniform(20,90):.0f}%"),
                             ("内存", f"{random.uniform(40,85):.0f}%"),
                             ("磁盘", f"{random.uniform(30,95):.0f}%")]:

```

```

        row = QHBoxLayout()
        row.addWidget(QLabel(name))
        v = QLabel(val)
        vnum = float(val.rstrip('%')) if '%' in val else float(val)
        v.setStyleSheet(f"color: {'#ff5252' if vnum > 80 else '#4fc3f7'};")
        row.addWidget(v); row.addStretch(); lay.addLayout(row)
    elif widget_type == "chart":
        chart = QFrame()
        chart.setStyleSheet("QFrame { background: #0b1426; border: 1px solid #1a3060;
border-radius: 4px; min-height: 60px; }")
        lay.addWidget(chart, 1)
        note = QLabel("[趋势图区域]"); note.setStyleSheet("color: #6a8aaa; font-size:
9px;")
        lay.addWidget(note)
    lay.addStretch()
class CustomDashboardModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._dashboards = ["默认仪表盘", "运维总览", "性能监控", "业务看板"]
        self._widgets = [
            ("日志处理量", "metric"), ("系统健康度", "progress"),
            ("资源使用率", "status"), ("异常趋势", "chart"),
            ("API调用量", "metric"), ("存储使用率", "progress"),
            ("服务状态", "status"), ("响应时间", "chart"),
        ]
        self._build_ui()
    def _build_ui(self):
        layout = QVBoxLayout(self)
        layout.setContentsMargins(16, 12, 16, 12)
        # Toolbar
        toolbar = QHBoxLayout()
        toolbar.addWidget(QLabel("当前仪表盘:"))
        self._dash_combo = QComboBox()
        self._dash_combo.addItem(self._dashboards)
        toolbar.addWidget(self._dash_combo)
        for text, slot in [("✚ 新建", self._new_dash), ("💾 保存", self._save_dash),
            ("📄 加载", self._load_dash), ("🗑 删除", self._del_dash)]:
            b = QPushButton(text); b.clicked.connect(slot)
            if text == "💾 保存": b.setProperty("primary", True)
            toolbar.addWidget(b)
        toolbar.addStretch()
        layout.addLayout(toolbar)
        # Widget grid
        self._grid = QGridLayout()
        self._grid.setSpacing(10)
        self._render_grid()
        layout.addLayout(self._grid, 1)
        # Add widget bar
        add_row = QHBoxLayout()
        add_row.addWidget(QLabel("添加组件:"))

```

```

self._widget_combo = QComboBox()
self._widget_combo.addItem("指标卡", "进度条", "状态面板", "趋势图", "告警列表
"]

add_row.addWidget(self._widget_combo)
add_row.addWidget(QPushButton("+ 添加", clicked=self._add_widget))
add_row.addWidget(QPushButton("✎ 编辑", clicked=self._edit_widget))
add_row.addWidget(QPushButton("✕ 移除", clicked=self._remove_widget))
add_row.addStretch()
layout.addLayout(add_row)
# Properties
prop_row = QHBoxLayout()
for text, slot in [("主题切换", self._switch_theme), ("刷新频率设置", self._set_refresh),
                  ("导出仪表盘", self._export), ("全屏展示", self._fullscreen)]:
    b = QPushButton(text); b.clicked.connect(slot); prop_row.addWidget(b)
prop_row.addStretch()
layout.addLayout(prop_row)
def _render_grid(self):
    for i in reversed(range(self._grid.count())):
        self._grid.itemAt(i).widget().setParent(None)
    types = ["metric", "progress", "status", "chart"]
    for idx, (title, wtype) in enumerate(self._widgets):
        row, col = divmod(idx, 4)
        w = DashboardWidget(title, wtype)
        self._grid.addWidget(w, row, col)
def _new_dash(self): show_info(self, "新建仪表盘", "输入新仪表盘名称:\n_____\n\n
预设布局: 2列 | 3列 | 4列")
def _save_dash(self): show_success(self, "已保存", f"仪表盘
「{self._dash_combo.currentText()}」已保存")
def _load_dash(self): show_info(self, "加载", "选择历史仪表盘版本:\n- 2026-06-23
10:00\n- 2026-06-22 10:00\n- 2026-06-21 10:00")
def _del_dash(self):
    if len(self._dashboards) <= 1: show_warning(self, "提示", "至少保留一个仪表盘");
return
self._dashboards.remove(self._dash_combo.currentText())
self._dash_combo.clear(); self._dash_combo.addItem(self._dashboards)
show_success(self, "已删除", "仪表盘已删除")
def _add_widget(self):
    widget_types = {"指标卡": "metric", "进度条": "progress", "状态面板": "status", "趋势
图": "chart", "告警列表": "metric"}
    wt = widget_types.get(self._widget_combo.currentText(), "metric")
    self._widgets.append((f"组件{len(self._widgets)+1}", wt))
    self._render_grid()
    show_success(self, "已添加", f"已添加「{self._widget_combo.currentText()}」组件")
def _edit_widget(self): show_info(self, "编辑组件", "组件标题:\n数据源:\n刷新间隔:\n尺
寸: 小/中/大")
def _remove_widget(self):
    if not self._widgets: return

```

```

        self._widgets.pop()
        self._render_grid()
        show_info(self, "已移除", "最后一个组件已移除")

    def _switch_theme(self): show_info(self, "主题切换", "浅色主题 | 深色主题 | 专业主题
| 高对比度主题")

    def _set_refresh(self): show_info(self, "刷新频率设置", "自动刷新: 开启\n频率: 10秒 | 30
秒 | 60秒 | 300秒")
    def _export(self):
        lines = ["自定义仪表盘导出", f"名称: {self._dash_combo.currentText()}", f"组件数:
{len(self._widgets)}"]
        DetailDialog("导出", "\n".join(lines), self).exec()

    def _fullscreen(self): show_info(self, "全屏", "按 Esc 退出全屏模式")
# ==== modules/config_center.py (141行) ====
import json, random
from datetime import datetime
from PyQt6.QtCore import Qt
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QTextEdit, QSplitter, QTreeWidget, QTreeWidgetItem,
    QLineEdit, QComboBox, QFormLayout, QSpinBox, QCheckBox,
)
from PyQt6.QtGui import QColor
from core.dialogs import show_info, show_success, show_warning, DetailDialog
class ConfigCenterModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._configs = {
            "系统设置": {
                "日志采集频率": {"val": "2秒", "type": "select", "desc": "日志采集轮询间隔"},
                "数据保留天数": {"val": "30天", "type": "int", "desc": "日志数据保留期限"},
                "最大缓存行数": {"val": "500", "type": "int", "desc": "内存中缓存的最大日志
条数"},
                "自动启动采集": {"val": "是", "type": "bool", "desc": "启动时自动开始日志
采集"},
            },
            "检测参数": {
                "异常检测阈值": {"val": "0.65", "type": "float", "desc": "异常分数阈值(0-1)"},
                "滑动窗口大小": {"val": "60", "type": "int", "desc": "趋势分析样本数量"},
                "预测模型": {"val": "ARIMA", "type": "select", "desc": "故障预测算法"},
                "告警抑制窗口": {"val": "300秒", "type": "int", "desc": "重复告警合并时间窗
口"},
            },
            "通知设置": {
                "邮件服务器": {"val": "smtp.company.com", "type": "string", "desc": "SMTP服

```

```

        "服务器地址": {"val": "192.168.1.1", "type": "string", "desc": "SMTP服务器地址"},
        "邮件端口": {"val": "587", "type": "int", "desc": "SMTP服务器端口"},
        "启用微信通知": {"val": "是", "type": "bool", "desc": "是否开启企业微信通知"},
        "告警静默时段": {"val": "23:00-07:00", "type": "string", "desc": "不发送通知的时间段"},
    },
    "日志解析": {
        "默认解析模板": {"val": "Syslog", "type": "select", "desc": "日志解析默认模板"},
        "自动解析": {"val": "是", "type": "bool", "desc": "采集后自动进行解析"},
        "错误模式提取": {"val": "开启", "type": "bool", "desc": "自动提取常见错误模式"},
    },
}

def _build_ui(self):
    layout = QVBoxLayout(self)
    layout.setContentsMargins(16, 12, 16, 12)
    splitter = QSplitter(Qt.Orientation.Horizontal)
    # Config tree
    left = QWidget()
    ll = QVBoxLayout(left)
    self._tree = QTreeWidget()
    self._tree.setHeaderLabels(["配置项", "当前值"])
    self._build_tree()
    self._tree.itemClicked.connect(self._show_config)
    ll.addWidget(QLabel("配置分类:"))
    ll.addWidget(self._tree, 1)
    splitter.addWidget(left)
    # Config detail
    right = QWidget()
    rl = QVBoxLayout(right)
    self._detail_group = QGroupBox("配置详情")
    self._detail_form = QFormLayout()
    self._key_label = QLabel("--")
    self._val_input = QLineEdit()
    self._desc_label = QLabel("")
    self._desc_label.setStyleSheet("color: #8fa8c8;")
    self._detail_form.addRow("配置项:", self._key_label)
    self._detail_form.addRow("当前值:", self._val_input)
    self._detail_form.addRow("说明:", self._desc_label)
    self._detail_group.setLayout(self._detail_form)
    rl.addWidget(self._detail_group)
    btn_row = QHBoxLayout()
    for text, slot in [( "保存修改", self._save_config), ( "恢复默认", self._reset_default),

```

```

        ("📁 导入配置", self._import), ("📁 导出配置", self._export)]:
    b = QPushButton(text); b.clicked.connect(slot)
    if text == "💾 保存修改": b.setProperty("primary", True)
    btn_row.addWidget(b)
    btn_row.addStretch()
    rl.addLayout(btn_row)
    self._log = QTextEdit()
    self._log.setReadOnly(True); self._log.setMaximumHeight(100)
    self._log.setText("[配置中心] 选择左侧配置项进行编辑")
    rl.addWidget(self._log)
    splitter.addWidget(right)
    layout.addWidget(splitter, 1)
    # Status bar
    status_row = QHBoxLayout()
    for text in [f"配置项总数: {sum(len(v) for v in self._configs.values())}",
                f"配置分类: {len(self._configs)}", "配置版本: V1.0"]:
        lbl = QLabel(text); lbl.setStyleSheet("color: #6a8aaa;"); status_row.addWidget(lbl)
    status_row.addStretch()
    layout.addLayout(status_row)
def _build_tree(self):
    self._tree.clear()
    for group, items in self._configs.items():
        g_item = QTreeWidgetItem(self._tree, [group, ""])
        for key, cfg in items.items():
            child = QTreeWidgetItem(g_item, [key, str(cfg["val"])])
            g_item.setExpanded(True)
def _show_config(self, item, col):
    if not item.parent(): return
    group = item.parent().text(0)
    key = item.text(0)
    if group in self._configs and key in self._configs[group]:
        cfg = self._configs[group][key]
        self._key_label.setText(f"{group} > {key}")
        self._val_input.setText(str(cfg["val"]))
        self._desc_label.setText(f"类型: {cfg['type']} | 说明: {cfg['desc']}")
        self._current_path = (group, key)
def _save_config(self):
    if not hasattr(self, '_current_path'): show_warning(self, "提示", "请先选择配置项");
return
    group, key = self._current_path
    if group in self._configs and key in self._configs[group]:
        self._configs[group][key]["val"] = self._val_input.text()
        self._build_tree()
        self._log.append(f"[{datetime.now().strftime('%H:%M:%S')}] ✔ 已保存: {group} > {key} = {self._val_input.text()}")
        show_success(self, "已保存", f"{key} 配置已更新")
def _reset_default(self):
    self._log.append(f"[{datetime.now().strftime('%H:%M:%S')}] 🔄 配置已恢复默认值")
    show_success(self, "已恢复", "当前配置分类已恢复为默认值")
def _import(self):

```

```
        show_info(self, "导入配置", "从文件导入配置:\n支持的格式: JSON | YAML\n建议导入前备份当前配置")
```

```
    def _export(self):
        lines = ["配置导出", f"时间: {datetime.now()}", ""]
        for group, items in self._configs.items():
            lines.append(f"[{group}]")
            for key, cfg in items.items():
                lines.append(f"    {key} = {cfg['val']}")
        DetailDialog("导出配置", "\n".join(lines), self).exec()

        self._log.append(f"[{datetime.now().strftime('%H:%M:%S')}] 📁 配置已导出")

# ==== modules/audit_archive.py (149行) ====
import random
from datetime import datetime, timedelta
from PyQt6.QtCore import Qt, QTimer
from PyQt6.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel,
    QGroupBox, QSplitter, QFrame, QSlider, QTextEdit,
    QSpinBox, QProgressBar, QCheckBox,
)
from PyQt6.QtGui import QColor, QFont, QPainter, QPen, QLinearGradient
from core.dialogs import show_info, show_success, DetailDialog
class StorageGauge(QWidget):
    def __init__(self):
        super().__init__()
        self._used = 0; self._total = 500
        self.setMinimumSize(200, 60)
    def set_used(self, v): self._used = v; self.update()
    def paintEvent(self, e):
        painter = QPainter(self)
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)
        w, h = self.width(), self.height()
        if w < 20 or h < 10: painter.end(); return
        bar_h = 24
        painter.fillRect(self.rect(), QColor("#0b1426"))
        painter.setPen(Qt.PenStyle.NoPen)
        painter.setBrush(QColor("#1a3060"))
        painter.drawRoundedRect(0, (h-bar_h)//2, w, bar_h, 12, 12)
        ratio = min(1.0, self._used/max(self._total,1))
        color = QColor("#ff5252") if ratio > 0.85 else QColor("#ffab40") if ratio > 0.6 else
        QColor("#4fc3f7")
        painter.setBrush(color)
        painter.drawRoundedRect(2, (h-bar_h)//2, int((w-4)*ratio), bar_h, 10, 10)
        painter.setPen(QColor("#e8edf5"))
        painter.setFont(QFont("Microsoft YaHei", 10, QFont.Weight.Bold))
        painter.drawText(0, (h-bar_h)//2, w, bar_h, Qt.AlignmentFlag.AlignCenter,
            f"{self._used:.0f}GB / {self._total}GB")
        painter.end()
class AuditArchiveModule(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._archived = 0; self._saved = 0; self._compressed = True; self._encrypted = True
        self._build_ui()
```

```

def _build_ui(self):
    layout = QVBoxLayout(self); layout.setContentsMargins(16,12,16,12)
    title_row = QHBoxLayout()
    title_row.addWidget(QLabel("📁 归档管理台", styleSheet="font-size:16px;font-
weight:bold;color:#4fc3f7;"))
    title_row.addStretch()
    layout.addLayout(title_row)
    # Storage gauge
    self._storage = StorageGauge()
    layout.addWidget(self._storage)
    # Stats cards
    stat_row = QHBoxLayout()
    for label, init, color in [("已归档", "0次", "#4fc3f7"), ("存储节省", "0%", "#81c784"),
                                ("压缩率", "--", "#ffab40"), ("保留天数", "180", "#ce93d8")]:
        card = QFrame(); card.setStyleSheet("QFrame{background:#121e3a;border:1px
solid #2a4070;border-radius:8px;}")
        cl = QVBoxLayout(card); cl.setContentsMargins(12,10,12,10)
        cl.addWidget(QLabel(label,styleSheet="color:#8fa8c8;font-size:10px;"))
        v = QLabel(init); v.setStyleSheet(f"color:{color};font-size:20px;font-weight:bold;")
        self.__setattr__(f"_lbl_{label[:2]}", v); cl.addWidget(v)
        stat_row.addWidget(card)
    layout.addLayout(stat_row)
    # Policy controls
    policy_panel = QFrame();
    policy_panel.setStyleSheet("QFrame{background:#121e3a;border:1px solid #2a4070;border-
radius:10px;}")
    pp = QHBoxLayout(policy_panel)
    left_p = QVBoxLayout()
    left_p.addWidget(QLabel("归档策略",styleSheet="color:#4fc3f7;font-weight:bold;"))
    left_p.addWidget(QLabel("自动执行时间:"))
    left_p.addWidget(QLabel("🕒 每天 03:00 (可配置)",styleSheet="color:#e8edf5;"))
    self._compress_cb = QCheckBox("启用压缩 (gzip)");
    self._compress_cb.setChecked(True); left_p.addWidget(self._compress_cb)
    self._encrypt_cb = QCheckBox("启用加密 (AES-256)");
    self._encrypt_cb.setChecked(True); left_p.addWidget(self._encrypt_cb)
    pp.addLayout(left_p)
    mid_p = QVBoxLayout()
    mid_p.addWidget(QLabel("保留期限:",styleSheet="color:#8fa8c8;"))
    self._ret_slider = QSlider(Qt.Orientation.Horizontal); self._ret_slider.setRange(30, 730);
    self._ret_slider.setValue(180)
    self._ret_lbl = QLabel("180天"); self._ret_slider.valueChanged.connect(lambda v:
self._ret_lbl.setText(f"{v}天"))
    mid_p.addWidget(self._ret_slider); mid_p.addWidget(self._ret_lbl)
    pp.addLayout(mid_p)
    right_p = QVBoxLayout()
    right_p.addWidget(QLabel("存储位置:",styleSheet="color:#8fa8c8;"))
    right_p.addWidget(QLabel("/data/archive/",styleSheet="color:#e8edf5;font-
family:monospace;font-size:12px;"))

```

```

right_p.addWidget(QLabel("可用空间: 342GB",styleSheet="color:#81c784;"))
pp.addLayout(right_p)
layout.addWidget(policy_panel)
# Action buttons
for buttons in [
    [("📦 立即归档", self._archive_now), ("🔄 恢复数据", self._restore),
     ("🧹 清理过期", self._cleanup), ("📊 归档报告", self._report)],
    [("🔍 审计日志", self._audit), ("⚙️ 归档设置", self._settings),
     ("📄 导出清单", self._export), ("🧪 测试归档", self._test)],
]:
    r = QHBoxLayout()
    for t, s in buttons:
        b = QPushButton(t); b.clicked.connect(s)
        if "立即" in t: b.setProperty("primary", True)
        r.addWidget(b)
    layout.addLayout(r)
self._log = QTextEdit(); self._log.setReadOnly(True); self._log.setMaximumHeight(80)
self._log.setText("[就绪] 归档引擎就绪")
layout.addWidget(self._log)
def _archive_now(self):
    self._archived += 1
    size = random.randint(500, 3000)
    self._saved += int(size * 0.68)
    self._storage.set_used(self._storage._used + size/1000)
    if hasattr(self, '_lbl_已归'): self._lbl_已归.setText(f"{self._archived}次")
    if hasattr(self, '_lbl_存储'): self._lbl_存储.setText(f"{int(self._saved/1024*100)}%")
    self._log.append(f"[[datetime.now().strftime('%H:%M:%S')]] ✓ 归档完成 {size}MB -> {int(size*0.32)}MB (压缩68%)")
    show_success(self,"归档",f"归档完成, 节省存储 {int(size*0.68)}MB")
def _restore(self):
    show_info(self,"恢复","选择恢复版本:\n① 2026-06-22 完整归档\n② 2026-06-15 增量归档\n③ 2026-06-08 完整归档")
def _cleanup(self):
    self._log.append("[清理] 已清理过期归档 3个, 释放 2.1GB")
    show_success(self,"清理","已清理3个过期归档")
def _report(self):
    lines = ["归档报告", f"总归档: {self._archived}次", f"节省存储: {self._saved/1024:.1f}GB",
            "压缩率: 68%", "加密: AES-256", "合规状态: ✓ 通过"]
    DetailDialog("报告","\n".join(lines),self).exec()
def _audit(self):
    lines = ["审计日志:", "", "2026-06-23 03:00 自动归档 完成",
            "2026-06-22 15:30 用户admin 恢复归档 ARC-20260615",
            "2026-06-22 03:00 自动归档 完成", "2026-06-21 03:00 自动归档 完成"]
    DetailDialog("审计","\n".join(lines),self).exec()

```

```
def _settings(self):
    show_info(self,"设置","归档路径: /data/archive/\n压缩格式: gzip\n加密算法: AES-
256\n线程数: 4\n限速: 100MB/s")
def _export(self):
    show_success(self,"导出","归档清单已导出")
def _test(self):
    self._log.append("[测试] 归档测试通过 读写正常 6ms")
    show_info(self,"测试","归档读写测试通过")
```